STAR RX72N - rx_ds18b20 Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Chapter 4

Topic Documentation

4.1 Internal Constants

Variables

- static const char s_tag [] = ”DS18B20”
  Logging tag for DS18B20 driver.
- static const uint8_t s_ds18b20_family_code = 0x28U
  DS18B20 family code (byte 0 of ROM).
- static const float s_temp_conversion_divisor = 16.0F
  Temperature conversion divisor (raw to Celsius).
- static const uint8_t s_ds18b20_reserved_byte_value = 0xFFU
  Reserved byte expected value in scratchpad.
- static const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U
  Invalid conversion time sentinel.

4.1.1 Detailed Description

4.1.2 Variable Documentation

4.1.2.1 s˙ds18b20˙conversion˙time˙invalid˙u32

const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U [static]

Invalid conversion time sentinel.

Returned by rx_ds18b20_get_conversion_time_ms() on error.

Definition at line 237 of file rx_ds18b20.c.

Referenced by rx_ds18b20_get_conversion_time_ms().

4.1.2.2 s˙ds18b20˙family˙code

const uint8_t s_ds18b20_family_code = 0x28U [static]

DS18B20 family code (byte 0 of ROM).

Definition at line 219 of file rx_ds18b20.c.

Referenced by internal_ds18b20_validate_config().

4.1.2.3 s˙ds18b20˙reserved˙byte˙value

const uint8_t s_ds18b20_reserved_byte_value = 0xFFU [static]

Reserved byte expected value in scratchpad.

Byte 5 of scratchpad should always be 0xFF per datasheet.

Definition at line 231 of file rx_ds18b20.c.

Referenced by internal_ds18b20_read_scratchpad_raw().

4.1.2.4 s˙tag

const char s_tag[] = "DS18B20" [static]

Logging tag for DS18B20 driver.

Definition at line 216 of file rx_ds18b20.c.

Referenced by internal_ds18b20_read_scratchpad_raw(), internal_ds18b20_select_device(), internal_ds18b20_validate
internal_ds18b20_verify_device_presence(), internal_ds18b20_write_scratchpad(), rx_ds18b20_deinit(),
rx_ds18b20_get_resolution(), rx_ds18b20_init(), rx_ds18b20_read_power_mode(), rx_ds18b20_read_scratchpad(),
rx_ds18b20_read_temperature(), rx_ds18b20_read_temperature_raw(), rx_ds18b20_recall_config(),
rx_ds18b20_save_config(), rx_ds18b20_set_resolution(), and rx_ds18b20_trigger_conversion().

4.1.2.5 s˙temp˙conversion˙divisor

const float s_temp_conversion_divisor = 16.0F [static]

Temperature conversion divisor (raw to Celsius).

DS18B20 stores temperature in 1/16degC units. Divide by 16 to get Celsius.

Definition at line 225 of file rx_ds18b20.c.

Referenced by internal_ds18b20_raw_to_celsius().

Chapter 5

Directory Documentation

5.1 libs/rx˙ds18b20/inc Directory Reference

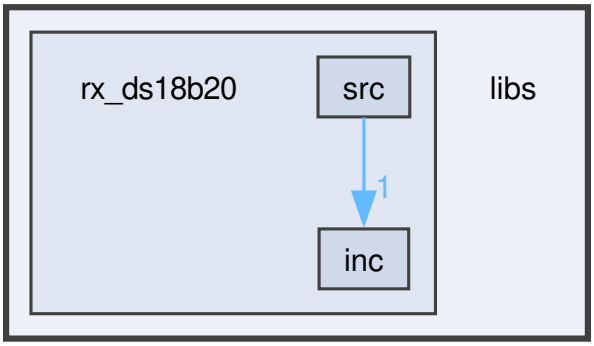
Directory dependency graph for inc:

Files

- file rx_ds18b20.h
  DS18B20 1-Wire Digital Temperature Sensor Driver API.

5.2 libs Directory Reference

Directory dependency graph for libs:

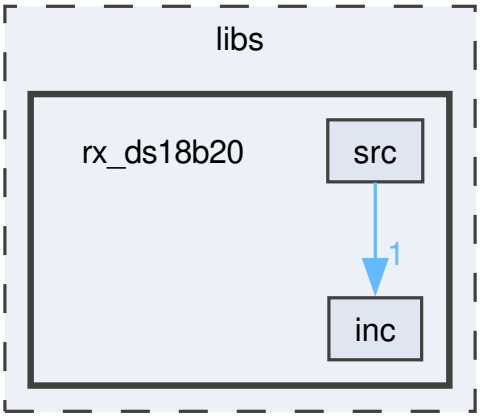


Directories

- directory rx_ds18b20

5.3 libs/rx˙ds18b20 Directory Reference

Directory dependency graph for rx_ds18b20:

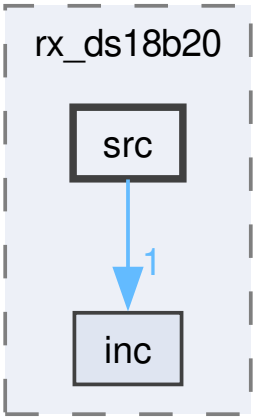


Directories

- directory inc
- directory src

5.4 libs/rx˙ds18b20/src Directory Reference

Directory dependency graph for src:



Files

- file rx_ds18b20.c

  DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

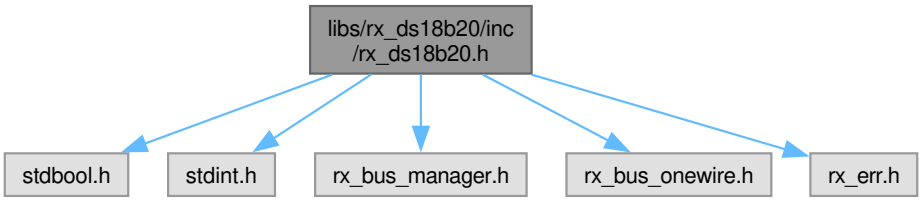
Chapter 6

File Documentation

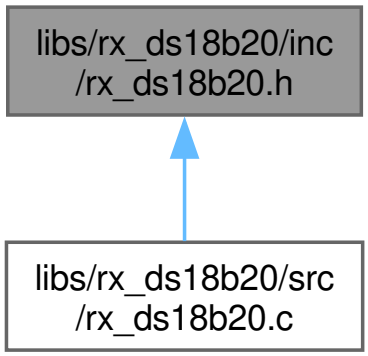
6.1 libs/rx˙ds18b20/inc/rx˙ds18b20.h File Reference

DS18B20 1-Wire Digital Temperature Sensor Driver API.

#include <stdbool.h>
#include <stdint.h>
#include "rx_bus_manager.h"
#include "rx_bus_onewire.h"
#include "rx_err.h"
Include dependency graph for rx_ds18b20.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_ds18b20_config_t
  DS18B20 configuration structure.
- struct rx_ds18b20_handle_t
  DS18B20 driver handle structure.

Enumerations

- enum ds18b20_command_t : uint8_t { k_ds18b20_cmd_convert_t = 0x44 , k_ds18b20_cmd_write_scratch = 0x4E , k_ds18b20_cmd_read_scratch = 0xBE , k_ds18b20_cmd_copy_scratch = 0x48 , k_ds18b20_cmd_recall_eeprom = 0xB8 , k_ds18b20_cmd_read_power = 0xB4 }

  DS18B20 function commands per datasheet specification.
- enum ds18b20_scratchpad_index_t : uint8_t { k_ds18b20_scratch_temp_lsb = 0 , k_ds18b20_scratch_temp_msb = 1 , k_ds18b20_scratch_th_reg = 2 , k_ds18b20_scratch_tl_reg = 3 , k_ds18b20_scratch_config = 4 , k_ds18b20_scratch_reserved1 = 5 , k_ds18b20_scratch_reserved2 = 6 , k_ds18b20_scratch_reserved3 = 7 , k_ds18b20_scratch_crc = 8 , k_ds18b20_scratchpad_bytes = 9 }

  DS18B20 scratchpad memory layout indices.
- enum ds18b20_config_bits_t : uint8_t { k_ds18b20_config_r0_bit = 5 , k_ds18b20_config_r1_bit = 6 }

  DS18B20 configuration register bit positions.
- enum ds18b20_resolution_t : uint8_t { k_ds18b20_resolution_9bit = 0 , k_ds18b20_resolution_10bit = 1 , k_ds18b20_resolution_11bit = 2 , k_ds18b20_resolution_12bit = 3 }

  DS18B20 temperature resolution modes.
- enum ds18b20_conversion_time_ms_t : uint16_t { k_ds18b20_conv_time_9bit_ms = 100 , k_ds18b20_conv_time_10bit_ms = 200 , k_ds18b20_conv_time_11bit_ms = 400 , k_ds18b20_conv_time_12bit_ms = 800 }

  DS18B20 conversion times (milliseconds).
- enum ds18b20_temp_mask_t : uint16_t { k_ds18b20_temp_mask_9bit = 0xFFF8 , k_ds18b20_temp_mask_10bit = 0xFFFC , k_ds18b20_temp_mask_11bit = 0xFFFE , k_ds18b20_temp_mask_12bit = 0xFFFF }

  DS18B20 temperature masks for resolution.
- enum ds18b20_conversion_constants_t : int32_t { k_ds18b20_temp_shift = 4 , k_ds18b20_sign_bit = 0x8000 , k_ds18b20_crc_bytes = 8 , k_ds18b20_shift_byte = 8 , k_ds18b20_scratchpad_write_bytes = 3 , k_ds18b20_temp_min_raw = -880 , k_ds18b20_temp_max_raw = 2000 }

  DS18B20 temperature conversion constants.
- enum ds18b20_write_idx_t : uint8_t { k_ds18b20_write_idx_th = 0 , k_ds18b20_write_idx_tl = 1 , k_ds18b20_write_idx_config = 2 }

  DS18B20 scratchpad write buffer indices.
- enum ds18b20_init_values_t : uint8_t { k_ds18b20_config_register_cleared = 0 }

  DS18B20 initialization values.
- enum ds18b20_power_mode_t : uint8_t { k_ds18b20_power_parasitic = 0 , k_ds18b20_power_external = 1 }

  DS18B20 power supply modes.

Functions

- rx_err_t rx_ds18b20_init (rx_ds18b20_handle_t *handle, const rx_ds18b20_config_t *config)

  Initialize DS18B20 sensor.
- rx_err_t rx_ds18b20_deinit (rx_ds18b20_handle_t *handle)

  Deinitialize DS18B20 sensor.
- rx_err_t rx_ds18b20_trigger_conversion (const rx_ds18b20_handle_t *handle)

  Trigger temperature conversion.
- rx_err_t rx_ds18b20_read_temperature (rx_ds18b20_handle_t *handle, float *temperature_↩ celsius)

Read temperature in Celsius.

- rx_err_t rx_ds18b20_read_temperature_raw (rx_ds18b20_handle_t *handle, int16_t *raw_↩ temp)

  Read raw temperature value.
- rx_err_t rx_ds18b20_set_resolution (rx_ds18b20_handle_t *handle, ds18b20_resolution_t resolution)

  Set temperature resolution.
- rx_err_t rx_ds18b20_get_resolution (const rx_ds18b20_handle_t *handle, ds18b20_resolution_t *resolution)

  Get current temperature resolution.
- rx_err_t rx_ds18b20_save_config (const rx_ds18b20_handle_t *handle)

  Save configuration to EEPROM.
- rx_err_t rx_ds18b20_recall_config (const rx_ds18b20_handle_t *handle)

  Recall configuration from EEPROM.
- rx_err_t rx_ds18b20_read_power_mode (const rx_ds18b20_handle_t *handle, bool *external↩ _power)

  Read power supply mode.
- rx_err_t rx_ds18b20_read_scratchpad (rx_ds18b20_handle_t *handle, uint8_t scratchpad↩ [k_ds18b20_scratchpad_bytes])

  Read scratchpad memory.
- uint32_t rx_ds18b20_get_conversion_time_ms (const rx_ds18b20_handle_t *handle)

  Get conversion time for current resolution.

6.1.1 Detailed Description

DS18B20 1-Wire Digital Temperature Sensor Driver API.

6.1.2 Overview

Platform-independent driver for Maxim DS18B20 programmable resolution 1-Wire digital thermometer with CRC-8 data integrity checking. Provides high-level API for temperature measurement, resolution configuration, and EEPROM persistence with automatic error handling and validation.

This driver abstracts the DS18B20's complex 1-Wire protocol into simple init, trigger, and read operations while maintaining full access to advanced features like ROM matching (multi-drop buses) and alarm thresholds.

6.1.3 Key Features

6.1.3.1 Measurement Capabilities

- Temperature range: -55degC to +125degC with +/-0.5degC accuracy (-10degC to +85degC)
- Programmable resolution: 9, 10, 11, or 12 bits
- Conversion times: 94ms (9-bit) to 750ms (12-bit)
- CRC-8 validation: Detects transmission errors (polynomial x^8 + x^5 + x^4 + 1)

6.1.3.2 Resolution vs. Precision vs. Conversion Time Trade-off

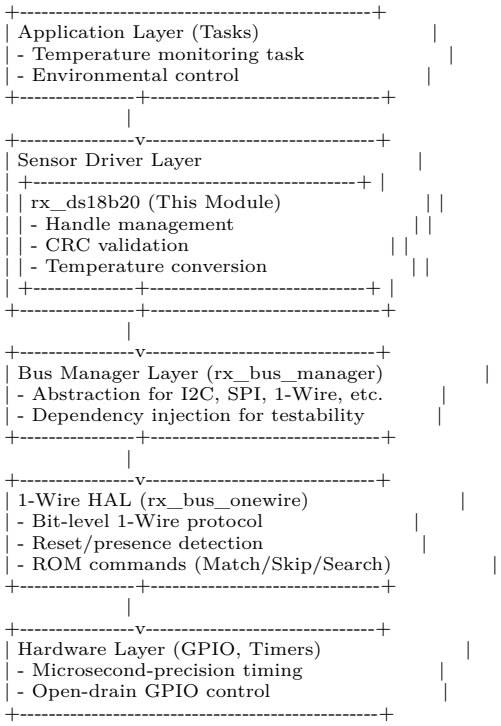
Resolution	Precision	Conversion Time	Use Case
9-bit	0.5degC	∼94ms	Fast updates, low precision needed
10-bit	0.25degC	∼188ms	Balanced for most applications
11-bit	0.125degC	∼375ms	High precision, moderate speed
12-bit	0.0625degC	∼750ms	Maximum precision, slow updates

6.1.3.3 Advanced Features

- ROM matching: Supports multi-device 1-Wire buses (up to 255 sensors)
- Skip ROM mode: Optimized for single-sensor configurations
- EEPROM storage: Persist resolution/alarm settings across power cycles
- Power mode detection: Identify parasitic vs. external power operation
- Alarm thresholds: Programmable TH/TL registers (not exposed in this API)

6.1.4 System Architecture Context

The DS18B20 driver sits at the sensor layer of the STAR firmware stack:



6.1.5 Design Rationale

6.1.5.1 Dependency Injection Pattern

The driver accepts a rx_bus_manager_t∗ instead of directly accessing GPIO, enabling:

- Unit testing with mock bus implementations (no hardware required)
- Hardware independence - works on any 1-Wire HAL implementation
- Multi-bus support - single driver code for multiple 1-Wire buses

This follows the Dependency Inversion Principle (high-level modules depend on abstractions, not concrete implementations).

6.1.5.2 Blocking API Design

All operations are blocking (caller must wait for conversion). Rationale:

- DS18B20 conversions are inherently slow (up to 750ms)
- Non-blocking would require state machines and polling infrastructure
- Simplicity favored: caller can use ThreadX semaphores if async needed
- Typical usage: Dedicated temperature monitoring task with periodic reads

6.1.5.3 Error Handling Strategy

- All functions return rx_err_t for consistent error propagation
- CRC validation on every read - detect bus noise/corruption
- Range checking - reject impossible temperatures (-880 to +2000 raw)
- nullptr checks - fail fast on API misuse
- State validation - ensure init before use, prevent double-init

6.1.6 Implementation Approach

6.1.6.1 Temperature Conversion Process

The DS18B20 uses a two-phase measurement cycle:

1. Trigger Phase (rx_ds18b20_trigger_conversion()):
   - Send Convert T command (0x44)
   - DS18B20 begins analog-to-digital conversion
   - Returns immediately (non-blocking)
2. Wait Phase (Caller's responsibility):
   - Wait for conversion time based on resolution
   - Use rx_ds18b20_get_conversion_time_ms() to get exact time
   - ThreadX: tx_thread_sleep(time_ms * TX_TIMER_TICKS_PER_SECOND / 1000)
3. Read Phase (rx_ds18b20_read_temperature()):
   - Read 9-byte scratchpad (includes CRC)
   - Validate CRC-8 checksum
   - Extract 16-bit raw temperature
   - Apply resolution mask (clear undefined bits)
   - Convert to Celsius (raw / 16.0)

6.1.6.2 Memory Layout and Scratchpad Structure

The DS18B20's scratchpad RAM is 9 bytes:

Byte	Name	Description
0	Temp LSB	Temperature low byte
1	Temp MSB	Temperature high byte (+ sign)
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config	Resolution bits [6:5]
5	Reserved	0xFF
6	Reserved	Factory value
7	Reserved	Factory value
8	CRC	CRC-8 of bytes 0-7

Configuration register format (byte 4):

```
Bit:  7  6  5  4  3  2  1  0
      0 R1 R0  1  1  1  1  1
        +---+- Resolution: 00=9b, 01=10b, 10=11b, 11=12b
```

6.1.7 Performance Characteristics

6.1.7.1 Timing Analysis

Operation	Duration	Notes
Init	~10ms	Presence detect + read
Trigger conversion	~1ms	Command transmission
Conversion (9-bit)	93.75ms (spec)	100ms recommended
Conversion (10-bit)	187.5ms (spec)	200ms recommended
Conversion (11-bit)	375ms (spec)	400ms recommended
Conversion (12-bit)	750ms (spec)	800ms recommended
Read scratchpad	~2ms	9 bytes + CRC validation
Set resolution	~5ms	Read + modify + write

Note: Recommended times include 10% safety margin above datasheet specs.

6.1.7.2 Memory Usage

Component	Size (bytes)	Location
rx_ds18b20_handle_t	28	Caller stack
rx_ds18b20_config_t	24	Caller stack

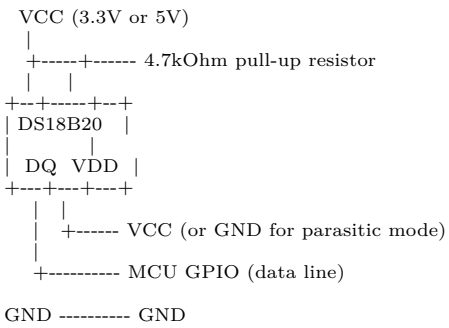
Component	Size (bytes)	Location
Scratchpad buffer	9	Stack (temp)
Driver code (.text)	∼2KB	Flash
Constant data (.rodata)	∼100 bytes	Flash

Total RAM per sensor: 28 bytes (handle only, no dynamic allocation)

6.1.8 Hardware Requirements

Resource	Requirement
CPU	Any (architecture-independent)
RAM	28 bytes per sensor + 9-byte temp buffer
Flash	∼2KB code + ∼100 bytes constants
GPIO	1 pin (open-drain or push-pull with resistor)
Timing	Microsecond-precision delays (1-Wire protocol)
Pull-up	4.7kOhm resistor to VCC (external)
Power	3.0V-5.5V (parasitic mode: data pin can power)

6.1.8.1 Typical Wiring (Single Sensor)



6.1.9 Example Usage Patterns

6.1.9.1 Example 1: Basic Single-Sensor Configuration

```
#include "rx_ds18b20.h"
#include "rx_bus_manager.h"
#include "tx_api.h"

void temperature_task_entry(ULONG input) {
    rx_ds18b20_handle_t sensor;
    rx_ds18b20_config_t config = {
        .bus_manager = &g_bus_manager,
        .bus_name = "onewire0",
        .resolution = k_ds18b20_resolution_12bit,  // 0.0625degC precision
        .use_rom_matching = false,                 // Skip ROM (single sensor)
    };

    rx_err_t err = rx_ds18b20_init(&sensor, &config);
    if (err != k_rx_ok) {
        // Handle error: check bus wiring, power, pull-up resistor
        return;
```

```
    }

    while (1) {
        // Trigger conversion
        err = rx_ds18b20_trigger_conversion(&sensor);
        if (err != k_rx_ok) continue;

        // Wait for conversion (800ms for 12-bit)
        tx_thread_sleep(80);  // 800ms at 100 Hz tick rate

        // Read temperature
        float temp_celsius;
        err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
        if (err == k_rx_ok) {
            // Process temperature (e.g., log, control, transmit)
            printf("Temperature: %.2fdegC\n", temp_celsius);
        } else if (err == k_rx_err_crc_mismatch) {
            // CRC error: retry or log fault
        }

        // Repeat every 5 seconds
        tx_thread_sleep(500);
    }
}
```

6.1.9.2 Example 2: Multi-Sensor Configuration (ROM Matching)

```
// Pre-discovered ROM codes (use ROM search to find these)
uint8_t rom_sensor1[8] = {0x28, 0xFF, 0x12, 0x34, 0x56, 0x78, 0x90, 0xAB};
uint8_t rom_sensor2[8] = {0x28, 0xFF, 0xAB, 0xCD, 0xEF, 0x01, 0x23, 0x45};

rx_ds18b20_handle_t sensors[2];
rx_ds18b20_config_t config_template = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_11bit,
    .use_rom_matching = true,  // Enable ROM matching for multi-drop
};

// Init sensor 1
memcpy(config_template.rom, rom_sensor1, 8);
rx_ds18b20_init(&sensors[0], &config_template);

// Init sensor 2
memcpy(config_template.rom, rom_sensor2, 8);
rx_ds18b20_init(&sensors[1], &config_template);

// Trigger both simultaneously (broadcast not shown, or trigger individually)
rx_ds18b20_trigger_conversion(&sensors[0]);
rx_ds18b20_trigger_conversion(&sensors[1]);

tx_thread_sleep(40);  // Wait 400ms for 11-bit

// Read both
float temps[2];
rx_ds18b20_read_temperature(&sensors[0], &temps[0]);
rx_ds18b20_read_temperature(&sensors[1], &temps[1]);
```

6.1.9.3 Example 3: Error Handling and Diagnostics

```
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_c);

switch (err) {
    case k_rx_ok:
        // Success: use temp_c
        break;

    case k_rx_err_crc_mismatch:
        // CRC error: possible bus noise, retry
        retry_count++;
        break;

    case k_rx_err_invalid_state:
        // Sensor disconnected or not responding
        // Check: wiring, power, pull-up resistor
        break;

    case k_rx_err_out_of_range:
        // Temperature outside -55degC to +125degC
        // Sensor fault or extreme environment
```

```
        break;

    default:
        // Unexpected error
        break;
}
```

6.1.9.4 Example 4: Runtime Resolution Change

```
// Switch to 9-bit for faster updates (0.5degC precision)
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_9bit);

// Persist to EEPROM (survives power cycle)
rx_ds18b20_save_config(&sensor);

// Now trigger with shorter wait time
rx_ds18b20_trigger_conversion(&sensor);
tx_thread_sleep(10);  // 100ms for 9-bit
rx_ds18b20_read_temperature(&sensor, &temp_c);
```

6.1.10 Thread Safety

Not thread-safe. Caller must serialize access if multiple threads use the same sensor handle. Use ThreadX mutexes:

```
TX_MUTEX sensor_mutex;
tx_mutex_create(&sensor_mutex, "DS18B20", TX_NO_INHERIT);

// Thread 1:
tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
rx_ds18b20_trigger_conversion(&sensor);
tx_mutex_put(&sensor_mutex);

// Thread 2:
tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
rx_ds18b20_read_temperature(&sensor, &temp_c);
tx_mutex_put(&sensor_mutex);
```

Note: Different sensor handles (different physical sensors) CAN be accessed concurrently without synchronization, even on the same 1-Wire bus (ROM matching addresses each sensor independently).

6.1.11 Known Limitations

1. No alarm threshold API: TH/TL registers accessible via scratchpad but not exposed. Users can read/write scratchpad directly if needed.
2. No broadcast/simultaneous conversion: Must trigger each sensor individually when using ROM matching. Skip ROM mode allows broadcast but only for single-sensor.
3. No ROM search: Must provide ROM codes manually for multi-sensor. Consider adding rx_↩ ds18b20_discover() helper in future.
4. Blocking API only: No asynchronous/callback interface. Caller must use RTOS primitives (tasks, timers) for non-blocking behavior.

6.1.12 References and Standards

- DS18B20 Datasheet: Maxim Integrated (now Analog Devices) https://www.analog.com/media/↩ en/technical-documentation/data-sheets/DS18B20.pdf
- 1-Wire Protocol: Book of iButton Standards, Maxim Integrated
- CRC-8 Polynomial: x^8 + x^5 + x^4 + 1 (standard for 1-Wire devices)
- ROM Structure: 8-byte format (1-byte family + 6-byte serial + 1-byte CRC)
- IEEE 802.3 Ethernet CRC: Different from DS18B20's CRC-8, not applicable here

See also

rx_bus_manager.h - Bus manager abstraction layer
rx_bus_onewire.h - 1-Wire protocol implementation
rx_crc.h - CRC calculation utilities (CRC-8 for DS18B20)
docs/sections/03_hardware_pinout.tex - GPIO pin assignments

NASA Power of 10 Compliance:

- Rule 1 (No goto): [OK] All control flow uses structured statements
- Rule 2 (Bounded loops): [OK] All loops have compile-time or runtime bounds
- Rule 3 (No heap): [OK] Zero dynamic allocation (static handles only)
- Rule 4 (Functions <=60 lines): [OK] All public functions <60 lines
- Rule 5 (Assertions): [OK] 2+ preconditions per function (NULL checks, state)
- Rule 6 (Data scope): [OK] Variables declared at smallest scope
- Rule 7 (Return checking): [OK] All HAL returns validated
- Rule 8 (Preprocessor limits): [OK] C23 typed enums only, minimal macros
- Rule 9 (Pointer restrictions): [WARN] Intentional deviation for DIP (bus_manager)
- Rule 10 (Compiler warnings): [OK] -Wall -Wextra -Werror, zero warnings

SOLID Principles:

- S (Single Responsibility): Module handles ONLY DS18B20 sensor operations
- O (Open/Closed): Extensible via bus_manager (add new buses w/o changes)
- L (Liskov Substitution): Any rx_bus_manager_t* is interchangeable
- I (Interface Segregation): Focused API (temp read, no unrelated operations)
- D (Dependency Inversion): Depends on rx_bus_manager_t abstraction, not GPIO

Module Dependencies:

```
rx_ds18b20
  +-- rx_bus_manager (bus abstraction, injected)
  +-- rx_bus_onewire (1-Wire protocol layer)
  +-- rx_crc (CRC-8 validation)
  +-- rx_err (error codes)
  +-- rx_log (diagnostic logging)
  +-- rx_check (nullptr validation macros)
```

Author

Locked, Inc.

Date

2026-01-30

Version

1.0.0

Copyright

Definition in file rx_ds18b20.h.

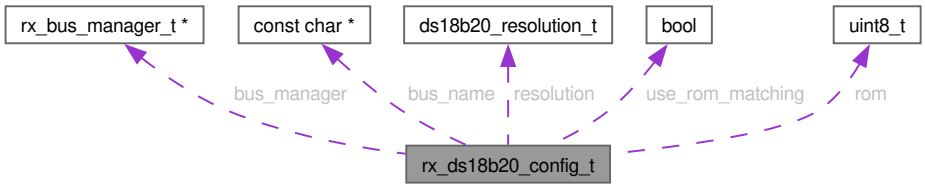
6.1.13 Data Structure Documentation

6.1.13.1 struct rx_ds18b20_config_t

DS18B20 configuration structure.

Definition at line 691 of file rx_ds18b20.h.

Collaboration diagram for rx_ds18b20_config_t:



Data Fields

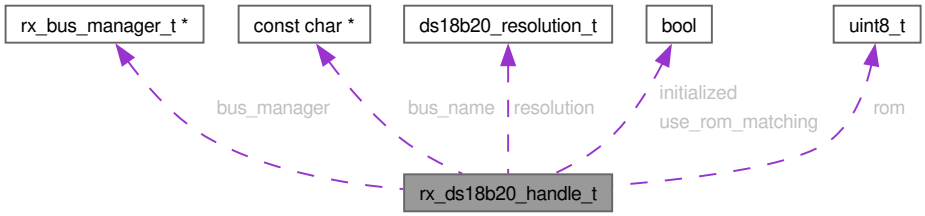
rx_bus_manager_t *	bus_manager	Bus manager instance (required)
const char *	bus_name	OneWire bus name (required)
ds18b20_resolution_t	resolution	Temperature resolution
uint8_t	rom[k_onewire_rom_bytes]	Device ROM (if use_rom_matching=true)
bool	use_rom_matching	Use ROM matching (true) or skip ROM (false)

6.1.13.2 struct rx_ds18b20_handle_t

DS18B20 driver handle structure.

Definition at line 702 of file rx_ds18b20.h.

Collaboration diagram for rx_ds18b20_handle_t:



Data Fields

rx_bus_manager_t *	bus_manager	Bus manager reference (not owned)
const char *	bus_name	OneWire bus name (not owned)

bool	initialized	True after successful init
ds18b20_resolution_t	resolution	Current temperature resolution
uint8_t	rom↩ [k_onewire_rom_bytes]	Device ROM code
bool	use_rom_matching	ROM matching enabled

6.1.14 Enumeration Type Documentation

6.1.14.1 ds18b20˙command˙t

enum ds18b20_command_t : uint8_t

DS18B20 function commands per datasheet specification.

Command codes sent after ROM selection (Match ROM or Skip ROM) to initiate DS18B20 operations. Each command triggers a specific sensor function.

Protocol Sequence:

1. Reset pulse + presence detection
2. ROM command (Match ROM 0x55 or Skip ROM 0xCC)
3. Function command (one of these values)
4. Additional data bytes (if required by command)

Command Details:

Command	Code	Data Bytes	Duration	Description
Convert T	0x44	0	94-750ms	Start temperature conversion
Write Scratchpad	0x4E	3	∼2ms	Write TH, TL, Config registers
Read Scratchpad	0xBE	0	∼2ms	Read 9-byte scratchpad + CRC
Copy Scratchpad	0x48	0	∼10ms	Save scratchpad to EEPROM
Recall E^2	0xB8	0	∼1ms	Restore EEPROM to scratchpad
Read Power Supply	0xB4	0	<1ms	Check parasitic vs. external

Usage Example:

```
// Trigger conversion
rx_bus_onewire_reset(bus_mgr, "onewire0", &presence);
rx_bus_onewire_skip_rom(bus_mgr, "onewire0");
rx_bus_onewire_write_byte(bus_mgr, "onewire0", k_ds18b20_cmd_convert_t);
// Wait conversion time...
```

See also

DS18B20 datasheet section "Function Commands" for timing specifications

rx_bus_onewire.h for ROM command functions

Since

Version 1.0.0

Enumerator

k_ds18b20_cmd_convert_t	Trigger temperature conversion (ADC operation)
k_ds18b20_cmd_write_scratch	Write to scratchpad memory (TH, TL, Config bytes)
k_ds18b20_cmd_read_scratch	Read from scratchpad memory (9 bytes: data + CRC)
k_ds18b20_cmd_copy_scratch	Copy scratchpad to EEPROM (non-volatile storage)
k_ds18b20_cmd_recall_eeprom	Recall EEPROM to scratchpad (restore saved values)
k_ds18b20_cmd_read_power	Read power supply mode (0=parasitic, 1=external)

Definition at line 494 of file rx_ds18b20.h.

```
00494             : uint8_t {
00495     k_ds18b20_cmd_convert_t     = 0x44, /**< Trigger temperature conversion (ADC operation) */
00496     k_ds18b20_cmd_write_scratch = 0x4E, /**< Write to scratchpad memory (TH, TL, Config bytes) */
00497     k_ds18b20_cmd_read_scratch  = 0xBE, /**< Read from scratchpad memory (9 bytes: data + CRC) */
00498     k_ds18b20_cmd_copy_scratch  = 0x48, /**< Copy scratchpad to EEPROM (non-volatile storage) */
00499     k_ds18b20_cmd_recall_eeprom = 0xB8, /**< Recall EEPROM to scratchpad (restore saved values) */
00500     k_ds18b20_cmd_read_power    = 0xB4, /**< Read power supply mode (0=parasitic, 1=external) */
00501 } ds18b20_command_t;
```

6.1.14.2 ds18b20˙config˙bits˙t

enum ds18b20_config_bits_t : uint8_t

DS18B20 configuration register bit positions.

Enumerator

k_ds18b20_config_r0_bit	Resolution bit 0
k_ds18b20_config_r1_bit	Resolution bit 1

Definition at line 605 of file rx_ds18b20.h.

```
00605             : uint8_t {
00606     k_ds18b20_config_r0_bit = 5, /**< Resolution bit 0 */
00607     k_ds18b20_config_r1_bit = 6, /**< Resolution bit 1 */
00608 } ds18b20_config_bits_t;
```

6.1.14.3 ds18b20˙conversion˙constants˙t

enum ds18b20_conversion_constants_t : int32_t

DS18B20 temperature conversion constants.

Enumerator

k_ds18b20_temp_shift	Shift to get integer temperature
k_ds18b20_sign_bit	Sign bit in 16-bit temperature
k_ds18b20_crc_bytes	Number of bytes for CRC calculation

k_ds18b20_shift_byte	Shift for byte positioning
k_ds18b20_scratchpad_write_bytes	Scratchpad write size (TH, TL, Config)
k_ds18b20_temp_min_raw	Minimum temperature (-55degC raw value)
k_ds18b20_temp_max_raw	Maximum temperature (+125degC raw value)

Definition at line 649 of file rx_ds18b20.h.

```
00649             : int32_t {
00650   k_ds18b20_temp_shift            = 4,      /**< Shift to get integer temperature */
00651   k_ds18b20_sign_bit              = 0x8000, /**< Sign bit in 16-bit temperature */
00652   k_ds18b20_crc_bytes             = 8,      /**< Number of bytes for CRC calculation */
00653   k_ds18b20_shift_byte            = 8,      /**< Shift for byte positioning */
00654   k_ds18b20_scratchpad_write_bytes = 3,     /**< Scratchpad write size (TH, TL, Config) */
00655   k_ds18b20_temp_min_raw          = -880,   /**< Minimum temperature (-55degC raw value) */
00656   k_ds18b20_temp_max_raw          = 2000,   /**< Maximum temperature (+125degC raw value) */
00657 } ds18b20_conversion_constants_t;
```

6.1.14.4 ds18b20˙conversion˙time˙ms˙t

enum ds18b20_conversion_time_ms_t : uint16_t

DS18B20 conversion times (milliseconds).

Maximum conversion time for each resolution. Add margin for safety (spec values + 10%).

Enumerator

k_ds18b20_conv_time_9bit_ms	9-bit conversion: 100ms
k_ds18b20_conv_time_10bit_ms	10-bit conversion: 200ms
k_ds18b20_conv_time_11bit_ms	11-bit conversion: 400ms
k_ds18b20_conv_time_12bit_ms	12-bit conversion: 800ms

Definition at line 626 of file rx_ds18b20.h.

```
00626             : uint16_t {
00627   k_ds18b20_conv_time_9bit_ms  = 100, /**< 9-bit conversion: 100ms */
00628   k_ds18b20_conv_time_10bit_ms = 200, /**< 10-bit conversion: 200ms */
00629   k_ds18b20_conv_time_11bit_ms = 400, /**< 11-bit conversion: 400ms */
00630   k_ds18b20_conv_time_12bit_ms = 800, /**< 12-bit conversion: 800ms */
00631 } ds18b20_conversion_time_ms_t;
```

6.1.14.5 ds18b20˙init˙values˙t

enum ds18b20_init_values_t : uint8_t

DS18B20 initialization values.

Enumerator

k_ds18b20_config_register_cleared	Cleared config register value

Definition at line 671 of file rx_ds18b20.h.

```
00671             : uint8_t {
00672   k_ds18b20_config_register_cleared = 0, /**< Cleared config register value */
00673 } ds18b20_init_values_t;
```

6.1.14.6 ds18b20˙power˙mode˙t

enum ds18b20_power_mode_t : uint8_t

DS18B20 power supply modes.

Enumerator

k_ds18b20_power_parasitic	Parasitic power mode
k_ds18b20_power_external	External power mode

Definition at line 678 of file rx_ds18b20.h.

```
00678             : uint8_t {
00679   k_ds18b20_power_parasitic = 0, /**< Parasitic power mode */
00680   k_ds18b20_power_external  = 1, /**< External power mode */
00681 } ds18b20_power_mode_t;
```

6.1.14.7 ds18b20˙resolution˙t

enum ds18b20_resolution_t : uint8_t

DS18B20 temperature resolution modes.

Enumerator

k_ds18b20_resolution_9bit	9-bit: 0.5degC, 93.75ms
k_ds18b20_resolution_10bit	10-bit: 0.25degC, 187.5ms
k_ds18b20_resolution_11bit	11-bit: 0.125degC, 375ms
k_ds18b20_resolution_12bit	12-bit: 0.0625degC, 750ms

Definition at line 613 of file rx_ds18b20.h.

```
00613             : uint8_t {
00614   k_ds18b20_resolution_9bit  = 0, /**< 9-bit: 0.5degC, 93.75ms */
00615   k_ds18b20_resolution_10bit = 1, /**< 10-bit: 0.25degC, 187.5ms */
00616   k_ds18b20_resolution_11bit = 2, /**< 11-bit: 0.125degC, 375ms */
00617   k_ds18b20_resolution_12bit = 3, /**< 12-bit: 0.0625degC, 750ms */
00618 } ds18b20_resolution_t;
```

6.1.14.8 ds18b20˙scratchpad˙index˙t

enum ds18b20_scratchpad_index_t : uint8_t

DS18B20 scratchpad memory layout indices.

The DS18B20's 9-byte scratchpad RAM contains temperature data, alarm thresholds, configuration, and a CRC-8 checksum. Use these indices to access specific bytes when reading via Read Scratchpad command (0xBE).

Memory Layout:

Index	Byte	Name	Description	Access
0	0	Temp LSB	Temperature low byte (bits 0-7)	Read-only
1	1	Temp MSB	Temperature high byte (sign + bits 8-11)	Read-only
2	2	TH Register	High alarm threshold (-55 to +125degC)	R/W
3	3	TL Register	Low alarm threshold (-55 to +125degC)	R/W
4	4	Config	Resolution bits [6:5]: 00=9b, 01=10b, 10=11b, 11=12b	R/W
5	5	Reserved	Always 0xFF (ignore)	Read-only
6	6	Reserved	Factory-programmed (ignore)	Read-only
7	7	Reserved	Factory-programmed (ignore)	Read-only
8	8	CRC	CRC-8 checksum of bytes 0-7	Read-only

Temperature Format (Bytes 0-1):

16-bit signed integer in 1/16degC units (two's complement):

```
MSB (byte 1):  S  S  S  S  S  T10 T9  T8
LSB (byte 0):  T7 T6 T5 T4 T3 T2  T1  T0
          +-----------------------+- Temperature bits (12-bit for 12-bit resolution)
```

- S: Sign bits (all 0 for positive, all 1 for negative)
- T10-T0: Temperature bits (11 bits used, T0-T2 undefined for lower resolutions)
- Value range: -880 to +2000 (represents -55.0degC to +125.0degC)
- Conversion: temp_celsius = raw_value / 16.0

Configuration Register (Byte 4):

Format: 0 R1 R0 1 1 1 1 1 where R1:R0 = resolution bits

R1	R0	Resolution	Precision	Conversion Time
0	0	9-bit	0.5degC	93.75ms
0	1	10-bit	0.25degC	187.5ms
1	0	11-bit	0.125degC	375ms
1	1	12-bit	0.0625degC	750ms

CRC-8 Calculation (Byte 8):

- Polynomial: x^8 + x^5 + x^4 + 1 (0x8C)
- Initial value: 0x00
- Data: Bytes 0-7 (inclusive)
- Purpose: Detect transmission errors on 1-Wire bus

Usage Example:

```
uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
rx_ds18b20_read_scratchpad(&sensor, scratchpad);

// Extract temperature
uint16_t temp_raw = scratchpad[k_ds18b20_scratch_temp_lsb] |
                    (scratchpad[k_ds18b20_scratch_temp_msb] « 8);

// Validate CRC
uint32_t crc_out = 0U;
(void)rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes, &crc_out);
if ((uint8_t)crc_out != scratchpad[k_ds18b20_scratch_crc]) {
    // CRC mismatch: retry read
}

// Read alarm thresholds
int8_t th = (int8_t)scratchpad[k_ds18b20_scratch_th_reg];
int8_t tl = (int8_t)scratchpad[k_ds18b20_scratch_tl_reg];
```

Invariant

Byte 5 (reserved1) is always 0xFF

CRC (byte 8) is valid for bytes 0-7

Temperature bytes (0-1) represent value in range [-880, +2000]

See also

rx_ds18b20_read_scratchpad() Read entire scratchpad with CRC validation

rx_crc8_maxim() CRC-8 calculation function

DS18B20 datasheet section ”Memory” for detailed scratchpad specification

Since

Version 1.0.0

Enumerator

k_ds18b20_scratch_temp_lsb	Temperature LSB (bits 0-7 of 16-bit value)
k_ds18b20_scratch_temp_msb	Temperature MSB (sign bits + bits 8-11)
k_ds18b20_scratch_th_reg	TH (high alarm) register (signed byte, -55 to +125)
k_ds18b20_scratch_tl_reg	TL (low alarm) register (signed byte, -55 to +125)
k_ds18b20_scratch_config	Configuration register (resolution bits at [6:5])
k_ds18b20_scratch_reserved1	Reserved byte (always 0xFF, do not write)
k_ds18b20_scratch_reserved2	Reserved byte (factory-programmed, do not write)
k_ds18b20_scratch_reserved3	Reserved byte (factory-programmed, do not write)
k_ds18b20_scratch_crc	CRC-8 of bytes 0-7 (polynomial 0x8C)
k_ds18b20_scratchpad_bytes	Total scratchpad size in bytes

Definition at line 589 of file rx_ds18b20.h.

```
00589             : uint8_t {
00590   k_ds18b20_scratch_temp_lsb  = 0, /**< Temperature LSB (bits 0-7 of 16-bit value) */
00591   k_ds18b20_scratch_temp_msb  = 1, /**< Temperature MSB (sign bits + bits 8-11) */
00592   k_ds18b20_scratch_th_reg    = 2, /**< TH (high alarm) register (signed byte, -55 to +125) */
00593   k_ds18b20_scratch_tl_reg    = 3, /**< TL (low alarm) register (signed byte, -55 to +125) */
00594   k_ds18b20_scratch_config    = 4, /**< Configuration register (resolution bits at [6:5]) */
00595   k_ds18b20_scratch_reserved1 = 5, /**< Reserved byte (always 0xFF, do not write) */
00596   k_ds18b20_scratch_reserved2 = 6, /**< Reserved byte (factory-programmed, do not write) */
00597   k_ds18b20_scratch_reserved3 = 7, /**< Reserved byte (factory-programmed, do not write) */
00598   k_ds18b20_scratch_crc       = 8, /**< CRC-8 of bytes 0-7 (polynomial 0x8C) */
00599   k_ds18b20_scratchpad_bytes  = 9, /**< Total scratchpad size in bytes */
00600 } ds18b20_scratchpad_index_t;
```

6.1.14.9 ds18b20˙temp˙mask˙t

enum ds18b20_temp_mask_t : uint16_t

DS18B20 temperature masks for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked. This prevents garbage bits from corrupting the temperature reading.

Enumerator

k_ds18b20_temp_mask_9bit	Mask for 9-bit (discard bits 0-2)
k_ds18b20_temp_mask_10bit	Mask for 10-bit (discard bits 0-1)
k_ds18b20_temp_mask_11bit	Mask for 11-bit (discard bit 0)
k_ds18b20_temp_mask_12bit	Mask for 12-bit (all bits valid)

Definition at line 639 of file rx_ds18b20.h.

```
00639             : uint16_t {
00640   k_ds18b20_temp_mask_9bit  = 0xFFF8, /**< Mask for 9-bit (discard bits 0-2) */
00641   k_ds18b20_temp_mask_10bit = 0xFFFC, /**< Mask for 10-bit (discard bits 0-1) */
00642   k_ds18b20_temp_mask_11bit = 0xFFFE, /**< Mask for 11-bit (discard bit 0) */
00643   k_ds18b20_temp_mask_12bit = 0xFFFF, /**< Mask for 12-bit (all bits valid) */
00644 } ds18b20_temp_mask_t;
```

6.1.14.10 ds18b20˙write˙idx˙t

enum ds18b20_write_idx_t : uint8_t

DS18B20 scratchpad write buffer indices.

Enumerator

k_ds18b20_write_idx_th	TH (high alarm) register index
k_ds18b20_write_idx_tl	TL (low alarm) register index
k_ds18b20_write_idx_config	Configuration register index

Definition at line 662 of file rx_ds18b20.h.

```
00662             : uint8_t {
00663   k_ds18b20_write_idx_th     = 0, /**< TH (high alarm) register index */
00664   k_ds18b20_write_idx_tl     = 1, /**< TL (low alarm) register index */
00665   k_ds18b20_write_idx_config = 2, /**< Configuration register index */
00666 } ds18b20_write_idx_t;
```

6.1.15 Function Documentation

6.1.15.1 rx˙ds18b20˙deinit()

```
rx_err_t rx_ds18b20_deinit (
                rx_ds18b20_handle_t * handle)   [nodiscard]
```

Deinitialize DS18B20 sensor.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle is nullptr

k_rx_err_invalid_state if not initialized

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via memset. After this call the handle is safe to reinitialize with rx_ds18b20_init().

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with memset (zeros all fields)
4. Log deinitialization success

Precondition

handle must be a valid non-null pointer

handle must have been successfully initialized via rx_ds18b20_init()

Postcondition

All handle fields are zeroed (initialized flag is false)

handle may be safely reused with rx_ds18b20_init()

Note

Not thread-safe; caller must ensure no concurrent access to handle

Example:

```
rx_ds18b20_handle_t sensor;
rx_ds18b20_init(&sensor, &config);
// ... use sensor ...
rx_err_t err = rx_ds18b20_deinit(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to deinit DS18B20");
}
```

See also

rx_ds18b20_init() Initialize the handle before use

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

Definition at line 434 of file rx_ds18b20.c.

```
00435 {
00436   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00437
00438   if (!handle->initialized) {
00439     rx_log_warn(s_tag, "DS18B20 not initialized");
00440     return k_rx_err_invalid_state;
00441   }
00442
00443   /* Clear handle */
00444   *handle = (rx_ds18b20_handle_t){};
00445
00446   rx_log_info(s_tag, "DS18B20 deinitialized");
00447   return k_rx_ok;
00448 }
```

References rx_ds18b20_handle_t::initialized, and s_tag.

6.1.15.2 rx˙ds18b20˙get˙conversion˙time˙ms()

```
uint32_t rx_ds18b20_get_conversion_time_ms (
                const rx_ds18b20_handle_t * handle)
```

Get conversion time for current resolution.

Returns the maximum conversion time in milliseconds for the configured resolution. Caller should wait this long after triggering conversion before reading temperature.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.

Returns

Conversion time in milliseconds, or 0 if handle is nullptr or not initialized

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Definition at line 1121 of file rx_ds18b20.c.

```
01122 {
01123   if (handle == nullptr || !handle->initialized) {
01124     return s_ds18b20_conversion_time_invalid_u32;
01125   }
01126
01127   switch (handle->resolution) {
01128     case k_ds18b20_resolution_9bit:
01129       return k_ds18b20_conv_time_9bit_ms;
01130     case k_ds18b20_resolution_10bit:
01131       return k_ds18b20_conv_time_10bit_ms;
01132     case k_ds18b20_resolution_11bit:
01133       return k_ds18b20_conv_time_11bit_ms;
01134     case k_ds18b20_resolution_12bit:
01135       return k_ds18b20_conv_time_12bit_ms;
01136     default:
01137       return s_ds18b20_conversion_time_invalid_u32;
01138   }
01139 }
```

References rx_ds18b20_handle_t::initialized, k_ds18b20_conv_time_10bit_ms, k_ds18b20_conv_time_11bit_ms, k_ds18b20_conv_time_12bit_ms, k_ds18b20_conv_time_9bit_ms, k_ds18b20_resolution_10bit, k_ds18b20_resolution_11bit, k_ds18b20_resolution_12bit, k_ds18b20_resolution_9bit, rx_ds18b20_handle_t::resolu
and s_ds18b20_conversion_time_invalid_u32.

6.1.15.3 rx˙ds18b20˙get˙resolution()

```
rx_err_t rx_ds18b20_get_resolution (
                const rx_ds18b20_handle_t * handle,
                ds18b20_resolution_t * resolution)   [nodiscard]
```

Get current temperature resolution.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	resolution	Pointer to store resolution. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if any pointer is nullptr

k_rx_err_invalid_state if not initialized

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via rx_ds18b20_set_resolution()). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Precondition

handle must be initialized via rx_ds18b20_init()

resolution must be a valid non-null pointer

Postcondition

*resolution contains the current resolution value on k_rx_ok

handle state is unchanged

Note

Not thread-safe; caller must ensure handle is not concurrently modified

Returns cached value from handle – does not issue 1-Wire bus transaction

Example:

```
ds18b20_resolution_t res;
rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
if (err == k_rx_ok) {
    uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
    rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
}
```

See also

rx_ds18b20_set_resolution() Change the ADC resolution

rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 807 of file rx_ds18b20.c.

```
00809 {
00810   CHECK_DS18B20_HANDLE(handle, s_tag);
00811   RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00812
00813   *resolution = handle->resolution;
00814   return k_rx_ok;
00815 }
```

References CHECK_DS18B20_HANDLE, rx_ds18b20_handle_t::resolution, and s_tag.

6.1.15.4 rx˙ds18b20˙init()

```
rx_err_t rx_ds18b20_init (
            rx_ds18b20_handle_t * handle,
            const rx_ds18b20_config_t * config)   [nodiscard]
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters

out	handle	Pointer to DS18B20 handle. Must not be nullptr.
in	config	Pointer to configuration. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle or config is nullptr

k_rx_err_invalid_arg if bus_manager or bus_name is nullptr

k_rx_err_invalid_state if device not responding

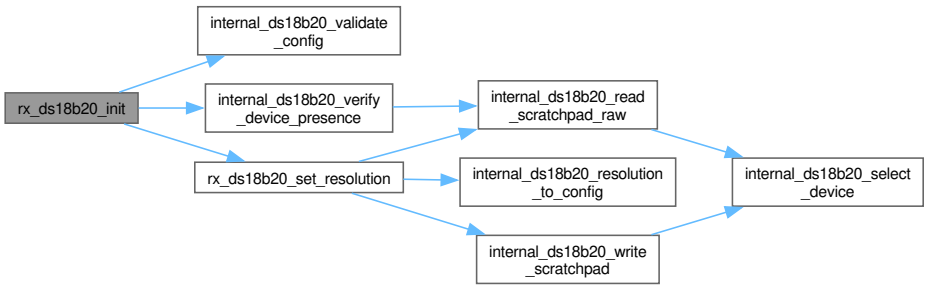
k_rx_err_crc if scratchpad CRC check fails

Definition at line 352 of file rx_ds18b20.c.

```
00353 {
00354   /* Validate inputs */
00355   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00356
00357   rx_err_t err = internal_ds18b20_validate_config(config, handle);
00358   if (err != k_rx_ok) {
00359     return err;
00360   }
00361
00362   /* Initialize handle */
00363   *handle                 = (rx_ds18b20_handle_t){};
00364   handle->bus_manager     = config->bus_manager;
00365   handle->bus_name        = config->bus_name;
00366   handle->resolution      = config->resolution;
00367   handle->use_rom_matching = config->use_rom_matching;
00368
00369   if (config->use_rom_matching) {
00370     for (uint8_t i = 0; i < k_onewire_rom_bytes; ++i) {
00371       handle->rom[i] = config->rom[i];
00372     }
00373   }
00374
00375   /* Verify device presence */
00376   err = internal_ds18b20_verify_device_presence(handle);
00377   if (err != k_rx_ok) {
00378     return err;
00379   }
00380
00381   /* Configure resolution */
00382   handle->initialized = true;
00383   err                 = rx_ds18b20_set_resolution(handle, config->resolution);
00384   if (err != k_rx_ok) {
00385     handle->initialized = false;
00386     return err;
00387   }
00388
00389   rx_log_info(s_tag, "DS18B20 initialized successfully");
00390   return k_rx_ok;
00391 }
```

References rx_ds18b20_config_t::bus_manager, rx_ds18b20_handle_t::bus_manager, rx_ds18b20_config_t::bus_nam
rx_ds18b20_handle_t::bus_name, rx_ds18b20_handle_t::initialized, internal_ds18b20_validate_config(),
internal_ds18b20_verify_device_presence(), rx_ds18b20_config_t::resolution, rx_ds18b20_handle_t::resolution,
rx_ds18b20_config_t::rom, rx_ds18b20_handle_t::rom, rx_ds18b20_set_resolution(), s_tag,
rx_ds18b20_config_t::use_rom_matching, and rx_ds18b20_handle_t::use_rom_matching.

Here is the call graph for this function:



6.1.15.5 rx_ds18b20_read_power_mode()

```
rx_err_t rx_ds18b20_read_power_mode (
                const rx_ds18b20_handle_t * handle,
                bool * external_power)   [nodiscard]
```

Read power supply mode.

Checks if sensor is using parasitic power or external power.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	external_power	True if external power, false if parasitic. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if any pointer is nullptr

k_rx_err_invalid_state if not initialized or device not present

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to *external_power

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected to the bus

Postcondition

*external_power reflects the device power supply mode on k_rx_ok

handle state is unchanged

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:

```
bool ext_power = false;
rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
if (err == k_rx_ok && !ext_power) {
    rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
}
```

See also

rx_ds18b20_save_config() EEPROM write (requires external power)

rx_ds18b20_init() Initialize sensor handle

Since

Version 1.0.0

NASA Power of 10 Compliance:

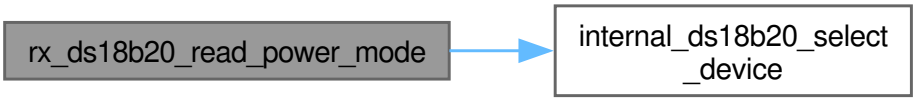
- Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

Definition at line 1012 of file rx_ds18b20.c.

```
01013 {
01014   CHECK_DS18B20_HANDLE(handle, s_tag);
01015   RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01016
01017   /* Reset and check presence */
01018   bool     presence = false;
01019   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01020   if (err != k_rx_ok || !presence) {
01021     rx_log_error(s_tag, "Device not present for power mode read");
01022     return k_rx_err_invalid_state;
01023   }
01024
01025   /* Select device */
01026   err = internal_ds18b20_select_device(handle);
01027   if (err != k_rx_ok) {
01028     return err;
01029   }
01030
01031   /* Send Read Power Supply command */
01032   err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01033   if (err != k_rx_ok) {
01034     rx_log_error(s_tag, "Failed to send read power command");
01035     return err;
01036   }
01037
01038   /* Read power bit (1 = external, 0 = parasitic) */
01039   *external_power = false;
01040   err             = rx_bus_onewire_read_bit(handle->bus_manager, handle->bus_name, external_power);
01041   if (err != k_rx_ok) {
01042     rx_log_error(s_tag, "Failed to read power bit");
01043     return err;
01044   }
01045
01046   return k_rx_ok;
01047 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_read_power, and s_tag.

Here is the call graph for this function:



6.1.15.6 rx_ds18b20_read_scratchpad()

```
rx_err_t rx_ds18b20_read_scratchpad (
                rx_ds18b20_handle_t * handle,
                uint8_t scratchpad[k_ds18b20_scratchpad_bytes])   [nodiscard]
```

Read scratchpad memory.

Reads the entire 9-byte scratchpad (8 data bytes + 1 CRC byte). Performs CRC validation.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	scratchpad	Pointer to 9-byte buffer. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if any pointer is nullptr

k_rx_err_invalid_state if not initialized or device not present

k_rx_err_crc if CRC check fails

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

Byte	Field	Description
0	Temperature LSB	Low byte of temperature register
1	Temperature MSB	High byte of temperature register
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config Register	Resolution configuration byte
5-7	Reserved	Reserved (0xFF, 0x10, 0x10)
8	CRC	CRC-8 of bytes 0-7

CRC is verified internally by internal_ds18b20_read_scratchpad_raw() before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to internal_ds18b20_read_scratchpad_raw() which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Precondition

handle must be initialized via rx_ds18b20_init()

scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes

Postcondition

scratchpad[0..8] contains valid data on k_rx_ok

CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok

Note

Not thread-safe; caller must serialize access to the same handle

Use rx_ds18b20_read_temperature() for converted Celsius value

Example:

```
uint8_t raw[k_ds18b20_scratchpad_bytes];
rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
if (err == k_rx_ok) {
    uint8_t config_reg = raw[4];
}
```

See also

rx_ds18b20_read_temperature() High-level temperature read returning float Celsius

rx_ds18b20_read_temperature_raw() Read raw 16-bit temperature with masking

Since

Version 1.0.0

NASA Power of 10 Compliance:

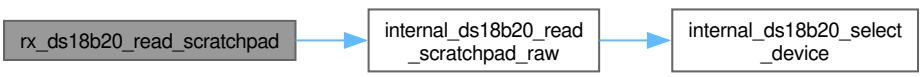
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 1104 of file rx_ds18b20.c.

```
01106 {
01107     CHECK_DS18B20_HANDLE(handle, s_tag);
01108     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01109
01110     return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01111 }
```

References CHECK_DS18B20_HANDLE, internal_ds18b20_read_scratchpad_raw(), k_ds18b20_scratchpad_bytes, and s_tag.

Here is the call graph for this function:



6.1.15.7 rx_ds18b20_read_temperature()

```
rx_err_t rx_ds18b20_read_temperature (
                rx_ds18b20_handle_t * handle,
                float * temperature_celsius)   [nodiscard]
```

Read temperature in Celsius.

Reads the scratchpad and converts raw temperature to Celsius. Temperature conversion must complete before calling (see conversion times).

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	temperature_celsius	Pointer to store temperature in degC. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if any pointer is nullptr

k_rx_err_invalid_state if not initialized or device not present

k_rx_err_crc if scratchpad CRC check fails

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call rx_ds18b20_read_temperature_raw() to obtain masked raw value
3. Convert raw signed 16-bit value to float: celsius = raw / 16.0f
4. Write result to *temperature_celsius

Precondition

handle must be initialized via rx_ds18b20_init()

rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed

Postcondition

*temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok

handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
float temp_celsius = 0.0f;
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
if (err == k_rx_ok) {
    rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
}
```

See also

rx_ds18b20_trigger_conversion() Must be called before reading

rx_ds18b20_read_temperature_raw() Read the raw 16-bit value

rx_ds18b20_get_conversion_time_ms() Determine required wait time

Since

Version 1.0.0

NASA Power of 10 Compliance:

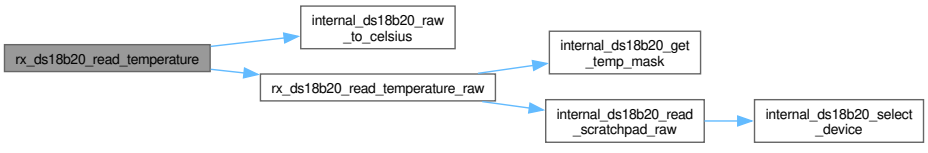
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 572 of file rx_ds18b20.c.

```
00573 {
00574   CHECK_DS18B20_HANDLE(handle, s_tag);
00575   RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00576
00577   /* Read raw temperature */
00578   int16_t  raw_temp = 0;
00579   rx_err_t err      = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00580   if (err != k_rx_ok) {
00581     return err;
00582   }
00583
00584   /* Convert to Celsius */
00585   *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00586
00587   return k_rx_ok;
00588 }
```

References CHECK_DS18B20_HANDLE, internal_ds18b20_raw_to_celsius(), rx_ds18b20_read_temperature_raw(), and s_tag.

Here is the call graph for this function:



6.1.15.8 rx_ds18b20_read_temperature_raw()

```
rx_err_t rx_ds18b20_read_temperature_raw (
                rx_ds18b20_handle_t * handle,
                int16_t * raw_temp)   [nodiscard]
```

Read raw temperature value.

Reads the raw 16-bit temperature from scratchpad without conversion. Useful for debugging or custom temperature processing.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	raw_temp	Pointer to store raw temperature. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if any pointer is nullptr

k_rx_err_invalid_state if not initialized or device not present

k_rx_err_crc if scratchpad CRC check fails

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed int16_t, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)
- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFF8 (3 undefined bits)
- 10-bit: 0xFFFC (2 undefined bits)
- 11-bit: 0xFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via internal_ds18b20_read_scratchpad_raw()
3. Combine temperature LSB and MSB bytes into uint16_t
4. Apply resolution mask from internal_ds18b20_get_temp_mask()
5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
6. Write validated result to *raw_temp

Precondition

handle must be initialized via rx_ds18b20_init()

rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed

Postcondition

∗raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok

Resolution mask has been applied; undefined bits are cleared

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
int16_t raw = 0;
rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
if (err == k_rx_ok) {
    float celsius = (float)raw / 16.0f;
}
```

See also

rx_ds18b20_read_temperature() Convenience wrapper returning float Celsius

rx_ds18b20_trigger_conversion() Must be called before reading

rx_ds18b20_set_resolution() Configure ADC resolution and mask

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

Definition at line 646 of file rx_ds18b20.c.

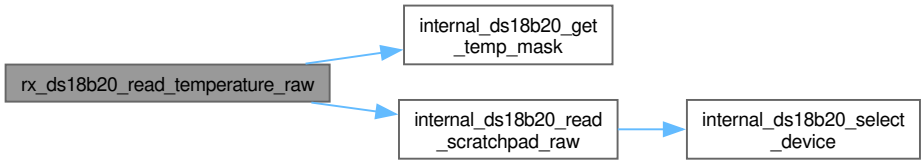
```
00647 {
00648   CHECK_DS18B20_HANDLE(handle, s_tag);
00649   RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00650
00651   /* Read scratchpad */
00652   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00653   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00654   if (err != k_rx_ok) {
00655     return err;
00656   }
00657
00658   /* Extract temperature (LSB + MSB) */
00659   uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00660   temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) « k_ds18b20_shift_byte;
00661
00662   /* Apply resolution mask to clear undefined bits */
00663   const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
```

```
00664   temp &= mask;
00665
00666   /* Post-condition: Validate temperature is within sensor range */
00667   *raw_temp = (int16_t)temp;
00668   if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00669     rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00670     return k_rx_err_out_of_range;
00671   }
00672
00673   return k_rx_ok;
00674 }
```

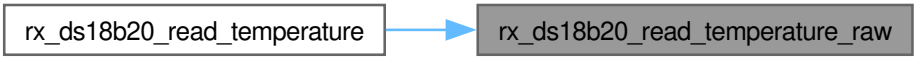
References CHECK_DS18B20_HANDLE, internal_ds18b20_get_temp_mask(), internal_ds18b20_read_scratchpad_r k_ds18b20_scratch_temp_lsb, k_ds18b20_scratch_temp_msb, k_ds18b20_scratchpad_bytes, k_ds18b20_shift_byte, k_ds18b20_temp_max_raw, rx_ds18b20_handle_t::resolution, and s_tag.

Referenced by rx_ds18b20_read_temperature().

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.15.9 rx˙ds18b20˙recall˙config()

```
rx_err_t rx_ds18b20_recall_config (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Recall configuration from EEPROM.

Restores scratchpad bytes 2-4 (TH, TL, Config) from EEPROM. Useful after power-up to restore saved settings.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle is nullptr

k_rx_err_invalid_state if not initialized or device not present

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected to the bus

Postcondition

DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded

Note

Not thread-safe; caller must serialize access to the same handle

The Recall E2 operation is performed automatically by the device on power-up

Example:

```
// Revert any unsaved configuration changes
rx_err_t err = rx_ds18b20_recall_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
}
```

See also

rx_ds18b20_save_config() Persist scratchpad configuration to EEPROM

rx_ds18b20_set_resolution() Modify the scratchpad configuration

Since

Version 1.0.0

NASA Power of 10 Compliance:

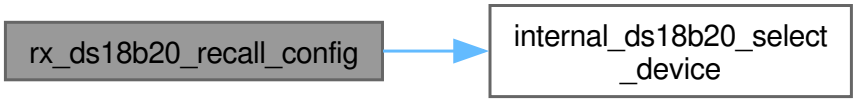
- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 936 of file rx_ds18b20.c.

```
00937 {
00938   CHECK_DS18B20_HANDLE(handle, s_tag);
00939
00940   /* Reset and check presence */
00941   bool     presence = false;
00942   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00943   if (err != k_rx_ok || !presence) {
00944     rx_log_error(s_tag, "Device not present for EEPROM recall");
00945     return k_rx_err_invalid_state;
00946   }
00947
00948   /* Select device */
00949   err = internal_ds18b20_select_device(handle);
00950   if (err != k_rx_ok) {
00951     return err;
00952   }
00953
00954   /* Send Recall E2 command */
00955   err =
00956     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
00957   if (err != k_rx_ok) {
00958     rx_log_error(s_tag, "Failed to send recall EEPROM command");
00959     return err;
00960   }
00961
00962   return k_rx_ok;
00963 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_recall_eeprom, and s_tag.

Here is the call graph for this function:



6.1.15.10 rx_ds18b20_save_config()

```
rx_err_t rx_ds18b20_save_config (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Save configuration to EEPROM.

Copies scratchpad bytes 2-4 (TH, TL, Config) to non-volatile EEPROM. Values persist across power cycles.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle is nullptr

k_rx_err_invalid_state if not initialized or device not present

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Postcondition

DS18B20 EEPROM contains current TH, TL, and configuration register values

Configuration will survive next power cycle

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Not reliable in parasitic power mode; use external power for EEPROM writes

Example:

```
// Configure resolution, then persist to EEPROM
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
rx_err_t err = rx_ds18b20_save_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
}
```

See also

rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad

rx_ds18b20_set_resolution() Configure resolution before saving

Since

Version 1.0.0

NASA Power of 10 Compliance:

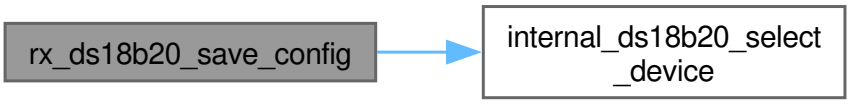
- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 862 of file rx_ds18b20.c.

```
00863 {
00864   CHECK_DS18B20_HANDLE(handle, s_tag);
00865
00866   /* Reset and check presence */
00867   bool     presence = false;
00868   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00869   if (err != k_rx_ok || !presence) {
00870     rx_log_error(s_tag, "Device not present for EEPROM save");
00871     return k_rx_err_invalid_state;
00872   }
00873
00874   /* Select device */
00875   err = internal_ds18b20_select_device(handle);
00876   if (err != k_rx_ok) {
00877     return err;
00878   }
00879
00880   /* Send Copy Scratchpad command */
00881   err =
00882     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00883   if (err != k_rx_ok) {
00884     rx_log_error(s_tag, "Failed to send copy scratchpad command");
00885     return err;
00886   }
00887
00888   return k_rx_ok;
00889 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_copy_scratch, and s_tag.

Here is the call graph for this function:



6.1.15.11 rx_ds18b20_set_resolution()

```
rx_err_t rx_ds18b20_set_resolution (
                rx_ds18b20_handle_t * handle,
                const ds18b20_resolution_t resolution)   [nodiscard]
```

Set temperature resolution.

Changes the sensor resolution (9, 10, 11, or 12 bits). Higher resolution provides more precision but takes longer to convert.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
in	resolution	Resolution mode (9-12 bits)

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle is nullptr

k_rx_err_invalid_arg if resolution out of range

k_rx_err_invalid_state if not initialized or device not present

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion
- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via internal_ds18b20_resolution_to_config()
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Precondition

handle must be initialized via rx_ds18b20_init()

resolution must be one of k_ds18b20_resolution_9bit/10bit/11bit/12bit

Postcondition

handle->resolution reflects the newly configured value on k_rx_ok

DS18B20 scratchpad configuration register updated with new resolution bits

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Change is not persistent across power cycles unless rx_ds18b20_save_config() is called

Example:

```
rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
if (err == k_rx_ok) {
    // Save to EEPROM so it persists across power cycles
    rx_ds18b20_save_config(&sensor);
}
```

See also

rx_ds18b20_get_resolution() Read back the currently configured resolution

rx_ds18b20_save_config() Persist resolution change to EEPROM

rx_ds18b20_get_conversion_time_ms() Get required conversion time for new resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

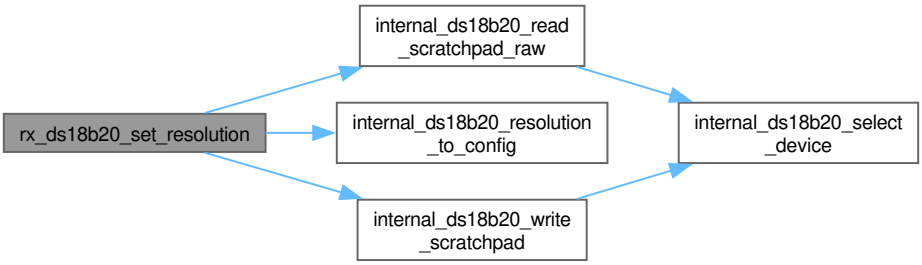
Definition at line 728 of file rx_ds18b20.c.

```
00730 {
00731   CHECK_DS18B20_HANDLE(handle, s_tag);
00732
00733   if (resolution != k_ds18b20_resolution_9bit && resolution != k_ds18b20_resolution_10bit &&
00734       resolution != k_ds18b20_resolution_11bit && resolution != k_ds18b20_resolution_12bit) {
00735     rx_log_error(s_tag, "Invalid resolution value");
00736     return k_rx_err_invalid_arg;
00737   }
00738
00739   /* Read current scratchpad */
00740   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00741   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00742   if (err != k_rx_ok) {
00743     return err;
00744   }
00745
00746   /* Generate new config byte */
00747   const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00748
00749   /* Write scratchpad with new config */
00750   ds18b20_scratchpad_write_t write_cfg;
00751   write_cfg.th     = scratchpad[k_ds18b20_scratch_th_reg];
00752   write_cfg.tl     = scratchpad[k_ds18b20_scratch_tl_reg];
00753   write_cfg.config = config;
00754   err              = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00755   if (err != k_rx_ok) {
00756     return err;
00757   }
00758
00759   /* Update handle */
00760   handle->resolution = resolution;
00761
00762   return k_rx_ok;
00763 }
```

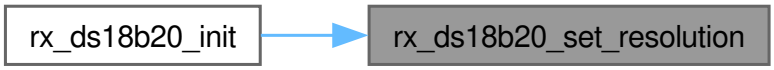
References CHECK_DS18B20_HANDLE, ds18b20_scratchpad_write_t::config, internal_ds18b20_read_scratchpad_ra internal_ds18b20_resolution_to_config(), internal_ds18b20_write_scratchpad(), k_ds18b20_resolution_10bit, k_ds18b20_resolution_11bit, k_ds18b20_resolution_12bit, k_ds18b20_resolution_9bit, k_ds18b20_scratch_th_reg, k_ds18b20_scratch_tl_reg, k_ds18b20_scratchpad_bytes, rx_ds18b20_handle_t::resolution, s_tag, ds18b20_scratchpad_write_t::th, and ds18b20_scratchpad_write_t::tl.

Referenced by rx_ds18b20_init().

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.15.12 rx˙ds18b20˙trigger˙conversion()

```
rx_err_t rx_ds18b20_trigger_conversion (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Trigger temperature conversion.

Sends Convert T command to start temperature measurement. Caller must wait for conversion time before reading temperature.

Conversion times:

- 9-bit: 100ms
- 10-bit: 200ms
- 11-bit: 400ms
- 12-bit: 800ms

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle is nullptr

k_rx_err_invalid_state if not initialized or device not present

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with rx_ds18b20_read_temperature().

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected and powered on the bus

Postcondition

DS18B20 ADC conversion is in progress

Caller must wait rx_ds18b20_get_conversion_time_ms() before reading

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Do not call rx_ds18b20_read_temperature() before the conversion time elapses

Example:

```
rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
if (err == k_rx_ok) {
    tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
    float temp_celsius = 0.0f;
    err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
}
```

See also

rx_ds18b20_read_temperature() Read converted temperature in Celsius

rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion

rx_ds18b20_set_resolution() Configure ADC resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

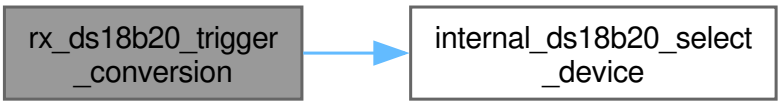
- Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

Definition at line 500 of file rx_ds18b20.c.

```
00501 {
00502   CHECK_DS18B20_HANDLE(handle, s_tag);
00503
00504   /* Reset and check presence */
00505   bool     presence = false;
00506   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00507   if (err != k_rx_ok || !presence) {
00508     rx_log_error(s_tag, "Device not present for conversion trigger");
00509     return k_rx_err_invalid_state;
00510   }
00511
00512   /* Select device */
00513   err = internal_ds18b20_select_device(handle);
00514   if (err != k_rx_ok) {
00515     return err;
00516   }
00517
00518   /* Send Convert T command */
00519   err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00520   if (err != k_rx_ok) {
00521     rx_log_error(s_tag, "Failed to send conversion command");
00522     return err;
00523   }
00524
00525   return k_rx_ok;
00526 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_convert_t, and s_tag.

Here is the call graph for this function:



6.2 rx˙ds18b20.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_ds18b20.h
00003  * @brief DS18B20 1-Wire Digital Temperature Sensor Driver API
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Platform-independent driver for Maxim DS18B20 programmable resolution 1-Wire
00009  * digital thermometer with CRC-8 data integrity checking. Provides high-level
00010  * API for temperature measurement, resolution configuration, and EEPROM
00011  * persistence with automatic error handling and validation.
00012  *
00013  * This driver abstracts the DS18B20's complex 1-Wire protocol into simple init,
00014  * trigger, and read operations while maintaining full access to advanced features
00015  * like ROM matching (multi-drop buses) and alarm thresholds.
00016  *
00017  * # Key Features
00018  *
00019  * ## Measurement Capabilities
00020  * - **Temperature range**: -55degC to +125degC with +/-0.5degC accuracy (-10degC to +85degC)
00021  * - **Programmable resolution**: 9, 10, 11, or 12 bits
00022  * - **Conversion times**: 94ms (9-bit) to 750ms (12-bit)
00023  * - **CRC-8 validation**: Detects transmission errors (polynomial x^8 + x^5 + x^4 + 1)
00024  *
00025  * ## Resolution vs. Precision vs. Conversion Time Trade-off
00026  *
00027  * | Resolution | Precision | Conversion Time | Use Case |
00028  * |------------|-----------|-----------------|----------|
00029  * | 9-bit      | 0.5degC     | ~94ms           | Fast updates, low precision needed |
00030  * | 10-bit     | 0.25degC    | ~188ms          | Balanced for most applications |
00031  * | 11-bit     | 0.125degC   | ~375ms          | High precision, moderate speed |
00032  * | 12-bit     | 0.0625degC  | ~750ms          | Maximum precision, slow updates |
00033  *
00034  * ## Advanced Features
00035  * - **ROM matching**: Supports multi-device 1-Wire buses (up to 255 sensors)
00036  * - **Skip ROM mode**: Optimized for single-sensor configurations
00037  * - **EEPROM storage**: Persist resolution/alarm settings across power cycles
00038  * - **Power mode detection**: Identify parasitic vs. external power operation
00039  * - **Alarm thresholds**: Programmable TH/TL registers (not exposed in this API)
00040  *
00041  * # System Architecture Context
00042  *
00043  * The DS18B20 driver sits at the sensor layer of the STAR firmware stack:
00044  *
00045  * @code{.unparsed}
00046  * +-----------------------------------------------+
00047  * | Application Layer (Tasks)                     |
00048  * | - Temperature monitoring task                 |
00049  * | - Environmental control                       |
00050  * +---------------+-------------------------------+
00051  *                 |
00052  * +---------------v-------------------------------+
00053  * | Sensor Driver Layer                           |
00054  * | +-------------------------------------------+ |
00055  * | | rx_ds18b20 (This Module)                  | |
00056  * | | - Handle management                       | |
00057  * | | - CRC validation                          | |
00058  * | | - Temperature conversion                  | |
00059  * | +-------------+-----------------------------+ |
00060  * +---------------+-------------------------------+
00061  *                 |
00062  * +---------------v-------------------------------+
00063  * | Bus Manager Layer (rx_bus_manager)            |
00064  * | - Abstraction for I2C, SPI, 1-Wire, etc.      |
00065  * | - Dependency injection for testability        |
00066  * +---------------+-------------------------------+
00067  *                 |
00068  * +---------------v-------------------------------+
00069  * | 1-Wire HAL (rx_bus_onewire)                   |
00070  * | - Bit-level 1-Wire protocol                   |
00071  * | - Reset/presence detection                    |
00072  * | - ROM commands (Match/Skip/Search)            |
00073  * +---------------+-------------------------------+
00074  *                 |
00075  * +---------------v-------------------------------+
00076  * | Hardware Layer (GPIO, Timers)                 |
00077  * | - Microsecond-precision timing                |
00078  * | - Open-drain GPIO control                     |
00079  * +-----------------------------------------------+
00080  * @endcode
00081  *
00082  * # Design Rationale
```

```
00083  *
00084  * ## Dependency Injection Pattern
00085  *
00086  * The driver accepts a `rx_bus_manager_t*` instead of directly accessing GPIO,
00087  * enabling:
00088  * - **Unit testing** with mock bus implementations (no hardware required)
00089  * - **Hardware independence** - works on any 1-Wire HAL implementation
00090  * - **Multi-bus support** - single driver code for multiple 1-Wire buses
00091  *
00092  * This follows the **Dependency Inversion Principle** (high-level modules depend
00093  * on abstractions, not concrete implementations).
00094  *
00095  * ## Blocking API Design
00096  *
00097  * All operations are blocking (caller must wait for conversion). Rationale:
00098  * - DS18B20 conversions are inherently slow (up to 750ms)
00099  * - Non-blocking would require state machines and polling infrastructure
00100  * - Simplicity favored: caller can use ThreadX semaphores if async needed
00101  * - Typical usage: Dedicated temperature monitoring task with periodic reads
00102  *
00103  * ## Error Handling Strategy
00104  *
00105  * - **All functions return `rx_err_t`** for consistent error propagation
00106  * - **CRC validation on every read** - detect bus noise/corruption
00107  * - **Range checking** - reject impossible temperatures (-880 to +2000 raw)
00108  * - **nullptr checks** - fail fast on API misuse
00109  * - **State validation** - ensure init before use, prevent double-init
00110  *
00111  * # Implementation Approach
00112  *
00113  * ## Temperature Conversion Process
00114  *
00115  * The DS18B20 uses a two-phase measurement cycle:
00116  *
00117  * 1. **Trigger Phase** (`rx_ds18b20_trigger_conversion()`):
00118  *    - Send Convert T command (0x44)
00119  *    - DS18B20 begins analog-to-digital conversion
00120  *    - Returns immediately (non-blocking)
00121  *
00122  * 2. **Wait Phase** (Caller's responsibility):
00123  *    - Wait for conversion time based on resolution
00124  *    - Use `rx_ds18b20_get_conversion_time_ms()` to get exact time
00125  *    - ThreadX: `tx_thread_sleep(time_ms * TX_TIMER_TICKS_PER_SECOND / 1000)`
00126  *
00127  * 3. **Read Phase** (`rx_ds18b20_read_temperature()`):
00128  *    - Read 9-byte scratchpad (includes CRC)
00129  *    - Validate CRC-8 checksum
00130  *    - Extract 16-bit raw temperature
00131  *    - Apply resolution mask (clear undefined bits)
00132  *    - Convert to Celsius (raw / 16.0)
00133  *
00134  * ## Memory Layout and Scratchpad Structure
00135  *
00136  * The DS18B20's scratchpad RAM is 9 bytes:
00137  *
00138  * | Byte | Name         | Description                    |
00139  * |------|--------------|--------------------------------|
00140  * | 0    | Temp LSB     | Temperature low byte           |
00141  * | 1    | Temp MSB     | Temperature high byte (+ sign) |
00142  * | 2    | TH Register  | High alarm threshold           |
00143  * | 3    | TL Register  | Low alarm threshold            |
00144  * | 4    | Config       | Resolution bits [6:5]          |
00145  * | 5    | Reserved     | 0xFF                           |
00146  * | 6    | Reserved     | Factory value                  |
00147  * | 7    | Reserved     | Factory value                  |
00148  * | 8    | CRC          | CRC-8 of bytes 0-7             |
00149  *
00150  * Configuration register format (byte 4):
00151  * @code
00152  * Bit:   7   6   5   4   3   2   1   0
00153  *        0  R1  R0   1   1   1   1   1
00154  *           +---+- Resolution: 00=9b, 01=10b, 10=11b, 11=12b
00155  * @endcode
00156  *
00157  * # Performance Characteristics
00158  *
00159  * ## Timing Analysis
00160  *
00161  * | Operation             | Duration        | Notes                    |
00162  * |-----------------------|-----------------|--------------------------|
00163  * | Init                  | ~10ms           | Presence detect + read   |
00164  * | Trigger conversion    | ~1ms            | Command transmission     |
00165  * | Conversion (9-bit)    | 93.75ms (spec)  | 100ms recommended        |
00166  * | Conversion (10-bit)   | 187.5ms (spec)  | 200ms recommended        |
00167  * | Conversion (11-bit)   | 375ms (spec)    | 400ms recommended        |
00168  * | Conversion (12-bit)   | 750ms (spec)    | 800ms recommended        |
00169  * | Read scratchpad       | ~2ms            | 9 bytes + CRC validation |
```

```
00170 * | Set resolution        | ~5ms            | Read + modify + write    |
00171 *
00172 * **Note**: Recommended times include 10% safety margin above datasheet specs.
00173 *
00174 * ## Memory Usage
00175 *
00176 * | Component              | Size (bytes) | Location     |
00177 * |------------------------|--------------|--------------|
00178 * | `rx_ds18b20_handle_t`  | 28           | Caller stack |
00179 * | `rx_ds18b20_config_t`  | 24           | Caller stack |
00180 * | Scratchpad buffer      | 9            | Stack (temp) |
00181 * | Driver code (.text)    | ~2KB         | Flash        |
00182 * | Constant data (.rodata)| ~100 bytes   | Flash        |
00183 *
00184 * **Total RAM per sensor**: 28 bytes (handle only, no dynamic allocation)
00185 *
00186 * # Hardware Requirements
00187 *
00188 * | Resource       | Requirement                                          |
00189 * |----------------|------------------------------------------------------|
00190 * | **CPU**        | Any (architecture-independent)                       |
00191 * | **RAM**        | 28 bytes per sensor + 9-byte temp buffer             |
00192 * | **Flash**      | ~2KB code + ~100 bytes constants                     |
00193 * | **GPIO**       | 1 pin (open-drain or push-pull with resistor)        |
00194 * | **Timing**     | Microsecond-precision delays (1-Wire protocol)       |
00195 * | **Pull-up**    | 4.7kOhm resistor to VCC (external)                   |
00196 * | **Power**      | 3.0V-5.5V (parasitic mode: data pin can power)       |
00197 *
00198 * ## Typical Wiring (Single Sensor)
00199 *
00200 * @code{.unparsed}
00201 *     VCC (3.3V or 5V)
00202 *      |
00203 *      +-----+------ 4.7kOhm pull-up resistor
00204 *      |     |
00205 *   +--+-----+--+
00206 *   | DS18B20   |
00207 *   |           |
00208 *   |  DQ  VDD  |
00209 *   +---+---+---+
00210 *       |   |
00211 *       |   +------ VCC (or GND for parasitic mode)
00212 *       |
00213 *       +---------- MCU GPIO (data line)
00214 *
00215 *   GND ---------- GND
00216 * @endcode
00217 *
00218 * # Example Usage Patterns
00219 *
00220 * ## Example 1: Basic Single-Sensor Configuration
00221 *
00222 * @code
00223 * #include "rx_ds18b20.h"
00224 * #include "rx_bus_manager.h"
00225 * #include "tx_api.h"
00226 *
00227 * void temperature_task_entry(ULONG input) {
00228 *     rx_ds18b20_handle_t sensor;
00229 *     rx_ds18b20_config_t config = {
00230 *         .bus_manager = &g_bus_manager,
00231 *         .bus_name = "onewire0",
00232 *         .resolution = k_ds18b20_resolution_12bit,  // 0.0625degC precision
00233 *         .use_rom_matching = false,                 // Skip ROM (single sensor)
00234 *     };
00235 *
00236 *     rx_err_t err = rx_ds18b20_init(&sensor, &config);
00237 *     if (err != k_rx_ok) {
00238 *         // Handle error: check bus wiring, power, pull-up resistor
00239 *         return;
00240 *     }
00241 *
00242 *     while (1) {
00243 *         // Trigger conversion
00244 *         err = rx_ds18b20_trigger_conversion(&sensor);
00245 *         if (err != k_rx_ok) continue;
00246 *
00247 *         // Wait for conversion (800ms for 12-bit)
00248 *         tx_thread_sleep(80);  // 800ms at 100 Hz tick rate
00249 *
00250 *         // Read temperature
00251 *         float temp_celsius;
00252 *         err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00253 *         if (err == k_rx_ok) {
00254 *             // Process temperature (e.g., log, control, transmit)
00255 *             printf("Temperature: %.2fdegC\n", temp_celsius);
00256 *         } else if (err == k_rx_err_crc_mismatch) {
```

```
00257 *           // CRC error: retry or log fault
00258 *       }
00259 *
00260 *       // Repeat every 5 seconds
00261 *       tx_thread_sleep(500);
00262 *   }
00263 * }
00264 * @endcode
00265 *
00266 * ## Example 2: Multi-Sensor Configuration (ROM Matching)
00267 *
00268 * @code
00269 * // Pre-discovered ROM codes (use ROM search to find these)
00270 * uint8_t rom_sensor1[8] = {0x28, 0xFF, 0x12, 0x34, 0x56, 0x78, 0x90, 0xAB};
00271 * uint8_t rom_sensor2[8] = {0x28, 0xFF, 0xAB, 0xCD, 0xEF, 0x01, 0x23, 0x45};
00272 *
00273 * rx_ds18b20_handle_t sensors[2];
00274 * rx_ds18b20_config_t config_template = {
00275 *     .bus_manager = &g_bus_manager,
00276 *     .bus_name = "onewire0",
00277 *     .resolution = k_ds18b20_resolution_11bit,
00278 *     .use_rom_matching = true,  // Enable ROM matching for multi-drop
00279 * };
00280 *
00281 * // Init sensor 1
00282 * memcpy(config_template.rom, rom_sensor1, 8);
00283 * rx_ds18b20_init(&sensors[0], &config_template);
00284 *
00285 * // Init sensor 2
00286 * memcpy(config_template.rom, rom_sensor2, 8);
00287 * rx_ds18b20_init(&sensors[1], &config_template);
00288 *
00289 * // Trigger both simultaneously (broadcast not shown, or trigger individually)
00290 * rx_ds18b20_trigger_conversion(&sensors[0]);
00291 * rx_ds18b20_trigger_conversion(&sensors[1]);
00292 *
00293 * tx_thread_sleep(40);  // Wait 400ms for 11-bit
00294 *
00295 * // Read both
00296 * float temps[2];
00297 * rx_ds18b20_read_temperature(&sensors[0], &temps[0]);
00298 * rx_ds18b20_read_temperature(&sensors[1], &temps[1]);
00299 * @endcode
00300 *
00301 * ## Example 3: Error Handling and Diagnostics
00302 *
00303 * @code
00304 * rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_c);
00305 *
00306 * switch (err) {
00307 *     case k_rx_ok:
00308 *         // Success: use temp_c
00309 *         break;
00310 *
00311 *     case k_rx_err_crc_mismatch:
00312 *         // CRC error: possible bus noise, retry
00313 *         retry_count++;
00314 *         break;
00315 *
00316 *     case k_rx_err_invalid_state:
00317 *         // Sensor disconnected or not responding
00318 *         // Check: wiring, power, pull-up resistor
00319 *         break;
00320 *
00321 *     case k_rx_err_out_of_range:
00322 *         // Temperature outside -55degC to +125degC
00323 *         // Sensor fault or extreme environment
00324 *         break;
00325 *
00326 *     default:
00327 *         // Unexpected error
00328 *         break;
00329 * }
00330 * @endcode
00331 *
00332 * ## Example 4: Runtime Resolution Change
00333 *
00334 * @code
00335 * // Switch to 9-bit for faster updates (0.5degC precision)
00336 * rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_9bit);
00337 *
00338 * // Persist to EEPROM (survives power cycle)
00339 * rx_ds18b20_save_config(&sensor);
00340 *
00341 * // Now trigger with shorter wait time
00342 * rx_ds18b20_trigger_conversion(&sensor);
00343 * tx_thread_sleep(10);  // 100ms for 9-bit
```

```
00344  * rx_ds18b20_read_temperature(&sensor, &temp_c);
00345  * @endcode
00346  *
00347  * # Thread Safety
00348  *
00349  * **Not thread-safe**. Caller must serialize access if multiple threads use the
00350  * same sensor handle. Use ThreadX mutexes:
00351  *
00352  * @code
00353  * TX_MUTEX sensor_mutex;
00354  * tx_mutex_create(&sensor_mutex, "DS18B20", TX_NO_INHERIT);
00355  *
00356  * // Thread 1:
00357  * tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
00358  * rx_ds18b20_trigger_conversion(&sensor);
00359  * tx_mutex_put(&sensor_mutex);
00360  *
00361  * // Thread 2:
00362  * tx_mutex_get(&sensor_mutex, TX_WAIT_FOREVER);
00363  * rx_ds18b20_read_temperature(&sensor, &temp_c);
00364  * tx_mutex_put(&sensor_mutex);
00365  * @endcode
00366  *
00367  * **Note**: Different sensor handles (different physical sensors) CAN be accessed
00368  * concurrently without synchronization, even on the same 1-Wire bus (ROM matching
00369  * addresses each sensor independently).
00370  *
00371  * # Known Limitations
00372  *
00373  * 1. **No alarm threshold API**: TH/TL registers accessible via scratchpad but
00374  *    not exposed. Users can read/write scratchpad directly if needed.
00375  *
00376  * 2. **No broadcast/simultaneous conversion**: Must trigger each sensor individually
00377  *    when using ROM matching. Skip ROM mode allows broadcast but only for single-sensor.
00378  *
00379  * 3. **No ROM search**: Must provide ROM codes manually for multi-sensor. Consider
00380  *    adding `rx_ds18b20_discover()` helper in future.
00381  *
00382  * 4. **Blocking API only**: No asynchronous/callback interface. Caller must use
00383  *    RTOS primitives (tasks, timers) for non-blocking behavior.
00384  *
00385  * # References and Standards
00386  *
00387  * - **DS18B20 Datasheet**: Maxim Integrated (now Analog Devices)
00388  *   https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf
00389  * - **1-Wire Protocol**: Book of iButton Standards, Maxim Integrated
00390  * - **CRC-8 Polynomial**: x^8 + x^5 + x^4 + 1 (standard for 1-Wire devices)
00391  * - **ROM Structure**: 8-byte format (1-byte family + 6-byte serial + 1-byte CRC)
00392  * - **IEEE 802.3 Ethernet CRC**: Different from DS18B20's CRC-8, not applicable here
00393  *
00394  * @see rx_bus_manager.h - Bus manager abstraction layer
00395  * @see rx_bus_onewire.h - 1-Wire protocol implementation
00396  * @see rx_crc.h - CRC calculation utilities (CRC-8 for DS18B20)
00397  * @see docs/sections/03_hardware_pinout.tex - GPIO pin assignments
00398  *
00399  * @par NASA Power of 10 Compliance:
00400  * - **Rule 1** (No goto): [OK] All control flow uses structured statements
00401  * - **Rule 2** (Bounded loops): [OK] All loops have compile-time or runtime bounds
00402  * - **Rule 3** (No heap): [OK] Zero dynamic allocation (static handles only)
00403  * - **Rule 4** (Functions <=60 lines): [OK] All public functions <60 lines
00404  * - **Rule 5** (Assertions): [OK] 2+ preconditions per function (NULL checks, state)
00405  * - **Rule 6** (Data scope): [OK] Variables declared at smallest scope
00406  * - **Rule 7** (Return checking): [OK] All HAL returns validated
00407  * - **Rule 8** (Preprocessor limits): [OK] C23 typed enums only, minimal macros
00408  * - **Rule 9** (Pointer restrictions): [WARN] Intentional deviation for DIP (bus_manager)
00409  * - **Rule 10** (Compiler warnings): [OK] -Wall -Wextra -Werror, zero warnings
00410  *
00411  * @par SOLID Principles:
00412  * - **S (Single Responsibility)**: Module handles ONLY DS18B20 sensor operations
00413  * - **O (Open/Closed)**: Extensible via bus_manager (add new buses w/o changes)
00414  * - **L (Liskov Substitution)**: Any rx_bus_manager_t* is interchangeable
00415  * - **I (Interface Segregation)**: Focused API (temp read, no unrelated operations)
00416  * - **D (Dependency Inversion)**: Depends on rx_bus_manager_t abstraction, not GPIO
00417  *
00418  * @par Module Dependencies:
00419  * @code{.unparsed}
00420  * rx_ds18b20
00421  *   +-- rx_bus_manager (bus abstraction, injected)
00422  *   +-- rx_bus_onewire (1-Wire protocol layer)
00423  *   +-- rx_crc (CRC-8 validation)
00424  *   +-- rx_err (error codes)
00425  *   +-- rx_log (diagnostic logging)
00426  *   +-- rx_check (nullptr validation macros)
00427  * @endcode
00428  *
00429  * @author Locked, Inc.
00430  * @date 2026-01-30
```

```
00431  * @version 1.0.0
00432  * @copyright Copyright (c) 2026 Locked Inc.
00433  * SPDX-License-Identifier: MIT
00434  */
00435
00436 #pragma once
00437
00438 #include <stdbool.h>
00439
00440 #ifdef __cplusplus
00441 extern "C" {
00442 #endif
00443
00444 #include <stdint.h>
00445
00446 #include "rx_bus_manager.h"
00447 #include "rx_bus_onewire.h"
00448 #include "rx_err.h"
00449
00450 /*
      ============================================================================
00451  * DS18B20 Constants
00452  *
      ============================================================================
00453  */
00454
00455 /**
00456  * @enum ds18b20_command_t
00457  * @brief DS18B20 function commands per datasheet specification
00458  *
00459  * @details
00460  * Command codes sent after ROM selection (Match ROM or Skip ROM) to initiate
00461  * DS18B20 operations. Each command triggers a specific sensor function.
00462  *
00463  * **Protocol Sequence:**
00464  * 1. Reset pulse + presence detection
00465  * 2. ROM command (Match ROM 0x55 or Skip ROM 0xCC)
00466  * 3. Function command (one of these values)
00467  * 4. Additional data bytes (if required by command)
00468  *
00469  * **Command Details:**
00470  *
00471  * | Command              | Code | Data Bytes | Duration    | Description                    |
00472  * |--------------------|------|------------|------------|--------------------------------|
00473  * | Convert T          | 0x44 | 0          | 94-750ms    | Start temperature conversion   |
00474  * | Write Scratchpad   | 0x4E | 3          | ~2ms        | Write TH, TL, Config registers |
00475  * | Read Scratchpad    | 0xBE | 0          | ~2ms        | Read 9-byte scratchpad + CRC   |
00476  * | Copy Scratchpad    | 0x48 | 0          | ~10ms       | Save scratchpad to EEPROM      |
00477  * | Recall E^2         | 0xB8 | 0          | ~1ms        | Restore EEPROM to scratchpad   |
00478  * | Read Power Supply  | 0xB4 | 0          | <1ms        | Check parasitic vs. external   |
00479  *
00480  * @par Usage Example:
00481  * @code
00482  * // Trigger conversion
00483  * rx_bus_onewire_reset(bus_mgr, "onewire0", &presence);
00484  * rx_bus_onewire_skip_rom(bus_mgr, "onewire0");
00485  * rx_bus_onewire_write_byte(bus_mgr, "onewire0", k_ds18b20_cmd_convert_t);
00486  * // Wait conversion time...
00487  * @endcode
00488  *
00489  * @see DS18B20 datasheet section "Function Commands" for timing specifications
00490  * @see rx_bus_onewire.h for ROM command functions
00491  *
00492  * @since Version 1.0.0
00493  */
00494 typedef enum : uint8_t {
00495     k_ds18b20_cmd_convert_t     = 0x44, /**< Trigger temperature conversion (ADC operation) */
00496     k_ds18b20_cmd_write_scratch = 0x4E, /**< Write to scratchpad memory (TH, TL, Config bytes) */
00497     k_ds18b20_cmd_read_scratch  = 0xBE, /**< Read from scratchpad memory (9 bytes: data + CRC) */
00498     k_ds18b20_cmd_copy_scratch  = 0x48, /**< Copy scratchpad to EEPROM (non-volatile storage) */
00499     k_ds18b20_cmd_recall_eeprom = 0xB8, /**< Recall EEPROM to scratchpad (restore saved values) */
00500     k_ds18b20_cmd_read_power    = 0xB4, /**< Read power supply mode (0=parasitic, 1=external) */
00501 } ds18b20_command_t;
00502
00503 /**
00504  * @enum ds18b20_scratchpad_index_t
00505  * @brief DS18B20 scratchpad memory layout indices
00506  *
00507  * @details
00508  * The DS18B20's 9-byte scratchpad RAM contains temperature data, alarm thresholds,
00509  * configuration, and a CRC-8 checksum. Use these indices to access specific bytes
00510  * when reading via Read Scratchpad command (0xBE).
00511  *
00512  * **Memory Layout:**
00513  *
00514  * | Index | Byte | Name          | Description                                    | Access    |
00515  * |-------|------|---------------|------------------------------------------------|-----------|
```

```
00516  * | 0    | 0   | Temp LSB      | Temperature low byte (bits 0-7)                | Read-only |
00517  * | 1    | 1   | Temp MSB      | Temperature high byte (sign + bits 8-11)       | Read-only |
00518  * | 2    | 2   | TH Register   | High alarm threshold (-55 to +125degC)         | R/W       |
00519  * | 3    | 3   | TL Register   | Low alarm threshold (-55 to +125degC)          | R/W       |
00520  * | 4    | 4   | Config        | Resolution bits [6:5]: 00=9b, 01=10b, 10=11b, 11=12b | R/W |
00521  * | 5    | 5   | Reserved      | Always 0xFF (ignore)                           | Read-only |
00522  * | 6    | 6   | Reserved      | Factory-programmed (ignore)                    | Read-only |
00523  * | 7    | 7   | Reserved      | Factory-programmed (ignore)                    | Read-only |
00524  * | 8    | 8   | CRC           | CRC-8 checksum of bytes 0-7                    | Read-only |
00525  *
00526  * **Temperature Format (Bytes 0-1):**
00527  *
00528  * 16-bit signed integer in 1/16degC units (two's complement):
00529  * @code
00530  * MSB (byte 1):  S  S  S  S  S  T10 T9  T8
00531  * LSB (byte 0):  T7 T6 T5 T4 T3 T2  T1  T0
00532  *                +------------------------+- Temperature bits (12-bit for 12-bit resolution)
00533  * @endcode
00534  *
00535  * - **S**: Sign bits (all 0 for positive, all 1 for negative)
00536  * - **T10-T0**: Temperature bits (11 bits used, T0-T2 undefined for lower resolutions)
00537  * - **Value range**: -880 to +2000 (represents -55.0degC to +125.0degC)
00538  * - **Conversion**: `temp_celsius = raw_value / 16.0`
00539  *
00540  * **Configuration Register (Byte 4):**
00541  *
00542  * Format: `0 R1 R0 1 1 1 1 1` where R1:R0 = resolution bits
00543  *
00544  * | R1 | R0 | Resolution | Precision | Conversion Time |
00545  * |----|----|-----------:|-----------|-----------------|
00546  * | 0  | 0  | 9-bit      | 0.5degC   | 93.75ms         |
00547  * | 0  | 1  | 10-bit     | 0.25degC  | 187.5ms         |
00548  * | 1  | 0  | 11-bit     | 0.125degC | 375ms           |
00549  * | 1  | 1  | 12-bit     | 0.0625degC| 750ms           |
00550  *
00551  * **CRC-8 Calculation (Byte 8):**
00552  *
00553  * - **Polynomial**: x^8 + x^5 + x^4 + 1 (0x8C)
00554  * - **Initial value**: 0x00
00555  * - **Data**: Bytes 0-7 (inclusive)
00556  * - **Purpose**: Detect transmission errors on 1-Wire bus
00557  *
00558  * @par Usage Example:
00559  * @code
00560  * uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
00561  * rx_ds18b20_read_scratchpad(&sensor, scratchpad);
00562  *
00563  * // Extract temperature
00564  * uint16_t temp_raw = scratchpad[k_ds18b20_scratch_temp_lsb] |
00565  *                     (scratchpad[k_ds18b20_scratch_temp_msb] << 8);
00566  *
00567  * // Validate CRC
00568  * uint32_t crc_out = 0U;
00569  * (void)rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes, &crc_out);
00570  * if ((uint8_t)crc_out != scratchpad[k_ds18b20_scratch_crc]) {
00571  *     // CRC mismatch: retry read
00572  * }
00573  *
00574  * // Read alarm thresholds
00575  * int8_t th = (int8_t)scratchpad[k_ds18b20_scratch_th_reg];
00576  * int8_t tl = (int8_t)scratchpad[k_ds18b20_scratch_tl_reg];
00577  * @endcode
00578  *
00579  * @invariant Byte 5 (reserved1) is always 0xFF
00580  * @invariant CRC (byte 8) is valid for bytes 0-7
00581  * @invariant Temperature bytes (0-1) represent value in range [-880, +2000]
00582  *
00583  * @see rx_ds18b20_read_scratchpad() Read entire scratchpad with CRC validation
00584  * @see rx_crc8_maxim() CRC-8 calculation function
00585  * @see DS18B20 datasheet section "Memory" for detailed scratchpad specification
00586  *
00587  * @since Version 1.0.0
00588  */
00589 typedef enum : uint8_t {
00590     k_ds18b20_scratch_temp_lsb  = 0, /**< Temperature LSB (bits 0-7 of 16-bit value) */
00591     k_ds18b20_scratch_temp_msb  = 1, /**< Temperature MSB (sign bits + bits 8-11) */
00592     k_ds18b20_scratch_th_reg    = 2, /**< TH (high alarm) register (signed byte, -55 to +125) */
00593     k_ds18b20_scratch_tl_reg    = 3, /**< TL (low alarm) register (signed byte, -55 to +125) */
00594     k_ds18b20_scratch_config    = 4, /**< Configuration register (resolution bits at [6:5]) */
00595     k_ds18b20_scratch_reserved1 = 5, /**< Reserved byte (always 0xFF, do not write) */
00596     k_ds18b20_scratch_reserved2 = 6, /**< Reserved byte (factory-programmed, do not write) */
00597     k_ds18b20_scratch_reserved3 = 7, /**< Reserved byte (factory-programmed, do not write) */
00598     k_ds18b20_scratch_crc       = 8, /**< CRC-8 of bytes 0-7 (polynomial 0x8C) */
00599     k_ds18b20_scratchpad_bytes  = 9, /**< Total scratchpad size in bytes */
00600 } ds18b20_scratchpad_index_t;
00601
00602 /**
```

```
00603  * @brief DS18B20 configuration register bit positions
00604  */
00605 typedef enum : uint8_t {
00606   k_ds18b20_config_r0_bit = 5, /**< Resolution bit 0 */
00607   k_ds18b20_config_r1_bit = 6, /**< Resolution bit 1 */
00608 } ds18b20_config_bits_t;
00609
00610 /**
00611  * @brief DS18B20 temperature resolution modes
00612  */
00613 typedef enum : uint8_t {
00614   k_ds18b20_resolution_9bit  = 0, /**< 9-bit: 0.5degC, 93.75ms */
00615   k_ds18b20_resolution_10bit = 1, /**< 10-bit: 0.25degC, 187.5ms */
00616   k_ds18b20_resolution_11bit = 2, /**< 11-bit: 0.125degC, 375ms */
00617   k_ds18b20_resolution_12bit = 3, /**< 12-bit: 0.0625degC, 750ms */
00618 } ds18b20_resolution_t;
00619
00620 /**
00621  * @brief DS18B20 conversion times (milliseconds)
00622  *
00623  * Maximum conversion time for each resolution.
00624  * Add margin for safety (spec values + 10%).
00625  */
00626 typedef enum : uint16_t {
00627   k_ds18b20_conv_time_9bit_ms  = 100, /**< 9-bit conversion: 100ms */
00628   k_ds18b20_conv_time_10bit_ms = 200, /**< 10-bit conversion: 200ms */
00629   k_ds18b20_conv_time_11bit_ms = 400, /**< 11-bit conversion: 400ms */
00630   k_ds18b20_conv_time_12bit_ms = 800, /**< 12-bit conversion: 800ms */
00631 } ds18b20_conversion_time_ms_t;
00632
00633 /**
00634  * @brief DS18B20 temperature masks for resolution
00635  *
00636  * Lower bits are undefined for resolutions < 12-bit and must be masked.
00637  * This prevents garbage bits from corrupting the temperature reading.
00638  */
00639 typedef enum : uint16_t {
00640   k_ds18b20_temp_mask_9bit  = 0xFFF8, /**< Mask for 9-bit (discard bits 0-2) */
00641   k_ds18b20_temp_mask_10bit = 0xFFFC, /**< Mask for 10-bit (discard bits 0-1) */
00642   k_ds18b20_temp_mask_11bit = 0xFFFE, /**< Mask for 11-bit (discard bit 0) */
00643   k_ds18b20_temp_mask_12bit = 0xFFFF, /**< Mask for 12-bit (all bits valid) */
00644 } ds18b20_temp_mask_t;
00645
00646 /**
00647  * @brief DS18B20 temperature conversion constants
00648  */
00649 typedef enum : int32_t {
00650   k_ds18b20_temp_shift            = 4,      /**< Shift to get integer temperature */
00651   k_ds18b20_sign_bit              = 0x8000, /**< Sign bit in 16-bit temperature */
00652   k_ds18b20_crc_bytes             = 8,      /**< Number of bytes for CRC calculation */
00653   k_ds18b20_shift_byte            = 8,      /**< Shift for byte positioning */
00654   k_ds18b20_scratchpad_write_bytes = 3,     /**< Scratchpad write size (TH, TL, Config) */
00655   k_ds18b20_temp_min_raw          = -880,   /**< Minimum temperature (-55degC raw value) */
00656   k_ds18b20_temp_max_raw          = 2000,   /**< Maximum temperature (+125degC raw value) */
00657 } ds18b20_conversion_constants_t;
00658
00659 /**
00660  * @brief DS18B20 scratchpad write buffer indices
00661  */
00662 typedef enum : uint8_t {
00663   k_ds18b20_write_idx_th     = 0, /**< TH (high alarm) register index */
00664   k_ds18b20_write_idx_tl     = 1, /**< TL (low alarm) register index */
00665   k_ds18b20_write_idx_config = 2, /**< Configuration register index */
00666 } ds18b20_write_idx_t;
00667
00668 /**
00669  * @brief DS18B20 initialization values
00670  */
00671 typedef enum : uint8_t {
00672   k_ds18b20_config_register_cleared = 0, /**< Cleared config register value */
00673 } ds18b20_init_values_t;
00674
00675 /**
00676  * @brief DS18B20 power supply modes
00677  */
00678 typedef enum : uint8_t {
00679   k_ds18b20_power_parasitic = 0, /**< Parasitic power mode */
00680   k_ds18b20_power_external  = 1, /**< External power mode */
00681 } ds18b20_power_mode_t;
00682
00683 /*
      ============================================================================
00684  * Type Definitions
00685  *
      ============================================================================
00686  */
00687
```

```
00688 /**
00689  * @brief DS18B20 configuration structure
00690  */
00691 typedef struct {
00692     rx_bus_manager_t*    bus_manager;             /**< Bus manager instance (required) */
00693     const char*          bus_name;              /**< OneWire bus name (required) */
00694     ds18b20_resolution_t resolution;              /**< Temperature resolution */
00695     bool                 use_rom_matching;        /**< Use ROM matching (true) or skip ROM (false) */
00696     uint8_t              rom[k_onewire_rom_bytes]; /**< Device ROM (if use_rom_matching=true) */
00697 } rx_ds18b20_config_t;
00698
00699 /**
00700  * @brief DS18B20 driver handle structure
00701  */
00702 typedef struct {
00703     rx_bus_manager_t*    bus_manager;             /**< Bus manager reference (not owned) */
00704     const char*          bus_name;              /**< OneWire bus name (not owned) */
00705     ds18b20_resolution_t resolution;              /**< Current temperature resolution */
00706     bool                 use_rom_matching;        /**< ROM matching enabled */
00707     uint8_t              rom[k_onewire_rom_bytes]; /**< Device ROM code */
00708     bool                 initialized;             /**< True after successful init */
00709 } rx_ds18b20_handle_t;
00710
00711 /*
    ============================================================================
00712  * Public API
00713  *
    ============================================================================
00714  */
00715
00716 /**
00717  * @brief Initialize DS18B20 sensor
00718  *
00719  * Initializes the DS18B20 sensor and configures resolution.
00720  * If ROM matching is disabled, uses Skip ROM (single device mode).
00721  *
00722  * @param[out] handle Pointer to DS18B20 handle. Must not be nullptr.
00723  * @param[in] config Pointer to configuration. Must not be nullptr.
00724  *
00725  * @return k_rx_ok on success
00726  * @return k_rx_err_null_ptr if handle or config is nullptr
00727  * @return k_rx_err_invalid_arg if bus_manager or bus_name is nullptr
00728  * @return k_rx_err_invalid_state if device not responding
00729  * @return k_rx_err_crc if scratchpad CRC check fails
00730  */
00731 [[nodiscard]] rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t*       handle,
00732                                        const rx_ds18b20_config_t* config);
00733
00734 /**
00735  * @brief Deinitialize DS18B20 sensor
00736  *
00737  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00738  *
00739  * @return k_rx_ok on success
00740  * @return k_rx_err_null_ptr if handle is nullptr
00741  * @return k_rx_err_invalid_state if not initialized
00742  */
00743 [[nodiscard]] rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t* handle);
00744
00745 /**
00746  * @brief Trigger temperature conversion
00747  *
00748  * Sends Convert T command to start temperature measurement.
00749  * Caller must wait for conversion time before reading temperature.
00750  *
00751  * Conversion times:
00752  * - 9-bit: 100ms
00753  * - 10-bit: 200ms
00754  * - 11-bit: 400ms
00755  * - 12-bit: 800ms
00756  *
00757  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00758  *
00759  * @return k_rx_ok on success
00760  * @return k_rx_err_null_ptr if handle is nullptr
00761  * @return k_rx_err_invalid_state if not initialized or device not present
00762  */
00763 [[nodiscard]] rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t* handle);
00764
00765 /**
00766  * @brief Read temperature in Celsius
00767  *
00768  * Reads the scratchpad and converts raw temperature to Celsius.
00769  * Temperature conversion must complete before calling (see conversion times).
00770  *
00771  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00772  * @param[out] temperature_celsius Pointer to store temperature in degC. Must not be nullptr.
```

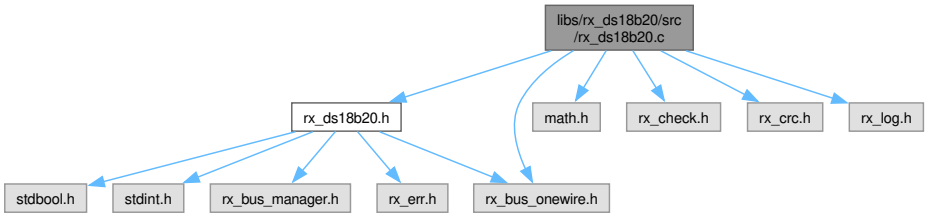
```
00773  *
00774  * @return k_rx_ok on success
00775  * @return k_rx_err_null_ptr if any pointer is nullptr
00776  * @return k_rx_err_invalid_state if not initialized or device not present
00777  * @return k_rx_err_crc if scratchpad CRC check fails
00778  */
00779 [[nodiscard]] rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle,
00780                                                    float*               temperature_celsius);
00781
00782 /**
00783  * @brief Read raw temperature value
00784  *
00785  * Reads the raw 16-bit temperature from scratchpad without conversion.
00786  * Useful for debugging or custom temperature processing.
00787  *
00788  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00789  * @param[out] raw_temp Pointer to store raw temperature. Must not be nullptr.
00790  *
00791  * @return k_rx_ok on success
00792  * @return k_rx_err_null_ptr if any pointer is nullptr
00793  * @return k_rx_err_invalid_state if not initialized or device not present
00794  * @return k_rx_err_crc if scratchpad CRC check fails
00795  */
00796 [[nodiscard]] rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t* handle,
00797                                                        int16_t*              raw_temp);
00798
00799 /**
00800  * @brief Set temperature resolution
00801  *
00802  * Changes the sensor resolution (9, 10, 11, or 12 bits).
00803  * Higher resolution provides more precision but takes longer to convert.
00804  *
00805  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00806  * @param[in] resolution Resolution mode (9-12 bits)
00807  *
00808  * @return k_rx_ok on success
00809  * @return k_rx_err_null_ptr if handle is nullptr
00810  * @return k_rx_err_invalid_arg if resolution out of range
00811  * @return k_rx_err_invalid_state if not initialized or device not present
00812  */
00813 [[nodiscard]] rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t* handle,
00814                                                  ds18b20_resolution_t resolution);
00815
00816 /**
00817  * @brief Get current temperature resolution
00818  *
00819  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00820  * @param[out] resolution Pointer to store resolution. Must not be nullptr.
00821  *
00822  * @return k_rx_ok on success
00823  * @return k_rx_err_null_ptr if any pointer is nullptr
00824  * @return k_rx_err_invalid_state if not initialized
00825  */
00826 [[nodiscard]] rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t* handle,
00827                                                  ds18b20_resolution_t*       resolution);
00828
00829 /**
00830  * @brief Save configuration to EEPROM
00831  *
00832  * Copies scratchpad bytes 2-4 (TH, TL, Config) to non-volatile EEPROM.
00833  * Values persist across power cycles.
00834  *
00835  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00836  *
00837  * @return k_rx_ok on success
00838  * @return k_rx_err_null_ptr if handle is nullptr
00839  * @return k_rx_err_invalid_state if not initialized or device not present
00840  */
00841 [[nodiscard]] rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t* handle);
00842
00843 /**
00844  * @brief Recall configuration from EEPROM
00845  *
00846  * Restores scratchpad bytes 2-4 (TH, TL, Config) from EEPROM.
00847  * Useful after power-up to restore saved settings.
00848  *
00849  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00850  *
00851  * @return k_rx_ok on success
00852  * @return k_rx_err_null_ptr if handle is nullptr
00853  * @return k_rx_err_invalid_state if not initialized or device not present
00854  */
00855 [[nodiscard]] rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t* handle);
00856
00857 /**
00858  * @brief Read power supply mode
00859  *
```

```
00860  * Checks if sensor is using parasitic power or external power.
00861  *
00862  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00863  * @param[out] external_power True if external power, false if parasitic. Must not be nullptr.
00864  *
00865  * @return k_rx_ok on success
00866  * @return k_rx_err_null_ptr if any pointer is nullptr
00867  * @return k_rx_err_invalid_state if not initialized or device not present
00868  */
00869 [[nodiscard]] rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t* handle,
00870                                                   bool*                       external_power);
00871
00872 /**
00873  * @brief Read scratchpad memory
00874  *
00875  * Reads the entire 9-byte scratchpad (8 data bytes + 1 CRC byte).
00876  * Performs CRC validation.
00877  *
00878  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00879  * @param[out] scratchpad Pointer to 9-byte buffer. Must not be nullptr.
00880  *
00881  * @return k_rx_ok on success
00882  * @return k_rx_err_null_ptr if any pointer is nullptr
00883  * @return k_rx_err_invalid_state if not initialized or device not present
00884  * @return k_rx_err_crc if CRC check fails
00885  */
00886 [[nodiscard]] rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t* handle,
00887                                                   uint8_t scratchpad[k_ds18b20_scratchpad_bytes]);
00888
00889 /**
00890  * @brief Get conversion time for current resolution
00891  *
00892  * Returns the maximum conversion time in milliseconds for the configured resolution.
00893  * Caller should wait this long after triggering conversion before reading temperature.
00894  *
00895  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00896  *
00897  * @return Conversion time in milliseconds, or 0 if handle is nullptr or not initialized
00898  */
00899 uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t* handle);
00900
00901 #ifdef ___cplusplus
00902 }
00903 #endif
```

6.3 libs/rx˙ds18b20/src/rx˙ds18b20.c File Reference

DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

#include ”rx_ds18b20.h”
#include <math.h>
#include ”rx_bus_onewire.h”
#include ”rx_check.h”
#include ”rx_crc.h”
#include ”rx_log.h”
Include dependency graph for rx_ds18b20.c:



Data Structures

- struct ds18b20_scratchpad_write_t

  Scratchpad write parameters for TH, TL, and config registers.

Macros

- #define CHECK_DS18B20_HANDLE(handle, tag)

  Validate DS18B20 handle pointer and initialization state.

Functions

- static rx_err_t internal_ds18b20_validate_config (const rx_ds18b20_config_t *config, const rx_ds18b20_handle_t *handle)

  Validate DS18B20 configuration parameters before initialization.
- static rx_err_t internal_ds18b20_verify_device_presence (rx_ds18b20_handle_t *handle)

  Verify DS18B20 device presence and establish communication.
- static rx_err_t internal_ds18b20_select_device (const rx_ds18b20_handle_t *handle)

  Select DS18B20 device using ROM matching or Skip ROM command.
- static rx_err_t internal_ds18b20_read_scratchpad_raw (const rx_ds18b20_handle_t *handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])

  Read DS18B20 scratchpad memory with CRC-8 validation.
- static rx_err_t internal_ds18b20_write_scratchpad (const rx_ds18b20_handle_t *handle, const ds18b20_scratchpad_write_t *scratchpad)

  Write DS18B20 scratchpad registers (TH, TL, Config).
- static uint8_t internal_ds18b20_resolution_to_config (const ds18b20_resolution_t resolution)

  Convert resolution enum to config register value.
- static uint16_t internal_ds18b20_get_temp_mask (const ds18b20_resolution_t resolution)

  Get temperature mask for resolution.
- static float internal_ds18b20_raw_to_celsius (const int16_t raw_temp)

  Convert raw temperature to Celsius.
- rx_err_t rx_ds18b20_init (rx_ds18b20_handle_t *handle, const rx_ds18b20_config_t *config)

  Initialize DS18B20 sensor.
- rx_err_t rx_ds18b20_deinit (rx_ds18b20_handle_t *handle)

  Deinitialize DS18B20 sensor handle and release all resources.
- rx_err_t rx_ds18b20_trigger_conversion (const rx_ds18b20_handle_t *handle)

  Trigger temperature conversion on DS18B20 sensor.
- rx_err_t rx_ds18b20_read_temperature (rx_ds18b20_handle_t *handle, float *temperature_↩ celsius)

  Read temperature from DS18B20 sensor in degrees Celsius.
- rx_err_t rx_ds18b20_read_temperature_raw (rx_ds18b20_handle_t *handle, int16_t *raw_↩ temp)

  Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units.
- rx_err_t rx_ds18b20_set_resolution (rx_ds18b20_handle_t *handle, const ds18b20_resolution_t resolution)

  Set ADC measurement resolution on DS18B20 sensor.
- rx_err_t rx_ds18b20_get_resolution (const rx_ds18b20_handle_t *handle, ds18b20_resolution_t *resolution)

  Get the currently configured ADC resolution from DS18B20 handle.
- rx_err_t rx_ds18b20_save_config (const rx_ds18b20_handle_t *handle)

  Save current scratchpad configuration to DS18B20 EEPROM.
- rx_err_t rx_ds18b20_recall_config (const rx_ds18b20_handle_t *handle)

  Recall configuration from DS18B20 EEPROM into scratchpad RAM.
- rx_err_t rx_ds18b20_read_power_mode (const rx_ds18b20_handle_t *handle, bool *external_↩ _power)

  Read DS18B20 power supply mode (external vs. parasitic).

- rx_err_t rx_ds18b20_read_scratchpad (rx_ds18b20_handle_t *handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])

  Read the full 9-byte scratchpad register from DS18B20 sensor.
- uint32_t rx_ds18b20_get_conversion_time_ms (const rx_ds18b20_handle_t *handle)

  Get temperature conversion time for current resolution.

Variables

- static const char s_tag [] = "DS18B20"

  Logging tag for DS18B20 driver.
- static const uint8_t s_ds18b20_family_code = 0x28U

  DS18B20 family code (byte 0 of ROM).
- static const float s_temp_conversion_divisor = 16.0F

  Temperature conversion divisor (raw to Celsius).
- static const uint8_t s_ds18b20_reserved_byte_value = 0xFFU

  Reserved byte expected value in scratchpad.
- static const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U

  Invalid conversion time sentinel.

6.3.1 Detailed Description

DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

Complete DS18B20 driver implementation with hardware abstraction via bus manager. Supports all DS18B20 features: temperature measurement, resolution configuration, ROM matching for multi-drop buses, and non-volatile EEPROM storage.

6.3.1.1 Architecture

Dependency Injection Pattern:

- Uses rx_bus_manager_t interface for 1-Wire communication
- Enables unit testing with mock bus implementations
- Zero direct hardware dependencies

Communication Flow:
[rx_ds18b20] -> [rx_bus_manager] -> [rx_bus_onewire] -> [GPIO HAL]

Thread Safety:

- NOT thread-safe - caller must provide synchronization
- Multiple handles can coexist (different sensors)
- Concurrent access to same handle causes race conditions

6.3.1.2 NASA Power of 10 Compliance

Rule	Status	Implementation
1. Simple control flow	↩ [OK]↩	No goto, no recursion
2. Fixed loop bounds	↩ [OK]↩	All loops bounded by enum constants
3. No dynamic allocation	↩ [OK]↩	Zero malloc/free
4. Short functions	↩ [OK]↩	Max 60 lines per function
5. Assertions	↩ [OK]↩	Min 2 checks per function (nullptr, state)
6. Data scope	↩ [OK]↩	Variables declared at smallest scope
7. Check returns	↩ [OK]↩	All function returns validated
8. Limit preprocessor	↩ [OK]↩	Typed enums only, no magic numbers
9. Pointer restrictions	↩ [OK]↩	Single-level dereferencing
10. Compile warnings	↩ [OK]↩	-Wall -Wextra -Werror

6.3.1.3 SOLID Principles

Single Responsibility:

- One purpose: DS18B20 sensor communication
- Bus operations delegated to rx_bus_onewire
- CRC validation delegated to rx_crc

Open/Closed:

- Extensible via bus manager interface
- New bus types added without modifying this driver

Liskov Substitution:

- Any rx_bus_manager_t implementation works
- Mock bus substitutes for testing

Interface Segregation:

- Clean API with focused functions
- Separate read/write/config operations

Dependency Inversion:

- Depends on rx_bus_manager_t abstraction
- Bus manager injected via config structure

6.3.1.4 Performance

Typical Operation Times:

- Initialization: ~100ms (bus init + scratchpad read)
- Trigger conversion: ~2ms (command send)
- Read temperature: ~5ms (scratchpad read + CRC check)
- Resolution change: ~10ms (read + modify + write scratchpad)

Conversion Times (sensor internal):

- 9-bit: 94ms
- 10-bit: 188ms
- 11-bit: 375ms
- 12-bit: 750ms (default)

6.3.1.5 Memory Layout

DS18B20 Scratchpad Structure (9 bytes):

```
Byte | Field         | Description
-----|---------------|-------------------------------------------
0    | Temp LSB      | Temperature low byte (1/16degC units)
1    | Temp MSB      | Temperature high byte (sign-extended)
2    | TH Register   | High alarm threshold (user-defined)
3    | TL Register   | Low alarm threshold (user-defined)
4    | Config        | Resolution bits R1:R0 (bits 6:5)
5    | Reserved      | 0xFF (always)
6    | Reserved      | Factory programmed (implementation-specific)
7    | Reserved      | 0x10 (count remaining, legacy)
8    | CRC           | CRC-8/MAXIM checksum
```

Temperature Data Format:

```
15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
| S| S| S| S| S| 2^6| 2^5| 2^4| 2^3| 2^2| 2^1| 2^0|2^-^1|2^-^2|2^-^3|2^-^4|
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
  Sign extend    Integer part      Fractional (1/16degC)
```

Resolution vs Temperature Bits:

- 9-bit: Bits 15:3 valid, bits 2:0 undefined (mask 0xFFF8)
- 10-bit: Bits 15:2 valid, bits 1:0 undefined (mask 0xFFFC)
- 11-bit: Bits 15:1 valid, bit 0 undefined (mask 0xFFFE)
- 12-bit: All bits valid (mask 0xFFFF)

6.3.1.6 Error Handling

Error Propagation Strategy:

```
// Check bus errors -> return immediately
err = rx_bus_onewire_reset(...);
if (err != k_rx_ok) return err;

// Check CRC -> return k_rx_err_crc_mismatch
if (crc_calc != crc_device) return k_rx_err_crc_mismatch;

// Check range -> return k_rx_err_out_of_range
if (temp < MIN || temp > MAX) return k_rx_err_out_of_range;
```

Common Error Codes:

- k_rx_err_null_ptr - nullptr handle/config/output pointer
- k_rx_err_invalid_state - Sensor not initialized or not present
- k_rx_err_invalid_arg - Invalid resolution or ROM family code
- k_rx_err_crc_mismatch - Scratchpad CRC validation failed
- k_rx_err_out_of_range - Temperature outside -55degC to +125degC

Example: Basic Temperature Reading

```
rx_ds18b20_handle_t sensor;
rx_ds18b20_config_t config = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_12bit,
    .use_rom_matching = false  // Single sensor
};

// 1. Initialize
rx_err_t err = rx_ds18b20_init(&sensor, &config);
if (err != k_rx_ok) handle_error(err);

// 2. Trigger conversion
err = rx_ds18b20_trigger_conversion(&sensor);
if (err != k_rx_ok) handle_error(err);

// 3. Wait for conversion (12-bit = 750ms)
delay_ms(rx_ds18b20_get_conversion_time_ms(&sensor));

// 4. Read temperature
float temp_c = 0.0f;
err = rx_ds18b20_read_temperature(&sensor, &temp_c);
if (err == k_rx_ok) {
    printf("Temperature: %.2fdegC\n", temp_c);
}

// 5. Cleanup
rx_ds18b20_deinit(&sensor);
```

Example: Multi-Drop Bus with ROM Matching

```
// Read ROM codes first (use search algorithm or known values)
uint8_t sensor1_rom[8] = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE};
uint8_t sensor2_rom[8] = {0x28, 0xFE, 0xDC, 0xBA, 0x98, 0x76, 0x54, 0x32};

rx_ds18b20_handle_t sensor1, sensor2;
rx_ds18b20_config_t config1 = {
    .bus_manager = &g_bus_manager,
    .bus_name = "onewire0",
    .resolution = k_ds18b20_resolution_12bit,
    .use_rom_matching = true,
    .rom = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE}
};

// Initialize both sensors on same bus
rx_ds18b20_init(&sensor1, &config1);
config1.rom = sensor2_rom;  // Reuse config, change ROM
```

```
rx_ds18b20_init(&sensor2, &config1);

// Read from specific sensor
float temp1 = 0.0f;
rx_ds18b20_trigger_conversion(&sensor1);
delay_ms(750);
rx_ds18b20_read_temperature(&sensor1, &temp1);
```

See also

rx_ds18b20.h

rx_bus_onewire.h

rx_crc.h

Date

2026-01-05

Copyright

Definition in file rx_ds18b20.c.

6.3.2 Data Structure Documentation

6.3.2.1 struct ds18b20˙scratchpad˙write˙t

Scratchpad write parameters for TH, TL, and config registers.

Internal structure for writing DS18B20 scratchpad registers. DS18B20 Write Scratchpad command (0x4E) writes exactly 3 bytes: TH alarm, TL alarm, and configuration register.

Memory Layout (3 bytes total):

```
Offset | Field  | Description
-------|--------|-----------------------------------------------------
0      | th     | High temperature alarm threshold (-55 to +125degC)
1      | tl     | Low temperature alarm threshold (-55 to +125degC)
2      | config | Resolution bits R1:R0 at bits 6:5
```

Configuration Register Format:

```
Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0
------|-------|-------|-------|-------|-------|-------|------
  0   |  R1   |  R0   |   1   |   1   |   1   |   1   |   1
      +-------+-------+
      Resolution bits
```

Resolution Encoding:

- R1=0, R0=0: 9-bit (0.5degC, 93.75ms)
- R1=0, R0=1: 10-bit (0.25degC, 187.5ms)
- R1=1, R0=0: 11-bit (0.125degC, 375ms)
- R1=1, R0=1: 12-bit (0.0625degC, 750ms)

Note

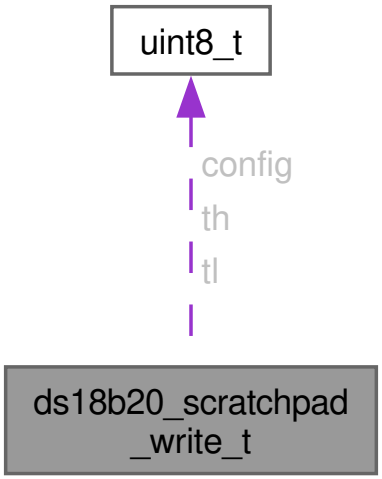
This structure is for internal use only - not exposed in public API.

See also

internal_ds18b20_write_scratchpad()
internal_ds18b20_resolution_to_config()

Definition at line 328 of file rx_ds18b20.c.

Collaboration diagram for ds18b20_scratchpad_write_t:



Data Fields

uint8_t	config	Configuration register (bits 6:5 = resolution)
uint8_t	th	High alarm threshold register (signed, stored as uint8_t)
uint8_t	tl	Low alarm threshold register (signed, stored as uint8_t)

6.3.3 Macro Definition Documentation

6.3.3.1 CHECK˙DS18B20˙HANDLE

```
#define CHECK_DS18B20_HANDLE(
                handle,
                tag)
```

Value:

```
do {                                                                    \
  RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");                  \
  if (!(handle)->initialized) {                                         \
    rx_log_error(tag, "DS18B20 not initialized");                       \
    return k_rx_err_invalid_state;                                      \
  }                                                                     \
} while (0)
```

Validate DS18B20 handle pointer and initialization state.

Composite validation macro that checks both nullptr and initialization. Used at the start of all public API functions to enforce preconditions.

Checks Performed:

1. Handle is not nullptr (via RX_CHECK_NULL_PTR)
2. Handle is initialized (handle->initialized == true)

Error Returns:

- k_rx_err_null_ptr if handle is nullptr
- k_rx_err_invalid_state if not initialized

Example:

```
rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temp)
{
    CHECK_DS18B20_HANDLE(handle, s_tag);  // Early return on error
    // ... function body ...
}
```

Note

This is a statement macro (uses do-while(0) pattern) - requires semicolon.

Warning

Do not use in expressions - this macro contains early returns.

See also

RX_CHECK_NULL_PTR

Definition at line 277 of file rx_ds18b20.c.

```
00277 #define CHECK_DS18B20_HANDLE(handle, tag)                                   \
00278   do {                                                                      \
00279     RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");                    \
00280     if (!(handle)->initialized) {                                           \
00281       rx_log_error(tag, "DS18B20 not initialized");                         \
00282       return k_rx_err_invalid_state;                                        \
00283     }                                                                       \
00284   } while (0)
```

Referenced by rx_ds18b20_get_resolution(), rx_ds18b20_read_power_mode(), rx_ds18b20_read_scratchpad(), rx_ds18b20_read_temperature(), rx_ds18b20_read_temperature_raw(), rx_ds18b20_recall_config(), rx_ds18b20_save_config(), rx_ds18b20_set_resolution(), and rx_ds18b20_trigger_conversion().

6.3.4 Function Documentation

6.3.4.1 internal_ds18b20_get_temp_mask()

```
uint16_t internal_ds18b20_get_temp_mask (
            const ds18b20_resolution_t resolution)   [static]
```

Get temperature mask for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked.

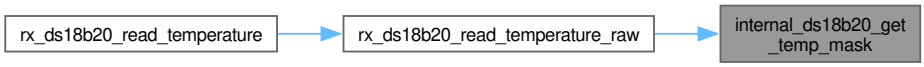
Definition at line 1705 of file rx_ds18b20.c.

```
01706 {
01707   switch (resolution) {
01708     case k_ds18b20_resolution_9bit:
01709       return k_ds18b20_temp_mask_9bit;
01710     case k_ds18b20_resolution_10bit:
01711       return k_ds18b20_temp_mask_10bit;
01712     case k_ds18b20_resolution_11bit:
01713       return k_ds18b20_temp_mask_11bit;
01714     case k_ds18b20_resolution_12bit: /* fall through */
01715     default:
01716       return k_ds18b20_temp_mask_12bit;
01717   }
01718 }
```

References k_ds18b20_resolution_10bit, k_ds18b20_resolution_11bit, k_ds18b20_resolution_12bit, k_ds18b20_resolution_9bit, k_ds18b20_temp_mask_10bit, k_ds18b20_temp_mask_11bit, k_ds18b20_temp_mask_ and k_ds18b20_temp_mask_9bit.

Referenced by rx_ds18b20_read_temperature_raw().

Here is the caller graph for this function:



6.3.4.2 internal˙ds18b20˙raw˙to˙celsius()

```
float internal_ds18b20_raw_to_celsius (
                const int16_t raw_temp)   [static]
```

Convert raw temperature to Celsius.

DS18B20 stores temperature as 16-bit signed value in 1/16degC units. Divide by 16 to get Celsius.

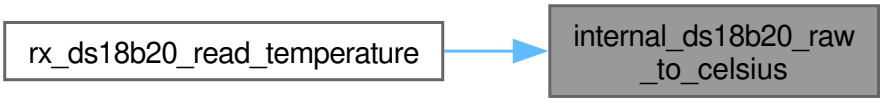
Definition at line 1728 of file rx_ds18b20.c.

```
01729 {
01730   /* Pre-condition: caller validates range; no RX_ASSERT to avoid unreachable false branch */
01731   /* Convert from 1/16degC to degC */
01732   float result = (float)raw_temp / s_temp_conversion_divisor;
01733
01734   /* Post-condition: result is always finite for finite integer input; no RX_ASSERT needed */
01735
01736   return result;
01737 }
```

References s_temp_conversion_divisor.

Referenced by rx_ds18b20_read_temperature().

Here is the caller graph for this function:



6.3.4.3 internal_ds18b20_read_scratchpad_raw()

```
rx_err_t internal_ds18b20_read_scratchpad_raw (
                const rx_ds18b20_handle_t * handle,
                uint8_t scratchpad[k_ds18b20_scratchpad_bytes])   [static]
```

Read DS18B20 scratchpad memory with CRC-8 validation.

Reads all 9 bytes of scratchpad memory and validates data integrity using CRC-8/MAXIM checksum. This is the primary method for reading temperature and configuration data from the sensor.

Algorithm:

```
@startuml
start
:Send reset pulse;
if (Presence detected?) then (yes)
  :Select device (Match/Skip ROM);
  :Send Read Scratchpad (0xBE);
  :Read 9 bytes (8 data + 1 CRC);
  :Calculate CRC-8 of first 8 bytes;
  if (CRC matches byte 8?) then (yes)
    if (Reserved byte == 0xFF?) then (yes)
      :Return k_rx_ok;
      stop
    else (reserved invalid)
      :Log warning;
      :Return k_rx_ok anyway;
      note right: Non-fatal, continue
    endif
  else (CRC mismatch)
    :Log error;
    :Return k_rx_err_crc_mismatch;
    stop
  endif
else (no presence)
  :Return k_rx_err_invalid_state;
endif
@enduml
```

Scratchpad Memory Map:

Byte	Field	Description
0	Temp LSB	Temperature low byte
1	Temp MSB	Temperature high byte
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config	Resolution bits (6:5)
5	Reserved	Always 0xFF
6	Reserved	Factory-programmed
7	Reserved	Count remain (legacy)
8	CRC	CRC-8/MAXIM checksum

CRC-8/MAXIM Calculation:

- Polynomial: 0x31 (x^8 + x^5 + x^4 + 1)
- Initial value: 0x00
- Input: First 8 bytes of scratchpad
- Result: Must match byte 8

Post-Conditions:

- Scratchpad buffer contains valid sensor data
- CRC verified (data integrity guaranteed)
- Reserved byte checked (warning if unexpected)

Precondition

handle != nullptr

handle->initialized == true

scratchpad != nullptr

scratchpad size >= 9 bytes

Postcondition

If k_rx_ok: scratchpad contains validated sensor data

If error: scratchpad contents undefined

Note

Reserved byte validation is non-fatal (log warning only)

Warning

Do not use scratchpad data if CRC check fails

Performance:

Execution time: ~5ms (reset 1ms + select 1ms + read 3ms)

Example: Manual Scratchpad Access

```
uint8_t scratchpad[9];
rx_err_t err = internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad);
if (err == k_rx_ok) {
    int16_t temp_raw = scratchpad[0] | (scratchpad[1] « 8);
    uint8_t config = scratchpad[4];
    uint8_t resolution = (config » 5) & 0x03;
}
```

See also

rx_crc8_maxim()

internal_ds18b20_select_device()

Definition at line 1486 of file rx_ds18b20.c.

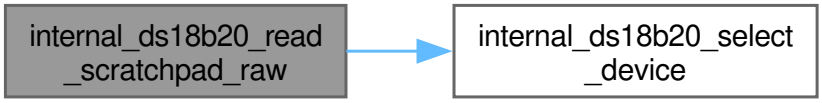
```
01488 {
01489   bool     presence   = false;
01490   rx_err_t err        = k_rx_ok;
01491   uint32_t crc_calc   = 0U;
01492   uint8_t  crc_device = 0;
01493
01494   /* Callers always validate handle and scratchpad before calling this internal function.
01495    * No RX_CHECK_NULL_PTR here to avoid unreachable branches. */
01496
01497   /* Reset and check presence */
01498   err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01499   if (err != k_rx_ok || !presence) {
01500     rx_log_error(s_tag, "Device not present for scratchpad read");
01501     return k_rx_err_invalid_state;
01502   }
01503
01504   /* Select device */
01505   err = internal_ds18b20_select_device(handle);
01506   if (err != k_rx_ok) {
01507     return err;
```

```
01508 }
01509
01510 /* Send Read Scratchpad command */
01511 err =
01512   rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_scratch);
01513 if (err != k_rx_ok) {
01514   rx_log_error(s_tag, "Failed to send read scratchpad command");
01515   return err;
01516 }
01517
01518 /* Read 9 bytes (8 data + 1 CRC) */
01519 err = rx_bus_onewire_read(handle->bus_manager,
01520                           handle->bus_name,
01521                           scratchpad,
01522                           k_ds18b20_scratchpad_bytes);
01523 if (err != k_rx_ok) {
01524   rx_log_error(s_tag, "Failed to read scratchpad data");
01525   return err;
01526 }
01527
01528 /* Validate CRC */
01529 crc_calc = 0U;
01530 /* rx_crc8_maxim always succeeds for valid inputs (pre-validated above) */
01531 (void)rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes, &crc_calc);
01532 crc_device = scratchpad[k_ds18b20_scratch_crc];
01533
01534 if ((uint8_t)crc_calc != crc_device) {
01535   rx_log_error(s_tag, "Scratchpad CRC mismatch");
01536   return k_rx_err_crc_mismatch;
01537 }
01538
01539 /* Post-condition: Validate scratchpad structure (reserved byte should be 0xFF) */
01540 if (scratchpad[k_ds18b20_scratch_reserved1] != s_ds18b20_reserved_byte_value) {
01541   rx_log_warn(s_tag, "Scratchpad reserved byte unexpected value");
01542 }
01543
01544 return k_rx_ok;
01545 }
```

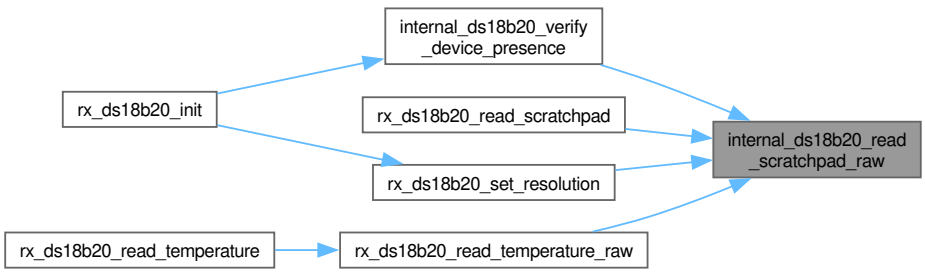
References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, internal_ds18b20_select_device(), k_ds18b20_cmd_read_scratch, k_ds18b20_crc_bytes, k_ds18b20_scratch_crc, k_ds18b20_scratch_reserved1, k_ds18b20_scratchpad_bytes, s_ds18b20_reserved_byte_value, and s_tag.

Referenced by internal_ds18b20_verify_device_presence(), rx_ds18b20_read_scratchpad(), rx_ds18b20_read_tempera and rx_ds18b20_set_resolution().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.4 internal˙ds18b20˙resolution˙to˙config()

```
uint8_t internal_ds18b20_resolution_to_config (
                ds18b20_resolution_t resolution)   [static]
```

Convert resolution enum to config register value.

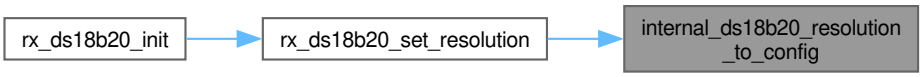
Definition at line 1690 of file rx_ds18b20.c.

```
01691 {
01692   /* Resolution bits are R1:R0 at bits 6:5 */
01693   const uint8_t config = (uint8_t)resolution « k_ds18b20_config_r0_bit;
01694
01695   return config;
01696 }
```

References k_ds18b20_config_r0_bit.

Referenced by rx_ds18b20_set_resolution().

Here is the caller graph for this function:



6.3.4.5 internal˙ds18b20˙select˙device()

```
rx_err_t internal_ds18b20_select_device (
                const rx_ds18b20_handle_t * handle)   [static]
```

Select DS18B20 device using ROM matching or Skip ROM command.

Sends appropriate ROM command based on bus configuration:

- Match ROM (0x55) - Address specific device on multi-drop bus
- Skip ROM (0xCC) - Broadcast to single device or all devices

Decision Flow:

```
@startuml
start
if (use_rom_matching?) then (yes)
  :Send Match ROM (0x55);
  :Send 64-bit ROM code;
  note right
    Addresses specific device
    by unique ROM ID
  end note
else (no)
  :Send Skip ROM (0xCC);
  note right
    Single device: addresses only device
    Multiple devices: broadcasts to all
  end note
endif
:Device selected;
:Ready for function command;
stop
@enduml
```

ROM Matching (Multi-Drop Bus):

- Used when multiple DS18B20 sensors on one bus
- Requires 64-bit ROM code (obtained via search or label)
- Ensures commands go to intended sensor only

Skip ROM (Single Device):

- Faster - no need to send 64-bit ROM
- Safe when only one device on bus
- Caution: broadcasts if multiple devices present

Timing:

- Match ROM: ~3ms (1 byte command + 8 bytes ROM)
- Skip ROM: ~1ms (1 byte command)

Precondition

handle->use_rom_matching is configured
If ROM matching: handle->rom contains valid 64-bit ID

Postcondition

Selected device ready for function commands

Note

Called before every DS18B20 function command

Warning

Skip ROM on multi-drop bus affects ALL devices

Example: ROM Code Format

```
Byte 0: Family Code (0x28 for DS18B20)
Byte 1-6: Serial Number (48-bit unique ID)
Byte 7: CRC-8 checksum
```

See also

rx_bus_onewire_match_rom()

rx_bus_onewire_skip_rom()

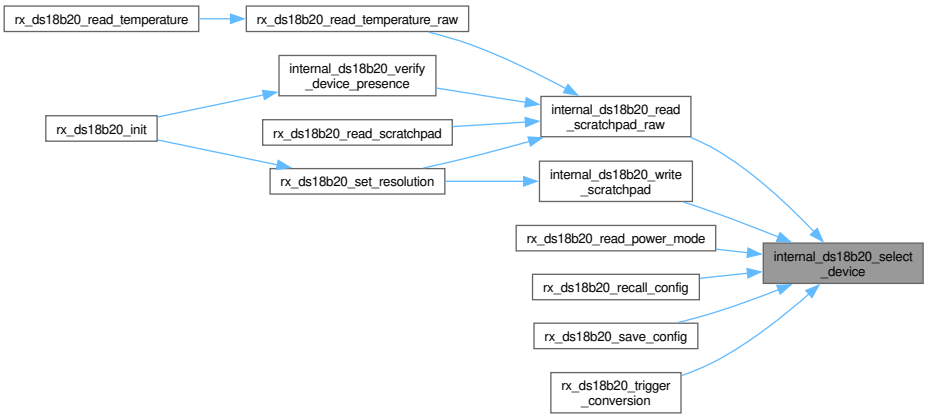
Definition at line 1370 of file rx_ds18b20.c.

```
01371 {
01372   if (handle->use_rom_matching) {
01373     /* Use Match ROM to address specific device */
01374     rx_err_t err = rx_bus_onewire_match_rom(handle->bus_manager, handle->bus_name, handle->rom);
01375     if (err != k_rx_ok) {
01376       rx_log_error(s_tag, "Failed to send Match ROM");
01377       return err;
01378     }
01379   } else {
01380     /* Use Skip ROM for single device or broadcast */
01381     rx_err_t err = rx_bus_onewire_skip_rom(handle->bus_manager, handle->bus_name);
01382     if (err != k_rx_ok) {
01383       rx_log_error(s_tag, "Failed to send Skip ROM");
01384       return err;
01385     }
01386   }
01387
01388   return k_rx_ok;
01389 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, rx_ds18b20_handle_t::rom, s_tag, and rx_ds18b20_handle_t::use_rom_matching.

Referenced by internal_ds18b20_read_scratchpad_raw(), internal_ds18b20_write_scratchpad(), rx_ds18b20_read_power_mode(), rx_ds18b20_recall_config(), rx_ds18b20_save_config(), and rx_ds18b20_trigger_conversion().

Here is the caller graph for this function:



6.3.4.6 internal˙ds18b20˙validate˙config()

```
rx_err_t internal_ds18b20_validate_config (
                const rx_ds18b20_config_t * config,
                const rx_ds18b20_handle_t * handle)    [static]
```

Validate DS18B20 configuration parameters before initialization.

Comprehensive validation of configuration structure and handle state. Called by rx_ds18b20_init() to enforce preconditions and prevent misconfiguration.

Algorithm:

1. Check config pointer is not nullptr
2. Check bus_manager pointer is not nullptr
3. Check bus_name string is not nullptr
4. Verify handle is not already initialized (prevent re-initialization)
5. Validate resolution is in valid range (0-3)
6. If ROM matching enabled, verify family code is 0x28 (DS18B20)

Validation Rules:

Field	Check	Error Code
config	!= nullptr	k_rx_err_null_ptr
bus_manager	!= nullptr	k_rx_err_null_ptr
bus_name	!= nullptr	k_rx_err_null_ptr
initialized	== false	k_rx_err_invalid_state
resolution	<= 3	k_rx_err_invalid_arg
rom[0] (if ROM)	== 0x28	k_rx_err_invalid_arg

Precondition

config != nullptr

handle != nullptr (checked by caller)

Postcondition

If k_rx_ok returned, all fields are valid for initialization

Note

This is an internal helper - not exposed in public API

Warning

Do not call after initialization - will return error

See also

rx_ds18b20_init()

ds18b20_resolution_t

Definition at line 1188 of file rx_ds18b20.c.

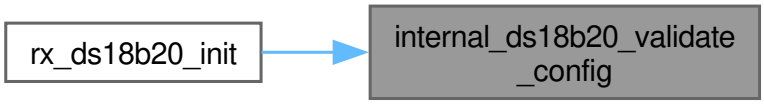
```
01190 {
01191   RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
01192   RX_CHECK_NULL_PTR(config->bus_manager, s_tag, "bus_manager is nullptr");
01193   RX_CHECK_NULL_PTR(config->bus_name, s_tag, "bus_name is nullptr");
01194
01195   if (handle->initialized) {
01196     rx_log_warn(s_tag, "DS18B20 already initialized");
01197     return k_rx_err_invalid_state;
01198   }
01199
01200   if (config->resolution > k_ds18b20_resolution_12bit) {
01201     rx_log_error(s_tag, "Invalid resolution setting");
01202     return k_rx_err_invalid_arg;
01203   }
01204
01205   if ((int)config->use_rom_matching && config->rom[0] != s_ds18b20_family_code) {
```

```
01206     rx_log_error(s_tag, "Invalid DS18B20 family code");
01207     return k_rx_err_invalid_arg;
01208   }
01209
01210   return k_rx_ok;
01211 }
```

References rx_ds18b20_config_t::bus_manager, rx_ds18b20_config_t::bus_name, rx_ds18b20_handle_t::initialized, k_ds18b20_resolution_12bit, rx_ds18b20_config_t::resolution, rx_ds18b20_config_t::rom, s_ds18b20_family_code, s_tag, and rx_ds18b20_config_t::use_rom_matching.

Referenced by rx_ds18b20_init().

Here is the caller graph for this function:



6.3.4.7 internal˙ds18b20˙verify˙device˙presence()

```
rx_err_t internal_ds18b20_verify_device_presence (
                rx_ds18b20_handle_t * handle)   [static]
```

Verify DS18B20 device presence and establish communication.

Three-step verification process to ensure sensor is physically connected and responding correctly before completing initialization.

Algorithm:

```
@startuml
start
:Initialize OneWire bus;
if (Bus init successful?) then (yes)
  :Send reset pulse;
  if (Presence pulse detected?) then (yes)
    :Read scratchpad (with CRC);
    if (Scratchpad valid?) then (yes)
      :Return k_rx_ok;
      stop
    else (CRC fail)
      :Return error;
    endif
  else (no presence)
    :Log "device not present";
    :Return k_rx_err_invalid_state;
    stop
  endif
else (bus init fail)
  :Return error;
endif
@enduml
```

Verification Steps:

1. Bus Initialization - Configure GPIO for 1-Wire timing
2. Presence Detection - Send reset, check for presence pulse
3. Communication Test - Read scratchpad with CRC validation

Timing:

- Reset pulse: 480-960 us LOW
- Presence detect: 60-240 us after reset
- Scratchpad read: ~5ms (9 bytes + CRC)

Precondition

handle->bus_manager is valid

handle->bus_name is valid

Postcondition

If k_rx_ok, device is ready for configuration

Note

This function is called during rx_ds18b20_init()

Warning

Blocks for ~100ms total (bus init + scratchpad read)

Performance:

Execution time: ~100ms (bus init 50ms + reset 1ms + scratchpad 5ms + margin)

See also

rx_bus_onewire_init()

rx_bus_onewire_reset()

internal_ds18b20_read_scratchpad_raw()

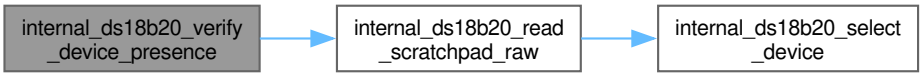
Definition at line 1272 of file rx_ds18b20.c.

```
01273 {
01274   /* Initialize OneWire bus */
01275   rx_err_t err = rx_bus_onewire_init(handle->bus_manager, handle->bus_name);
01276   if (err != k_rx_ok) {
01277     rx_log_error(s_tag, "Failed to initialize OneWire bus");
01278     return err;
01279   }
01280
01281   /* Check device presence */
01282   bool presence = false;
01283   err           = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01284   if (err != k_rx_ok) {
01285     rx_log_error(s_tag, "OneWire reset failed");
01286     return err;
01287   }
01288
01289   if (!presence) {
01290     rx_log_error(s_tag, "DS18B20 device not present on bus");
01291     return k_rx_err_invalid_state;
01292   }
01293
01294   /* Read scratchpad to verify communication */
01295   uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
01296   err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01297   if (err != k_rx_ok) {
01298     rx_log_error(s_tag, "Failed to read scratchpad during init");
01299     return err;
01300   }
01301
01302   return k_rx_ok;
01303 }
```

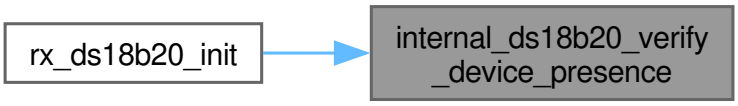
References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, internal_ds18b20_read_scratchpa
k_ds18b20_scratchpad_bytes, and s_tag.

Referenced by rx_ds18b20_init().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.8 internal˙ds18b20˙write˙scratchpad()

```
rx_err_t internal_ds18b20_write_scratchpad (
                const rx_ds18b20_handle_t * handle,
                const ds18b20_scratchpad_write_t * scratchpad)    [static]
```

Write DS18B20 scratchpad registers (TH, TL, Config).

Writes the three user-modifiable scratchpad registers: high alarm (TH), low alarm (TL), and configuration (resolution). This is the only way to change sensor resolution and alarm thresholds.

Algorithm:

```
@startuml
start
:Send reset pulse;
if (Presence detected?) then (yes)
  :Select device (Match/Skip ROM);
  :Send Write Scratchpad (0x4E);
  :Write TH byte;
  :Write TL byte;
  :Write Config byte;
  :Return k_rx_ok;
  note right
    Changes take effect immediately
    but are volatile (lost on power cycle)
  end note
  stop
else (no presence)
  :Return k_rx_err_invalid_state;
endif
@enduml
```

Write Sequence:

1. Reset bus and check presence
2. Select device (Match ROM or Skip ROM)
3. Send Write Scratchpad command (0x4E)
4. Write 3 bytes in order: TH, TL, Config

Configuration Register Encoding:

```
Bit:  7   6   5   4   3   2   1   0
    +---+---+---+---+---+---+---+---+
    | 0 |R1 |R0 | 1 | 1 | 1 | 1 | 1 |
    +---+---+---+---+---+---+---+---+
        +---+---+
      Resolution

R1 R0 | Resolution | Conversion Time
------+------------+----------------
 0  0 |   9-bit    |     94 ms
 0  1 |  10-bit    |    188 ms
 1  0 |  11-bit    |    375 ms
 1  1 |  12-bit    |    750 ms
```

Persistence:

- Scratchpad writes are volatile (RAM only)
- Lost on power cycle or reset
- Use Copy Scratchpad (0x48) to save to EEPROM

Precondition

handle != nullptr

handle->initialized == true

scratchpad != nullptr

Postcondition

Sensor resolution/alarms updated (volatile, RAM only)

Note

Changes are immediate but volatile (lost on power cycle)

Warning

No CRC verification - ensure bus integrity

Performance:

Execution time: ~3ms (reset 1ms + select 1ms + write 1ms)

Example: Change Resolution to 10-bit

```
uint8_t scratchpad_data[9];
internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad_data);

ds18b20_scratchpad_write_t write_cfg = {
    .th = scratchpad_data[2],  // Preserve TH
    .tl = scratchpad_data[3],  // Preserve TL
    .config = (0x01 « 5) | 0x1F  // 10-bit: R1=0, R0=1
};
internal_ds18b20_write_scratchpad(&sensor, &write_cfg);

// Save to EEPROM to make permanent
rx_ds18b20_save_config(&sensor);
```

See also

rx_ds18b20_save_config() Make changes permanent

internal_ds18b20_select_device()

ds18b20_scratchpad_write_t

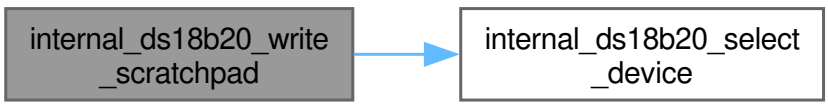
Definition at line 1639 of file rx_ds18b20.c.

```
01641 {
01642   /* Callers always validate handle and scratchpad before calling this internal function.
01643    * No RX_CHECK_NULL_PTR here to avoid unreachable branches. */
01644
01645   /* Reset and check presence */
01646   bool      presence = false;
01647   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01648   if (err != k_rx_ok || !presence) {
01649     rx_log_error(s_tag, "Device not present for scratchpad write");
01650     return k_rx_err_invalid_state;
01651   }
01652
01653   /* Select device */
01654   err = internal_ds18b20_select_device(handle);
01655   if (err != k_rx_ok) {
01656     return err;
01657   }
01658
01659   /* Send Write Scratchpad command */
01660   err =
01661     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_write_scratch);
01662   if (err != k_rx_ok) {
01663     rx_log_error(s_tag, "Failed to send write scratchpad command");
01664     return err;
01665   }
01666
01667   /* Write 3 bytes (TH, TL, Config) */
01668   uint8_t write_buf[k_ds18b20_scratchpad_write_bytes];
01669   write_buf[k_ds18b20_write_idx_th]     = scratchpad->th;
01670   write_buf[k_ds18b20_write_idx_tl]     = scratchpad->tl;
01671   write_buf[k_ds18b20_write_idx_config] = scratchpad->config;
01672
01673   err = rx_bus_onewire_write(handle->bus_manager,
01674                              handle->bus_name,
01675                              write_buf,
01676                              k_ds18b20_scratchpad_write_bytes);
01677   if (err != k_rx_ok) {
01678     rx_log_error(s_tag, "Failed to write scratchpad data");
01679     return err;
01680   }
01681
01682   return k_rx_ok;
01683 }
```

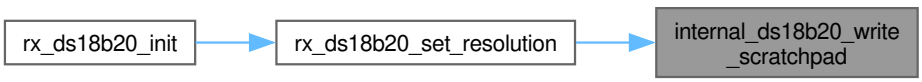
References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, ds18b20_scratchpad_write_t::con
internal_ds18b20_select_device(), k_ds18b20_cmd_write_scratch, k_ds18b20_scratchpad_write_bytes,
k_ds18b20_write_idx_config, k_ds18b20_write_idx_th, k_ds18b20_write_idx_tl, s_tag, ds18b20_scratchpad_write
and ds18b20_scratchpad_write_t::tl.

Referenced by rx_ds18b20_set_resolution().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.9 rx_ds18b20_deinit()

```
rx_err_t rx_ds18b20_deinit (
            rx_ds18b20_handle_t * handle)   [nodiscard]
```

Deinitialize DS18B20 sensor handle and release all resources.

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via memset. After this call the handle is safe to reinitialize with rx_ds18b20_init().

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with memset (zeros all fields)
4. Log deinitialization success

Precondition

handle must be a valid non-null pointer

handle must have been successfully initialized via rx_ds18b20_init()

Postcondition

All handle fields are zeroed (initialized flag is false)

handle may be safely reused with rx_ds18b20_init()

Note

Not thread-safe; caller must ensure no concurrent access to handle

Example:

```
rx_ds18b20_handle_t sensor;
rx_ds18b20_init(&sensor, &config);
// ... use sensor ...
rx_err_t err = rx_ds18b20_deinit(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to deinit DS18B20");
}
```

See also

rx_ds18b20_init() Initialize the handle before use

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

Definition at line 434 of file rx_ds18b20.c.

```
00435 {
00436   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00437
00438   if (!handle->initialized) {
00439     rx_log_warn(s_tag, "DS18B20 not initialized");
00440     return k_rx_err_invalid_state;
00441   }
00442
00443   /* Clear handle */
00444   *handle = (rx_ds18b20_handle_t){};
00445
00446   rx_log_info(s_tag, "DS18B20 deinitialized");
00447   return k_rx_ok;
00448 }
```

References rx_ds18b20_handle_t::initialized, and s_tag.

6.3.4.10 rx˙ds18b20˙get˙conversion˙time˙ms()

```
uint32_t rx_ds18b20_get_conversion_time_ms (
                const rx_ds18b20_handle_t * handle)
```

Get temperature conversion time for current resolution.

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Definition at line 1121 of file rx_ds18b20.c.

```
01122 {
01123   if (handle == nullptr || !handle->initialized) {
01124     return s_ds18b20_conversion_time_invalid_u32;
01125   }
01126
01127   switch (handle->resolution) {
01128     case k_ds18b20_resolution_9bit:
01129       return k_ds18b20_conv_time_9bit_ms;
01130     case k_ds18b20_resolution_10bit:
01131       return k_ds18b20_conv_time_10bit_ms;
01132     case k_ds18b20_resolution_11bit:
01133       return k_ds18b20_conv_time_11bit_ms;
01134     case k_ds18b20_resolution_12bit:
01135       return k_ds18b20_conv_time_12bit_ms;
01136     default:
01137       return s_ds18b20_conversion_time_invalid_u32;
01138   }
01139 }
```

References rx_ds18b20_handle_t::initialized, k_ds18b20_conv_time_10bit_ms, k_ds18b20_conv_time_11bit_ms, k_ds18b20_conv_time_12bit_ms, k_ds18b20_conv_time_9bit_ms, k_ds18b20_resolution_10bit, k_ds18b20_resolution_11bit, k_ds18b20_resolution_12bit, k_ds18b20_resolution_9bit, rx_ds18b20_handle_t::resolu and s_ds18b20_conversion_time_invalid_u32.

6.3.4.11 rx_ds18b20_get_resolution()

```
rx_err_t rx_ds18b20_get_resolution (
            const rx_ds18b20_handle_t * handle,
            ds18b20_resolution_t * resolution)   [nodiscard]
```

Get the currently configured ADC resolution from DS18B20 handle.

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via rx_ds18b20_set_resolution()). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Precondition

handle must be initialized via rx_ds18b20_init()

resolution must be a valid non-null pointer

Postcondition

*resolution contains the current resolution value on k_rx_ok

handle state is unchanged

Note

Not thread-safe; caller must ensure handle is not concurrently modified

Returns cached value from handle – does not issue 1-Wire bus transaction

Example:

```
ds18b20_resolution_t res;
rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
if (err == k_rx_ok) {
    uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
    rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
}
```

See also

rx_ds18b20_set_resolution() Change the ADC resolution

rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 807 of file rx_ds18b20.c.

```
00809 {
00810   CHECK_DS18B20_HANDLE(handle, s_tag);
00811   RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00812
00813   *resolution = handle->resolution;
00814   return k_rx_ok;
00815 }
```

References CHECK_DS18B20_HANDLE, rx_ds18b20_handle_t::resolution, and s_tag.

6.3.4.12 rx˙ds18b20˙init()

```
rx_err_t rx_ds18b20_init (
                rx_ds18b20_handle_t * handle,
                const rx_ds18b20_config_t * config)   [nodiscard]
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters

out	handle	Pointer to DS18B20 handle. Must not be nullptr.
in	config	Pointer to configuration. Must not be nullptr.

Returns

k_rx_ok on success

k_rx_err_null_ptr if handle or config is nullptr

k_rx_err_invalid_arg if bus_manager or bus_name is nullptr

k_rx_err_invalid_state if device not responding

k_rx_err_crc if scratchpad CRC check fails

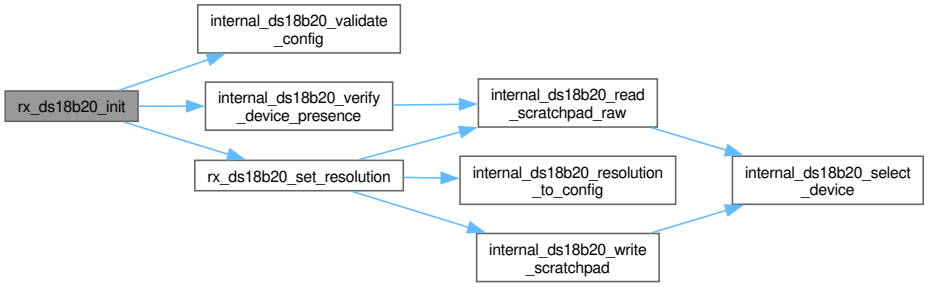
Definition at line 352 of file rx_ds18b20.c.

```
00353 {
00354   /* Validate inputs */
00355   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00356
00357   rx_err_t err = internal_ds18b20_validate_config(config, handle);
00358   if (err != k_rx_ok) {
00359     return err;
00360   }
00361
00362   /* Initialize handle */
00363   *handle                 = (rx_ds18b20_handle_t){};
00364   handle->bus_manager     = config->bus_manager;
00365   handle->bus_name        = config->bus_name;
00366   handle->resolution      = config->resolution;
00367   handle->use_rom_matching = config->use_rom_matching;
00368
00369   if (config->use_rom_matching) {
00370     for (uint8_t i = 0; i < k_onewire_rom_bytes; ++i) {
00371       handle->rom[i] = config->rom[i];
00372     }
00373   }
00374
00375   /* Verify device presence */
00376   err = internal_ds18b20_verify_device_presence(handle);
00377   if (err != k_rx_ok) {
00378     return err;
00379   }
00380
00381   /* Configure resolution */
00382   handle->initialized = true;
00383   err                 = rx_ds18b20_set_resolution(handle, config->resolution);
00384   if (err != k_rx_ok) {
00385     handle->initialized = false;
00386     return err;
00387   }
00388
00389   rx_log_info(s_tag, "DS18B20 initialized successfully");
00390   return k_rx_ok;
00391 }
```

References rx_ds18b20_config_t::bus_manager, rx_ds18b20_handle_t::bus_manager, rx_ds18b20_config_t::bus_nam
rx_ds18b20_handle_t::bus_name, rx_ds18b20_handle_t::initialized, internal_ds18b20_validate_config(),

internal_ds18b20_verify_device_presence(), rx_ds18b20_config_t::resolution, rx_ds18b20_handle_t::resolution, rx_ds18b20_config_t::rom, rx_ds18b20_handle_t::rom, rx_ds18b20_set_resolution(), s_tag, rx_ds18b20_config_t::use_rom_matching, and rx_ds18b20_handle_t::use_rom_matching.

Here is the call graph for this function:



6.3.4.13 rx_ds18b20_read_power_mode()

```
rx_err_t rx_ds18b20_read_power_mode (
                const rx_ds18b20_handle_t * handle,
                bool * external_power)   [nodiscard]
```

Read DS18B20 power supply mode (external vs. parasitic).

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to *external_power

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected to the bus

Postcondition

∗external_power reflects the device power supply mode on k_rx_ok

handle state is unchanged

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:

```
bool ext_power = false;
rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
if (err == k_rx_ok && !ext_power) {
    rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
}
```

See also

rx_ds18b20_save_config() EEPROM write (requires external power)

rx_ds18b20_init() Initialize sensor handle

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

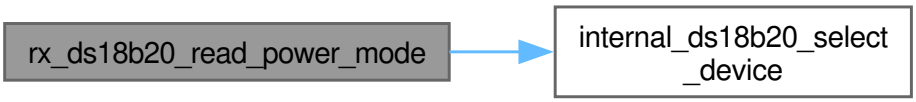
Definition at line 1012 of file rx_ds18b20.c.

```
01013 {
01014   CHECK_DS18B20_HANDLE(handle, s_tag);
01015   RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01016
01017   /* Reset and check presence */
01018   bool     presence = false;
01019   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01020   if (err != k_rx_ok || !presence) {
01021     rx_log_error(s_tag, "Device not present for power mode read");
01022     return k_rx_err_invalid_state;
01023   }
01024
01025   /* Select device */
01026   err = internal_ds18b20_select_device(handle);
01027   if (err != k_rx_ok) {
01028     return err;
01029   }
01030
01031   /* Send Read Power Supply command */
01032   err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01033   if (err != k_rx_ok) {
01034     rx_log_error(s_tag, "Failed to send read power command");
01035     return err;
01036   }
01037
01038   /* Read power bit (1 = external, 0 = parasitic) */
01039   *external_power = false;
01040   err             = rx_bus_onewire_read_bit(handle->bus_manager, handle->bus_name, external_power);
```

```
01041   if (err != k_rx_ok) {
01042     rx_log_error(s_tag, "Failed to read power bit");
01043     return err;
01044   }
01045
01046   return k_rx_ok;
01047 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_read_power, and s_tag.

Here is the call graph for this function:



6.3.4.14 rx_ds18b20_read_scratchpad()

```
rx_err_t rx_ds18b20_read_scratchpad (
                rx_ds18b20_handle_t * handle,
                uint8_t scratchpad[k_ds18b20_scratchpad_bytes])   [nodiscard]
```

Read the full 9-byte scratchpad register from DS18B20 sensor.

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

Byte	Field	Description
0	Temperature LSB	Low byte of temperature register
1	Temperature MSB	High byte of temperature register
2	TH Register	High alarm threshold
3	TL Register	Low alarm threshold
4	Config Register	Resolution configuration byte
5-7	Reserved	Reserved (0xFF, 0x10, 0x10)
8	CRC	CRC-8 of bytes 0-7

CRC is verified internally by internal_ds18b20_read_scratchpad_raw() before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to internal_ds18b20_read_scratchpad_raw() which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Precondition

handle must be initialized via rx_ds18b20_init()

scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes

Postcondition

scratchpad[0..8] contains valid data on k_rx_ok

CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok

Note

Not thread-safe; caller must serialize access to the same handle

Use rx_ds18b20_read_temperature() for converted Celsius value

Example:

```
uint8_t raw[k_ds18b20_scratchpad_bytes];
rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
if (err == k_rx_ok) {
    uint8_t config_reg = raw[4];
}
```

See also

rx_ds18b20_read_temperature() High-level temperature read returning float Celsius

rx_ds18b20_read_temperature_raw() Read raw 16-bit temperature with masking

Since

Version 1.0.0

NASA Power of 10 Compliance:

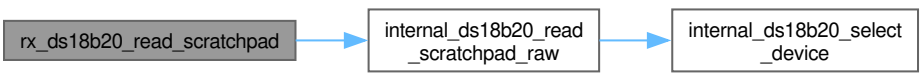
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 1104 of file rx_ds18b20.c.

```
01106 {
01107   CHECK_DS18B20_HANDLE(handle, s_tag);
01108   RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01109
01110   return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01111 }
```

References CHECK_DS18B20_HANDLE, internal_ds18b20_read_scratchpad_raw(), k_ds18b20_scratchpad_bytes, and s_tag.

Here is the call graph for this function:



6.3.4.15 rx_ds18b20_read_temperature()

```
rx_err_t rx_ds18b20_read_temperature (
                rx_ds18b20_handle_t * handle,
                float * temperature_celsius)   [nodiscard]
```

Read temperature from DS18B20 sensor in degrees Celsius.

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call rx_ds18b20_read_temperature_raw() to obtain masked raw value
3. Convert raw signed 16-bit value to float: celsius = raw / 16.0f
4. Write result to *temperature_celsius

Precondition

handle must be initialized via rx_ds18b20_init()

rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed

Postcondition

*temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok

handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
float temp_celsius = 0.0f;
rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
if (err == k_rx_ok) {
    rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
}
```

See also

rx_ds18b20_trigger_conversion() Must be called before reading

rx_ds18b20_read_temperature_raw() Read the raw 16-bit value

rx_ds18b20_get_conversion_time_ms() Determine required wait time

Since

Version 1.0.0

NASA Power of 10 Compliance:

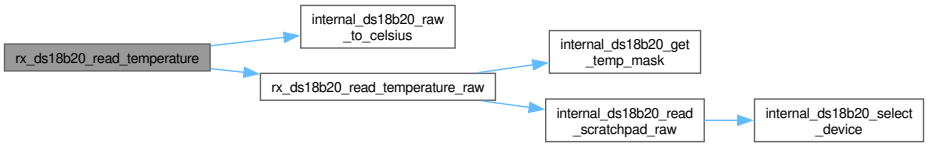
- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

Definition at line 572 of file rx_ds18b20.c.

```
00573 {
00574   CHECK_DS18B20_HANDLE(handle, s_tag);
00575   RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00576
00577   /* Read raw temperature */
00578   int16_t  raw_temp = 0;
00579   rx_err_t err      = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00580   if (err != k_rx_ok) {
00581     return err;
00582   }
00583
00584   /* Convert to Celsius */
00585   *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00586
00587   return k_rx_ok;
00588 }
```

References CHECK_DS18B20_HANDLE, internal_ds18b20_raw_to_celsius(), rx_ds18b20_read_temperature_raw(), and s_tag.

Here is the call graph for this function:



6.3.4.16 rx˙ds18b20˙read˙temperature˙raw()

```
rx_err_t rx_ds18b20_read_temperature_raw (
                rx_ds18b20_handle_t * handle,
                int16_t * raw_temp)    [nodiscard]
```

Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units.

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed int16_t, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)
- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFF8 (3 undefined bits)
- 10-bit: 0xFFFC (2 undefined bits)
- 11-bit: 0xFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via internal_ds18b20_read_scratchpad_raw()
3. Combine temperature LSB and MSB bytes into uint16_t
4. Apply resolution mask from internal_ds18b20_get_temp_mask()
5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
6. Write validated result to *raw_temp

Precondition

handle must be initialized via rx_ds18b20_init()
rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed

Postcondition

*raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok
Resolution mask has been applied; undefined bits are cleared

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Returns stale data if conversion time has not elapsed after trigger

Example:

```
int16_t raw = 0;
rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
if (err == k_rx_ok) {
    float celsius = (float)raw / 16.0f;
}
```

See also

rx_ds18b20_read_temperature() Convenience wrapper returning float Celsius

rx_ds18b20_trigger_conversion() Must be called before reading

rx_ds18b20_set_resolution() Configure ADC resolution and mask

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

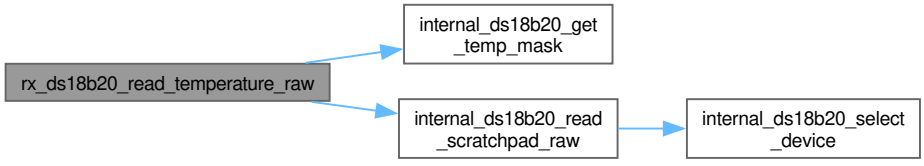
Definition at line 646 of file rx_ds18b20.c.

```
00647 {
00648   CHECK_DS18B20_HANDLE(handle, s_tag);
00649   RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00650
00651   /* Read scratchpad */
00652   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00653   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00654   if (err != k_rx_ok) {
00655     return err;
00656   }
00657
00658   /* Extract temperature (LSB + MSB) */
00659   uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00660   temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) << k_ds18b20_shift_byte;
00661
00662   /* Apply resolution mask to clear undefined bits */
00663   const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
00664   temp &= mask;
00665
00666   /* Post-condition: Validate temperature is within sensor range */
00667   *raw_temp = (int16_t)temp;
00668   if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00669     rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00670     return k_rx_err_out_of_range;
00671   }
00672
00673   return k_rx_ok;
00674 }
```

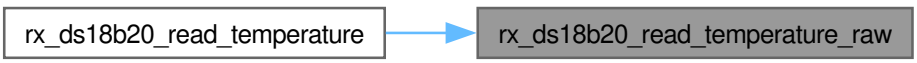
References CHECK_DS18B20_HANDLE, internal_ds18b20_get_temp_mask(), internal_ds18b20_read_scratchpad_r k_ds18b20_scratch_temp_lsb, k_ds18b20_scratch_temp_msb, k_ds18b20_scratchpad_bytes, k_ds18b20_shift_byte, k_ds18b20_temp_max_raw, rx_ds18b20_handle_t::resolution, and s_tag.

Referenced by rx_ds18b20_read_temperature().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.17 rx_ds18b20_recall_config()

```
rx_err_t rx_ds18b20_recall_config (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Recall configuration from DS18B20 EEPROM into scratchpad RAM.

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected to the bus

Postcondition

DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded

Note

Not thread-safe; caller must serialize access to the same handle

The Recall E2 operation is performed automatically by the device on power-up

Example:

```
// Revert any unsaved configuration changes
rx_err_t err = rx_ds18b20_recall_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
}
```

See also

rx_ds18b20_save_config() Persist scratchpad configuration to EEPROM

rx_ds18b20_set_resolution() Modify the scratchpad configuration

Since

Version 1.0.0

NASA Power of 10 Compliance:

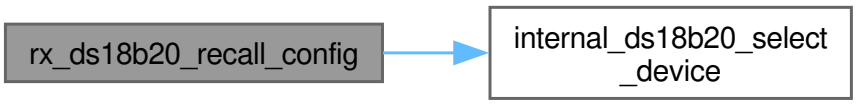
- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

Definition at line 936 of file rx_ds18b20.c.

```
00937 {
00938   CHECK_DS18B20_HANDLE(handle, s_tag);
00939
00940   /* Reset and check presence */
00941   bool     presence = false;
00942   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00943   if (err != k_rx_ok || !presence) {
00944     rx_log_error(s_tag, "Device not present for EEPROM recall");
00945     return k_rx_err_invalid_state;
00946   }
00947
00948   /* Select device */
00949   err = internal_ds18b20_select_device(handle);
00950   if (err != k_rx_ok) {
00951     return err;
00952   }
00953
00954   /* Send Recall E2 command */
00955   err =
00956     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
00957   if (err != k_rx_ok) {
00958     rx_log_error(s_tag, "Failed to send recall EEPROM command");
00959     return err;
00960   }
00961
00962   return k_rx_ok;
00963 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_recall_eeprom, and s_tag.

Here is the call graph for this function:



6.3.4.18 rx˙ds18b20˙save˙config()

```
rx_err_t rx_ds18b20_save_config (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Save current scratchpad configuration to DS18B20 EEPROM.

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Precondition

handle must be initialized via rx_ds18b20_init()

DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Postcondition

DS18B20 EEPROM contains current TH, TL, and configuration register values

Configuration will survive next power cycle

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Not reliable in parasitic power mode; use external power for EEPROM writes

Example:

```
// Configure resolution, then persist to EEPROM
rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
rx_err_t err = rx_ds18b20_save_config(&sensor);
if (err != k_rx_ok) {
    rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
}
```

See also

rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad

rx_ds18b20_set_resolution() Configure resolution before saving

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

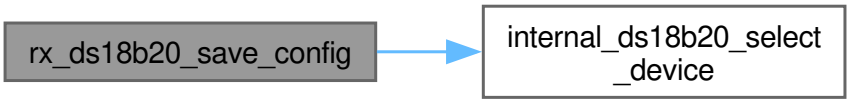
Definition at line 862 of file rx_ds18b20.c.

```
00863 {
00864   CHECK_DS18B20_HANDLE(handle, s_tag);
00865
00866   /* Reset and check presence */
00867   bool     presence = false;
00868   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00869   if (err != k_rx_ok || !presence) {
00870     rx_log_error(s_tag, "Device not present for EEPROM save");
00871     return k_rx_err_invalid_state;
00872   }
00873
00874   /* Select device */
00875   err = internal_ds18b20_select_device(handle);
00876   if (err != k_rx_ok) {
00877     return err;
00878   }
```

```
00879
00880   /* Send Copy Scratchpad command */
00881   err =
00882     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00883   if (err != k_rx_ok) {
00884     rx_log_error(s_tag, "Failed to send copy scratchpad command");
00885     return err;
00886   }
00887
00888   return k_rx_ok;
00889 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_copy_scratch, and s_tag.

Here is the call graph for this function:



6.3.4.19 rx˙ds18b20˙set˙resolution()

```
rx_err_t rx_ds18b20_set_resolution (
                rx_ds18b20_handle_t * handle,
                const ds18b20_resolution_t resolution)   [nodiscard]
```

Set ADC measurement resolution on DS18B20 sensor.

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion
- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via internal_ds18b20_resolution_to_config()
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Precondition

handle must be initialized via rx_ds18b20_init()

resolution must be one of k_ds18b20_resolution_9bit/10bit/11bit/12bit

Postcondition

handle->resolution reflects the newly configured value on k_rx_ok

DS18B20 scratchpad configuration register updated with new resolution bits

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Change is not persistent across power cycles unless rx_ds18b20_save_config() is called

Example:

```
rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
if (err == k_rx_ok) {
    // Save to EEPROM so it persists across power cycles
    rx_ds18b20_save_config(&sensor);
}
```

See also

rx_ds18b20_get_resolution() Read back the currently configured resolution

rx_ds18b20_save_config() Persist resolution change to EEPROM

rx_ds18b20_get_conversion_time_ms() Get required conversion time for new resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

- Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

Definition at line 728 of file rx_ds18b20.c.

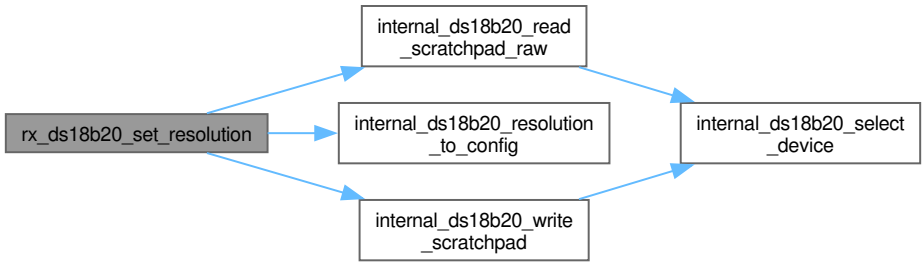
```
00730 {
00731   CHECK_DS18B20_HANDLE(handle, s_tag);
00732
00733   if (resolution != k_ds18b20_resolution_9bit && resolution != k_ds18b20_resolution_10bit &&
00734       resolution != k_ds18b20_resolution_11bit && resolution != k_ds18b20_resolution_12bit) {
00735     rx_log_error(s_tag, "Invalid resolution value");
00736     return k_rx_err_invalid_arg;
00737   }
00738
00739   /* Read current scratchpad */
00740   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00741   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00742   if (err != k_rx_ok) {
00743     return err;
00744   }
00745
```

```
00746   /* Generate new config byte */
00747   const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00748
00749   /* Write scratchpad with new config */
00750   ds18b20_scratchpad_write_t write_cfg;
00751   write_cfg.th     = scratchpad[k_ds18b20_scratch_th_reg];
00752   write_cfg.tl     = scratchpad[k_ds18b20_scratch_tl_reg];
00753   write_cfg.config = config;
00754   err              = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00755   if (err != k_rx_ok) {
00756     return err;
00757   }
00758
00759   /* Update handle */
00760   handle->resolution = resolution;
00761
00762   return k_rx_ok;
00763 }
```

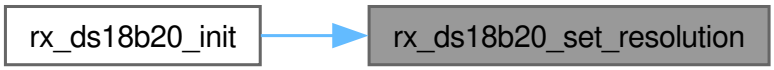
References CHECK_DS18B20_HANDLE, ds18b20_scratchpad_write_t::config, internal_ds18b20_read_scratchpad_ra internal_ds18b20_resolution_to_config(), internal_ds18b20_write_scratchpad(), k_ds18b20_resolution_10bit, k_ds18b20_resolution_11bit, k_ds18b20_resolution_12bit, k_ds18b20_resolution_9bit, k_ds18b20_scratch_th_reg, k_ds18b20_scratch_tl_reg, k_ds18b20_scratchpad_bytes, rx_ds18b20_handle_t::resolution, s_tag, ds18b20_scratchpad_write_t::th, and ds18b20_scratchpad_write_t::tl.

Referenced by rx_ds18b20_init().

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.4.20 rx˙ds18b20˙trigger˙conversion()

```
rx_err_t rx_ds18b20_trigger_conversion (
                const rx_ds18b20_handle_t * handle)   [nodiscard]
```

Trigger temperature conversion on DS18B20 sensor.

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with rx_ds18b20_read_temperature().

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Precondition

handle must be initialized via rx_ds18b20_init()
DS18B20 device must be physically connected and powered on the bus

Postcondition

DS18B20 ADC conversion is in progress
Caller must wait rx_ds18b20_get_conversion_time_ms() before reading

Note

Not thread-safe; caller must serialize access to the same handle

Warning

Do not call rx_ds18b20_read_temperature() before the conversion time elapses

Example:

```
rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
if (err == k_rx_ok) {
    tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
    float temp_celsius = 0.0f;
    err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
}
```

See also

rx_ds18b20_read_temperature() Read converted temperature in Celsius
rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion
rx_ds18b20_set_resolution() Configure ADC resolution

Since

Version 1.0.0

NASA Power of 10 Compliance:

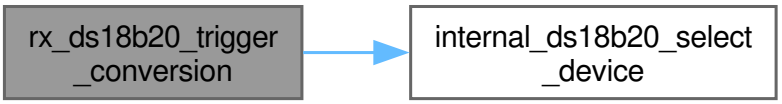
- Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

Definition at line 500 of file rx_ds18b20.c.

```
00501 {
00502   CHECK_DS18B20_HANDLE(handle, s_tag);
00503
00504   /* Reset and check presence */
00505   bool      presence = false;
00506   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00507   if (err != k_rx_ok || !presence) {
00508     rx_log_error(s_tag, "Device not present for conversion trigger");
00509     return k_rx_err_invalid_state;
00510   }
00511
00512   /* Select device */
00513   err = internal_ds18b20_select_device(handle);
00514   if (err != k_rx_ok) {
00515     return err;
00516   }
00517
00518   /* Send Convert T command */
00519   err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00520   if (err != k_rx_ok) {
00521     rx_log_error(s_tag, "Failed to send conversion command");
00522     return err;
00523   }
00524
00525   return k_rx_ok;
00526 }
```

References rx_ds18b20_handle_t::bus_manager, rx_ds18b20_handle_t::bus_name, CHECK_DS18B20_HANDLE, internal_ds18b20_select_device(), k_ds18b20_cmd_convert_t, and s_tag.

Here is the call graph for this function:



6.4 rx˙ds18b20.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_ds18b20.c
00003  * @brief DS18B20 1-Wire Digital Temperature Sensor Driver Implementation
00004  *
00005  * @details
00006  * Complete DS18B20 driver implementation with hardware abstraction via bus manager.
00007  * Supports all DS18B20 features: temperature measurement, resolution configuration,
00008  * ROM matching for multi-drop buses, and non-volatile EEPROM storage.
00009  *
00010  * ## Architecture
00011  *
00012  * **Dependency Injection Pattern:**
00013  * - Uses `rx_bus_manager_t` interface for 1-Wire communication
00014  * - Enables unit testing with mock bus implementations
00015  * - Zero direct hardware dependencies
00016  *
00017  * **Communication Flow:**
00018  * ```
00019  * [rx_ds18b20] -> [rx_bus_manager] -> [rx_bus_onewire] -> [GPIO HAL]
00020  * ```
```

```
00021  *
00022  * **Thread Safety:**
00023  * - NOT thread-safe - caller must provide synchronization
00024  * - Multiple handles can coexist (different sensors)
00025  * - Concurrent access to same handle causes race conditions
00026  *
00027  * ## NASA Power of 10 Compliance
00028  *
00029  * | Rule | Status | Implementation |
00030  * |------|--------|----------------|
00031  * | 1. Simple control flow | [OK] | No goto, no recursion |
00032  * | 2. Fixed loop bounds | [OK] | All loops bounded by enum constants |
00033  * | 3. No dynamic allocation | [OK] | Zero malloc/free |
00034  * | 4. Short functions | [OK] | Max 60 lines per function |
00035  * | 5. Assertions | [OK] | Min 2 checks per function (nullptr, state) |
00036  * | 6. Data scope | [OK] | Variables declared at smallest scope |
00037  * | 7. Check returns | [OK] | All function returns validated |
00038  * | 8. Limit preprocessor | [OK] | Typed enums only, no magic numbers |
00039  * | 9. Pointer restrictions | [OK] | Single-level dereferencing |
00040  * | 10. Compile warnings | [OK] | -Wall -Wextra -Werror |
00041  *
00042  * ## SOLID Principles
00043  *
00044  * **Single Responsibility:**
00045  * - One purpose: DS18B20 sensor communication
00046  * - Bus operations delegated to `rx_bus_onewire`
00047  * - CRC validation delegated to `rx_crc`
00048  *
00049  * **Open/Closed:**
00050  * - Extensible via bus manager interface
00051  * - New bus types added without modifying this driver
00052  *
00053  * **Liskov Substitution:**
00054  * - Any `rx_bus_manager_t` implementation works
00055  * - Mock bus substitutes for testing
00056  *
00057  * **Interface Segregation:**
00058  * - Clean API with focused functions
00059  * - Separate read/write/config operations
00060  *
00061  * **Dependency Inversion:**
00062  * - Depends on `rx_bus_manager_t` abstraction
00063  * - Bus manager injected via config structure
00064  *
00065  * ## Performance
00066  *
00067  * **Typical Operation Times:**
00068  * - Initialization: ~100ms (bus init + scratchpad read)
00069  * - Trigger conversion: ~2ms (command send)
00070  * - Read temperature: ~5ms (scratchpad read + CRC check)
00071  * - Resolution change: ~10ms (read + modify + write scratchpad)
00072  *
00073  * **Conversion Times (sensor internal):**
00074  * - 9-bit: 94ms
00075  * - 10-bit: 188ms
00076  * - 11-bit: 375ms
00077  * - 12-bit: 750ms (default)
00078  *
00079  * ## Memory Layout
00080  *
00081  * **DS18B20 Scratchpad Structure (9 bytes):**
00082  * ```
00083  * Byte | Field         | Description
00084  * -----|---------------|-----------------------------------------
00085  * 0    | Temp LSB      | Temperature low byte (1/16degC units)
00086  * 1    | Temp MSB      | Temperature high byte (sign-extended)
00087  * 2    | TH Register   | High alarm threshold (user-defined)
00088  * 3    | TL Register   | Low alarm threshold (user-defined)
00089  * 4    | Config        | Resolution bits R1:R0 (bits 6:5)
00090  * 5    | Reserved      | 0xFF (always)
00091  * 6    | Reserved      | Factory programmed (implementation-specific)
00092  * 7    | Reserved      | 0x10 (count remaining, legacy)
00093  * 8    | CRC           | CRC-8/MAXIM checksum
00094  * ```
00095  *
00096  * **Temperature Data Format:**
00097  * ```
00098  * 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
00099  * +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
00100  * | S| S| S| S| S| 2^6| 2^5| 2^4| 2^3| 2^2| 2^1| 2^0|2^-^1|2^-^2|2^-^3|2^-^4|
00101  * +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
00102  *   Sign extend    Integer part      Fractional (1/16degC)
00103  * ```
00104  *
00105  * **Resolution vs Temperature Bits:**
00106  * - 9-bit: Bits 15:3 valid, bits 2:0 undefined (mask 0xFFF8)
00107  * - 10-bit: Bits 15:2 valid, bits 1:0 undefined (mask 0xFFFC)
```

```
00108  * - 11-bit: Bits 15:1 valid, bit 0 undefined (mask 0xFFFE)
00109  * - 12-bit: All bits valid (mask 0xFFFF)
00110  *
00111  * ## Error Handling
00112  *
00113  * **Error Propagation Strategy:**
00114  * ```c
00115  * // Check bus errors -> return immediately
00116  * err = rx_bus_onewire_reset(...);
00117  * if (err != k_rx_ok) return err;
00118  *
00119  * // Check CRC -> return k_rx_err_crc_mismatch
00120  * if (crc_calc != crc_device) return k_rx_err_crc_mismatch;
00121  *
00122  * // Check range -> return k_rx_err_out_of_range
00123  * if (temp < MIN || temp > MAX) return k_rx_err_out_of_range;
00124  * ```
00125  *
00126  * **Common Error Codes:**
00127  * - `k_rx_err_null_ptr` - nullptr handle/config/output pointer
00128  * - `k_rx_err_invalid_state` - Sensor not initialized or not present
00129  * - `k_rx_err_invalid_arg` - Invalid resolution or ROM family code
00130  * - `k_rx_err_crc_mismatch` - Scratchpad CRC validation failed
00131  * - `k_rx_err_out_of_range` - Temperature outside -55degC to +125degC
00132  *
00133  * @par Example: Basic Temperature Reading
00134  * @code
00135  * rx_ds18b20_handle_t sensor;
00136  * rx_ds18b20_config_t config = {
00137  *     .bus_manager = &g_bus_manager,
00138  *     .bus_name = "onewire0",
00139  *     .resolution = k_ds18b20_resolution_12bit,
00140  *     .use_rom_matching = false  // Single sensor
00141  * };
00142  *
00143  * // 1. Initialize
00144  * rx_err_t err = rx_ds18b20_init(&sensor, &config);
00145  * if (err != k_rx_ok) handle_error(err);
00146  *
00147  * // 2. Trigger conversion
00148  * err = rx_ds18b20_trigger_conversion(&sensor);
00149  * if (err != k_rx_ok) handle_error(err);
00150  *
00151  * // 3. Wait for conversion (12-bit = 750ms)
00152  * delay_ms(rx_ds18b20_get_conversion_time_ms(&sensor));
00153  *
00154  * // 4. Read temperature
00155  * float temp_c = 0.0f;
00156  * err = rx_ds18b20_read_temperature(&sensor, &temp_c);
00157  * if (err == k_rx_ok) {
00158  *     printf("Temperature: %.2fdegC\n", temp_c);
00159  * }
00160  *
00161  * // 5. Cleanup
00162  * rx_ds18b20_deinit(&sensor);
00163  * @endcode
00164  *
00165  * @par Example: Multi-Drop Bus with ROM Matching
00166  * @code
00167  * // Read ROM codes first (use search algorithm or known values)
00168  * uint8_t sensor1_rom[8] = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE};
00169  * uint8_t sensor2_rom[8] = {0x28, 0xFE, 0xDC, 0xBA, 0x98, 0x76, 0x54, 0x32};
00170  *
00171  * rx_ds18b20_handle_t sensor1, sensor2;
00172  * rx_ds18b20_config_t config1 = {
00173  *     .bus_manager = &g_bus_manager,
00174  *     .bus_name = "onewire0",
00175  *     .resolution = k_ds18b20_resolution_12bit,
00176  *     .use_rom_matching = true,
00177  *     .rom = {0x28, 0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE}
00178  * };
00179  *
00180  * // Initialize both sensors on same bus
00181  * rx_ds18b20_init(&sensor1, &config1);
00182  * config1.rom = sensor2_rom;  // Reuse config, change ROM
00183  * rx_ds18b20_init(&sensor2, &config1);
00184  *
00185  * // Read from specific sensor
00186  * float temp1 = 0.0f;
00187  * rx_ds18b20_trigger_conversion(&sensor1);
00188  * delay_ms(750);
00189  * rx_ds18b20_read_temperature(&sensor1, &temp1);
00190  * @endcode
00191  *
00192  * @see rx_ds18b20.h
00193  * @see rx_bus_onewire.h
00194  * @see rx_crc.h
```

```
00195  *
00196  * @date 2026-01-05
00197  * @copyright Copyright (c) 2026 Locked Inc.
00198  * SPDX-License-Identifier: MIT
00199  */
00200
00201 #include "rx_ds18b20.h"
00202
00203 #include <math.h>
00204
00205 #include "rx_bus_onewire.h"
00206 #include "rx_check.h"
00207 #include "rx_crc.h"
00208 #include "rx_log.h"
00209
00210 /**
00211  * @defgroup ds18b20_internal_constants Internal Constants
00212  * @{
00213  */
00214
00215 /** @brief Logging tag for DS18B20 driver */
00216 static const char s_tag[] = "DS18B20";
00217
00218 /** @brief DS18B20 family code (byte 0 of ROM) */
00219 static const uint8_t s_ds18b20_family_code = 0x28U;
00220
00221 /**
00222  * @brief Temperature conversion divisor (raw to Celsius)
00223  * @details DS18B20 stores temperature in 1/16degC units. Divide by 16 to get Celsius.
00224  */
00225 static const float s_temp_conversion_divisor = 16.0F;
00226
00227 /**
00228  * @brief Reserved byte expected value in scratchpad
00229  * @details Byte 5 of scratchpad should always be 0xFF per datasheet.
00230  */
00231 static const uint8_t s_ds18b20_reserved_byte_value = 0xFFU;
00232
00233 /**
00234  * @brief Invalid conversion time sentinel
00235  * @details Returned by rx_ds18b20_get_conversion_time_ms() on error.
00236  */
00237 static const uint32_t s_ds18b20_conversion_time_invalid_u32 = 0U;
00238
00239 /** @} */
00240
00241 /*
      ============================================================================
00242  * Internal Validation Macros
00243  *
      ============================================================================
00244  */
00245
00246 /**
00247  * @def CHECK_DS18B20_HANDLE
00248  * @brief Validate DS18B20 handle pointer and initialization state
00249  *
00250  * @details
00251  * Composite validation macro that checks both nullptr and initialization.
00252  * Used at the start of all public API functions to enforce preconditions.
00253  *
00254  * **Checks Performed:**
00255  * 1. Handle is not nullptr (via RX_CHECK_NULL_PTR)
00256  * 2. Handle is initialized (handle->initialized == true)
00257  *
00258  * **Error Returns:**
00259  * - `k_rx_err_null_ptr` if handle is nullptr
00260  * - `k_rx_err_invalid_state` if not initialized
00261  *
00262  *
00263  * @par Example:
00264  * @code
00265  * rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temp)
00266  * {
00267  *     CHECK_DS18B20_HANDLE(handle, s_tag);  // Early return on error
00268  *     // ... function body ...
00269  * }
00270  * @endcode
00271  *
00272  * @note This is a statement macro (uses do-while(0) pattern) - requires semicolon.
00273  * @warning Do not use in expressions - this macro contains early returns.
00274  *
00275  * @see RX_CHECK_NULL_PTR
00276  */
00277 #define CHECK_DS18B20_HANDLE(handle, tag)                                   \
00278     do {                                                                    \
00279         RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");                \
```

```
00280     if (!(handle)->initialized) {                                                \
00281       rx_log_error(tag, "DS18B20 not initialized");                                \
00282       return k_rx_err_invalid_state;                                               \
00283     }                                                                            \
00284   } while (0)
00285
00286 /*
      ============================================================================
00287  * Internal Helper Functions
00288  *
      ============================================================================
00289  */
00290
00291 /**
00292  * @struct ds18b20_scratchpad_write_t
00293  * @brief Scratchpad write parameters for TH, TL, and config registers
00294  *
00295  * @details
00296  * Internal structure for writing DS18B20 scratchpad registers.
00297  * DS18B20 Write Scratchpad command (0x4E) writes exactly 3 bytes:
00298  * TH alarm, TL alarm, and configuration register.
00299  *
00300  * **Memory Layout (3 bytes total):**
00301  * ```
00302  * Offset | Field  | Description
00303  * -------|--------|-----------------------------------------------
00304  * 0      | th     | High temperature alarm threshold (-55 to +125degC)
00305  * 1      | tl     | Low temperature alarm threshold (-55 to +125degC)
00306  * 2      | config | Resolution bits R1:R0 at bits 6:5
00307  * ```
00308  *
00309  * **Configuration Register Format:**
00310  * ```
00311  * Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0
00312  * ------|-------|-------|-------|-------|-------|-------|------
00313  *   0   |  R1   |  R0   |   1   |   1   |   1   |   1   |   1
00314  *       +-------+-------+
00315  *       Resolution bits
00316  * ```
00317  *
00318  * **Resolution Encoding:**
00319  * - R1=0, R0=0: 9-bit (0.5degC, 93.75ms)
00320  * - R1=0, R0=1: 10-bit (0.25degC, 187.5ms)
00321  * - R1=1, R0=0: 11-bit (0.125degC, 375ms)
00322  * - R1=1, R0=1: 12-bit (0.0625degC, 750ms)
00323  *
00324  * @note This structure is for internal use only - not exposed in public API.
00325  * @see internal_ds18b20_write_scratchpad()
00326  * @see internal_ds18b20_resolution_to_config()
00327  */
00328 typedef struct {
00329   uint8_t th;     /**< High alarm threshold register (signed, stored as uint8_t) */
00330   uint8_t tl;     /**< Low alarm threshold register (signed, stored as uint8_t) */
00331   uint8_t config; /**< Configuration register (bits 6:5 = resolution) */
00332 } ds18b20_scratchpad_write_t;
00333
00334 static rx_err_t internal_ds18b20_validate_config(const rx_ds18b20_config_t* config,
00335                                                  const rx_ds18b20_handle_t* handle);
00336 static rx_err_t internal_ds18b20_verify_device_presence(rx_ds18b20_handle_t* handle);
00337 static rx_err_t internal_ds18b20_select_device(const rx_ds18b20_handle_t* handle);
00338 static rx_err_t
00339               internal_ds18b20_read_scratchpad_raw(const rx_ds18b20_handle_t* handle,
00340                                                    uint8_t scratchpad[k_ds18b20_scratchpad_bytes]);
00341 static rx_err_t internal_ds18b20_write_scratchpad(const rx_ds18b20_handle_t*        handle,
00342                                                   const ds18b20_scratchpad_write_t* scratchpad);
00343 static uint8_t  internal_ds18b20_resolution_to_config(ds18b20_resolution_t resolution);
00344 static uint16_t internal_ds18b20_get_temp_mask(ds18b20_resolution_t resolution);
00345 static float    internal_ds18b20_raw_to_celsius(int16_t raw_temp);
00346
00347 /*
      ============================================================================
00348  * Public API Implementation
00349  *
      ============================================================================
00350  */
00351
00352 rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t* handle, const rx_ds18b20_config_t* config)
00353 {
00354   /* Validate inputs */
00355   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00356
00357   rx_err_t err = internal_ds18b20_validate_config(config, handle);
00358   if (err != k_rx_ok) {
00359     return err;
00360   }
00361
00362   /* Initialize handle */
```

```
00363   *handle                  = (rx_ds18b20_handle_t){};
00364   handle->bus_manager      = config->bus_manager;
00365   handle->bus_name         = config->bus_name;
00366   handle->resolution       = config->resolution;
00367   handle->use_rom_matching = config->use_rom_matching;
00368
00369   if (config->use_rom_matching) {
00370     for (uint8_t i = 0; i < k_onewire_rom_bytes; ++i) {
00371       handle->rom[i] = config->rom[i];
00372     }
00373   }
00374
00375   /* Verify device presence */
00376   err = internal_ds18b20_verify_device_presence(handle);
00377   if (err != k_rx_ok) {
00378     return err;
00379   }
00380
00381   /* Configure resolution */
00382   handle->initialized = true;
00383   err                 = rx_ds18b20_set_resolution(handle, config->resolution);
00384   if (err != k_rx_ok) {
00385     handle->initialized = false;
00386     return err;
00387   }
00388
00389   rx_log_info(s_tag, "DS18B20 initialized successfully");
00390   return k_rx_ok;
00391 }
00392
00393 /**
00394  * @brief Deinitialize DS18B20 sensor handle and release all resources
00395  *
00396  * @details
00397  * Tears down a previously initialized DS18B20 handle by zeroing all internal
00398  * state via memset. After this call the handle is safe to reinitialize with
00399  * rx_ds18b20_init().
00400  *
00401  * Algorithm steps:
00402  * 1. Validate handle is not nullptr
00403  * 2. Verify handle is currently initialized
00404  * 3. Clear the entire handle structure with memset (zeros all fields)
00405  * 4. Log deinitialization success
00406  *
00407  *
00408  *
00409  * @pre handle must be a valid non-null pointer
00410  * @pre handle must have been successfully initialized via rx_ds18b20_init()
00411  * @post All handle fields are zeroed (initialized flag is false)
00412  * @post handle may be safely reused with rx_ds18b20_init()
00413  *
00414  * @note Not thread-safe; caller must ensure no concurrent access to handle
00415  *
00416  * @par Example:
00417  * @code
00418  * rx_ds18b20_handle_t sensor;
00419  * rx_ds18b20_init(&sensor, &config);
00420  * // ... use sensor ...
00421  * rx_err_t err = rx_ds18b20_deinit(&sensor);
00422  * if (err != k_rx_ok) {
00423  *     rx_log_error("app", "Failed to deinit DS18B20");
00424  * }
00425  * @endcode
00426  *
00427  * @see rx_ds18b20_init() Initialize the handle before use
00428  *
00429  * @since Version 1.0.0
00430  *
00431  * @par NASA Power of 10 Compliance:
00432  * - Rule 5: 2 preconditions (null check, initialized check), 2 postconditions
00433  */
00434 rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t* handle)
00435 {
00436   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00437
00438   if (!handle->initialized) {
00439     rx_log_warn(s_tag, "DS18B20 not initialized");
00440     return k_rx_err_invalid_state;
00441   }
00442
00443   /* Clear handle */
00444   *handle = (rx_ds18b20_handle_t){};
00445
00446   rx_log_info(s_tag, "DS18B20 deinitialized");
00447   return k_rx_ok;
00448 }
00449
```

```
00450 /**
00451  * @brief Trigger temperature conversion on DS18B20 sensor
00452  *
00453  * @details
00454  * Issues the Convert T (0x44) command over the 1-Wire bus to start an
00455  * autonomous temperature measurement. The DS18B20 will perform the ADC
00456  * conversion internally; the caller must wait the appropriate conversion
00457  * time before reading results with rx_ds18b20_read_temperature().
00458  *
00459  * Algorithm steps:
00460  * 1. Validate handle is initialized and non-null
00461  * 2. Issue 1-Wire bus reset and verify device presence pulse
00462  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00463  * 4. Write the Convert T command byte (0x44) to the bus
00464  *
00465  * Conversion time depends on resolution:
00466  * - 9-bit:  94 ms
00467  * - 10-bit: 188 ms
00468  * - 11-bit: 375 ms
00469  * - 12-bit: 750 ms
00470  *
00471  *
00472  *
00473  * @pre handle must be initialized via rx_ds18b20_init()
00474  * @pre DS18B20 device must be physically connected and powered on the bus
00475  * @post DS18B20 ADC conversion is in progress
00476  * @post Caller must wait rx_ds18b20_get_conversion_time_ms() before reading
00477  *
00478  * @note Not thread-safe; caller must serialize access to the same handle
00479  * @warning Do not call rx_ds18b20_read_temperature() before the conversion time elapses
00480  *
00481  * @par Example:
00482  * @code
00483  * rx_err_t err = rx_ds18b20_trigger_conversion(&sensor);
00484  * if (err == k_rx_ok) {
00485  *     tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor));
00486  *     float temp_celsius = 0.0f;
00487  *     err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00488  * }
00489  * @endcode
00490  *
00491  * @see rx_ds18b20_read_temperature() Read converted temperature in Celsius
00492  * @see rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion
00493  * @see rx_ds18b20_set_resolution() Configure ADC resolution
00494  *
00495  * @since Version 1.0.0
00496  *
00497  * @par NASA Power of 10 Compliance:
00498  * - Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions
00499  */
00500 rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t* handle)
00501 {
00502     CHECK_DS18B20_HANDLE(handle, s_tag);
00503
00504     /* Reset and check presence */
00505     bool     presence = false;
00506     rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00507     if (err != k_rx_ok || !presence) {
00508         rx_log_error(s_tag, "Device not present for conversion trigger");
00509         return k_rx_err_invalid_state;
00510     }
00511
00512     /* Select device */
00513     err = internal_ds18b20_select_device(handle);
00514     if (err != k_rx_ok) {
00515         return err;
00516     }
00517
00518     /* Send Convert T command */
00519     err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_convert_t);
00520     if (err != k_rx_ok) {
00521         rx_log_error(s_tag, "Failed to send conversion command");
00522         return err;
00523     }
00524
00525     return k_rx_ok;
00526 }
00527
00528 /**
00529  * @brief Read temperature from DS18B20 sensor in degrees Celsius
00530  *
00531  * @details
00532  * Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw
00533  * temperature value, applies the resolution mask to clear undefined low-order
00534  * bits, validates the result is within the sensor's physical range, and
00535  * converts to a floating-point Celsius value using the DS18B20 fixed-point
00536  * encoding (1 LSB = 1/16 degC).
```

```
00537  *
00538  * Algorithm steps:
00539  * 1. Validate handle and output pointer are non-null
00540  * 2. Call rx_ds18b20_read_temperature_raw() to obtain masked raw value
00541  * 3. Convert raw signed 16-bit value to float: celsius = raw / 16.0f
00542  * 4. Write result to *temperature_celsius
00543  *
00544  *
00545  *
00546  * @pre handle must be initialized via rx_ds18b20_init()
00547  * @pre rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed
00548  * @post *temperature_celsius contains valid temperature in [-55.0, +125.0] on k_rx_ok
00549  * @post handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)
00550  *
00551  * @note Not thread-safe; caller must serialize access to the same handle
00552  * @warning Returns stale data if conversion time has not elapsed after trigger
00553  *
00554  * @par Example:
00555  * @code
00556  * float temp_celsius = 0.0f;
00557  * rx_err_t err = rx_ds18b20_read_temperature(&sensor, &temp_celsius);
00558  * if (err == k_rx_ok) {
00559  *     rx_log_info("app", "Temperature: %.4f C", (double)temp_celsius);
00560  * }
00561  * @endcode
00562  *
00563  * @see rx_ds18b20_trigger_conversion() Must be called before reading
00564  * @see rx_ds18b20_read_temperature_raw() Read the raw 16-bit value
00565  * @see rx_ds18b20_get_conversion_time_ms() Determine required wait time
00566  *
00567  * @since Version 1.0.0
00568  *
00569  * @par NASA Power of 10 Compliance:
00570  * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
00571  */
00572 rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t* handle, float* temperature_celsius)
00573 {
00574     CHECK_DS18B20_HANDLE(handle, s_tag);
00575     RX_CHECK_NULL_PTR(temperature_celsius, s_tag, "temperature_celsius is nullptr");
00576
00577     /* Read raw temperature */
00578     int16_t  raw_temp = 0;
00579     rx_err_t err      = rx_ds18b20_read_temperature_raw(handle, &raw_temp);
00580     if (err != k_rx_ok) {
00581         return err;
00582     }
00583
00584     /* Convert to Celsius */
00585     *temperature_celsius = internal_ds18b20_raw_to_celsius(raw_temp);
00586
00587     return k_rx_ok;
00588 }
00589
00590 /**
00591  * @brief Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units
00592  *
00593  * @details
00594  * Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature
00595  * bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit
00596  * unsigned value, applies the resolution-dependent mask to clear undefined
00597  * low-order bits, casts to signed int16_t, and range-validates the result.
00598  *
00599  * The DS18B20 temperature encoding uses a signed fixed-point format where
00600  * 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:
00601  *   - Minimum: -55 degC = 0xFC90 (int16: -880)
00602  *   - Maximum: +125 degC = 0x07D0 (int16: +2000)
00603  *
00604  * Resolution masks (clears undefined bits in lower-resolution modes):
00605  * - 9-bit:  0xFFF8 (3 undefined bits)
00606  * - 10-bit: 0xFFFC (2 undefined bits)
00607  * - 11-bit: 0xFFFE (1 undefined bit)
00608  * - 12-bit: 0xFFFF (no undefined bits)
00609  *
00610  * Algorithm steps:
00611  * 1. Validate handle and output pointer are non-null and handle is initialized
00612  * 2. Read raw scratchpad bytes via internal_ds18b20_read_scratchpad_raw()
00613  * 3. Combine temperature LSB and MSB bytes into uint16_t
00614  * 4. Apply resolution mask from internal_ds18b20_get_temp_mask()
00615  * 5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
00616  * 6. Write validated result to *raw_temp
00617  *
00618  *
00619  *
00620  * @pre handle must be initialized via rx_ds18b20_init()
00621  * @pre rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed
00622  * @post *raw_temp is in [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw] on k_rx_ok
00623  * @post Resolution mask has been applied; undefined bits are cleared
```

```
00624  *
00625  * @note Not thread-safe; caller must serialize access to the same handle
00626  * @warning Returns stale data if conversion time has not elapsed after trigger
00627  *
00628  * @par Example:
00629  * @code
00630  * int16_t raw = 0;
00631  * rx_err_t err = rx_ds18b20_read_temperature_raw(&sensor, &raw);
00632  * if (err == k_rx_ok) {
00633  *     float celsius = (float)raw / 16.0f;
00634  * }
00635  * @endcode
00636  *
00637  * @see rx_ds18b20_read_temperature() Convenience wrapper returning float Celsius
00638  * @see rx_ds18b20_trigger_conversion() Must be called before reading
00639  * @see rx_ds18b20_set_resolution() Configure ADC resolution and mask
00640  *
00641  * @since Version 1.0.0
00642  *
00643  * @par NASA Power of 10 Compliance:
00644  * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)
00645  */
00646 rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t* handle, int16_t* raw_temp)
00647 {
00648   CHECK_DS18B20_HANDLE(handle, s_tag);
00649   RX_CHECK_NULL_PTR(raw_temp, s_tag, "raw_temp is nullptr");
00650
00651   /* Read scratchpad */
00652   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00653   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00654   if (err != k_rx_ok) {
00655     return err;
00656   }
00657
00658   /* Extract temperature (LSB + MSB) */
00659   uint16_t temp = (uint16_t)scratchpad[k_ds18b20_scratch_temp_lsb];
00660   temp |= ((uint16_t)scratchpad[k_ds18b20_scratch_temp_msb]) « k_ds18b20_shift_byte;
00661
00662   /* Apply resolution mask to clear undefined bits */
00663   const uint16_t mask = internal_ds18b20_get_temp_mask(handle->resolution);
00664   temp &= mask;
00665
00666   /* Post-condition: Validate temperature is within sensor range */
00667   *raw_temp = (int16_t)temp;
00668   if (*raw_temp < k_ds18b20_temp_min_raw || *raw_temp > k_ds18b20_temp_max_raw) {
00669     rx_log_error(s_tag, "Temperature out of sensor range (-55degC to +125degC)");
00670     return k_rx_err_out_of_range;
00671   }
00672
00673   return k_rx_ok;
00674 }
00675
00676 /**
00677  * @brief Set ADC measurement resolution on DS18B20 sensor
00678  *
00679  * @details
00680  * Updates the configuration register in the DS18B20 scratchpad to change
00681  * the ADC resolution. The existing TH and TL alarm threshold bytes are
00682  * preserved by reading the current scratchpad first, then writing back
00683  * only the updated configuration byte.
00684  *
00685  * Higher resolution provides finer temperature steps but increases
00686  * conversion time:
00687  * - 9-bit:  0.5 degC steps,  94 ms conversion
00688  * - 10-bit: 0.25 degC steps, 188 ms conversion
00689  * - 11-bit: 0.125 degC steps, 375 ms conversion
00690  * - 12-bit: 0.0625 degC steps, 750 ms conversion
00691  *
00692  * Algorithm steps:
00693  * 1. Validate handle is initialized and resolution is a legal enum value
00694  * 2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
00695  * 3. Convert resolution enum to configuration register byte via
00696  *    internal_ds18b20_resolution_to_config()
00697  * 4. Write scratchpad with new config byte (TH/TL unchanged)
00698  * 5. Update handle->resolution to reflect new setting
00699  *
00700  *
00701  *
00702  * @pre handle must be initialized via rx_ds18b20_init()
00703  * @pre resolution must be one of k_ds18b20_resolution_9bit/10bit/11bit/12bit
00704  * @post handle->resolution reflects the newly configured value on k_rx_ok
00705  * @post DS18B20 scratchpad configuration register updated with new resolution bits
00706  *
00707  * @note Not thread-safe; caller must serialize access to the same handle
00708  * @warning Change is not persistent across power cycles unless rx_ds18b20_save_config() is called
00709  *
00710  * @par Example:
```

```
00711  * @code
00712  * rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
00713  * if (err == k_rx_ok) {
00714  *     // Save to EEPROM so it persists across power cycles
00715  *     rx_ds18b20_save_config(&sensor);
00716  * }
00717  * @endcode
00718  *
00719  * @see rx_ds18b20_get_resolution() Read back the currently configured resolution
00720  * @see rx_ds18b20_save_config() Persist resolution change to EEPROM
00721  * @see rx_ds18b20_get_conversion_time_ms() Get required conversion time for new resolution
00722  *
00723  * @since Version 1.0.0
00724  *
00725  * @par NASA Power of 10 Compliance:
00726  * - Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions
00727  */
00728 rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t*       handle,
00729                                    const ds18b20_resolution_t resolution)
00730 {
00731   CHECK_DS18B20_HANDLE(handle, s_tag);
00732
00733   if (resolution != k_ds18b20_resolution_9bit && resolution != k_ds18b20_resolution_10bit &&
00734       resolution != k_ds18b20_resolution_11bit && resolution != k_ds18b20_resolution_12bit) {
00735     rx_log_error(s_tag, "Invalid resolution value");
00736     return k_rx_err_invalid_arg;
00737   }
00738
00739   /* Read current scratchpad */
00740   uint8_t  scratchpad[k_ds18b20_scratchpad_bytes];
00741   rx_err_t err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
00742   if (err != k_rx_ok) {
00743     return err;
00744   }
00745
00746   /* Generate new config byte */
00747   const uint8_t config = internal_ds18b20_resolution_to_config(resolution);
00748
00749   /* Write scratchpad with new config */
00750   ds18b20_scratchpad_write_t write_cfg;
00751   write_cfg.th     = scratchpad[k_ds18b20_scratch_th_reg];
00752   write_cfg.tl     = scratchpad[k_ds18b20_scratch_tl_reg];
00753   write_cfg.config = config;
00754   err              = internal_ds18b20_write_scratchpad(handle, &write_cfg);
00755   if (err != k_rx_ok) {
00756     return err;
00757   }
00758
00759   /* Update handle */
00760   handle->resolution = resolution;
00761
00762   return k_rx_ok;
00763 }
00764
00765 /**
00766  * @brief Get the currently configured ADC resolution from DS18B20 handle
00767  *
00768  * @details
00769  * Returns the resolution value stored in the handle, which reflects the
00770  * last successfully applied resolution (set during initialization or via
00771  * rx_ds18b20_set_resolution()). This is a cached read from the handle
00772  * state; it does not issue a 1-Wire bus transaction.
00773  *
00774  * Algorithm steps:
00775  * 1. Validate handle and output pointer are non-null
00776  * 2. Verify handle is initialized
00777  * 3. Copy handle->resolution to *resolution
00778  *
00779  *
00780  *
00781  * @pre handle must be initialized via rx_ds18b20_init()
00782  * @pre resolution must be a valid non-null pointer
00783  * @post *resolution contains the current resolution value on k_rx_ok
00784  * @post handle state is unchanged
00785  *
00786  * @note Not thread-safe; caller must ensure handle is not concurrently modified
00787  * @note Returns cached value from handle -- does not issue 1-Wire bus transaction
00788  *
00789  * @par Example:
00790  * @code
00791  * ds18b20_resolution_t res;
00792  * rx_err_t err = rx_ds18b20_get_resolution(&sensor, &res);
00793  * if (err == k_rx_ok) {
00794  *     uint32_t conv_ms = rx_ds18b20_get_conversion_time_ms(&sensor);
00795  *     rx_log_info("app", "Resolution %d, conv time %u ms", (int)res, (unsigned)conv_ms);
00796  * }
00797  * @endcode
```

```
00798  *
00799  * @see rx_ds18b20_set_resolution() Change the ADC resolution
00800  * @see rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution
00801  *
00802  * @since Version 1.0.0
00803  *
00804  * @par NASA Power of 10 Compliance:
00805  * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
00806  */
00807 rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t* handle,
00808                                    ds18b20_resolution_t*      resolution)
00809 {
00810   CHECK_DS18B20_HANDLE(handle, s_tag);
00811   RX_CHECK_NULL_PTR(resolution, s_tag, "resolution is nullptr");
00812
00813   *resolution = handle->resolution;
00814   return k_rx_ok;
00815 }
00816
00817 /**
00818  * @brief Save current scratchpad configuration to DS18B20 EEPROM
00819  *
00820  * @details
00821  * Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which
00822  * instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration
00823  * register bytes from volatile scratchpad RAM to non-volatile EEPROM. The
00824  * saved values persist across power cycles and are automatically restored
00825  * on the next power-up via the Recall E2 sequence.
00826  *
00827  * Algorithm steps:
00828  * 1. Validate handle is initialized and non-null
00829  * 2. Issue 1-Wire bus reset and verify device presence pulse
00830  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00831  * 4. Write the Copy Scratchpad command byte (0x48)
00832  *
00833  *
00834  *
00835  * @pre handle must be initialized via rx_ds18b20_init()
00836  * @pre DS18B20 device must be physically connected and externally powered
00837  *      (Copy Scratchpad is not guaranteed in parasitic power mode)
00838  * @post DS18B20 EEPROM contains current TH, TL, and configuration register values
00839  * @post Configuration will survive next power cycle
00840  *
00841  * @note Not thread-safe; caller must serialize access to the same handle
00842  * @warning Not reliable in parasitic power mode; use external power for EEPROM writes
00843  *
00844  * @par Example:
00845  * @code
00846  * // Configure resolution, then persist to EEPROM
00847  * rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);
00848  * rx_err_t err = rx_ds18b20_save_config(&sensor);
00849  * if (err != k_rx_ok) {
00850  *     rx_log_error("app", "Failed to save DS18B20 config to EEPROM");
00851  * }
00852  * @endcode
00853  *
00854  * @see rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad
00855  * @see rx_ds18b20_set_resolution() Configure resolution before saving
00856  *
00857  * @since Version 1.0.0
00858  *
00859  * @par NASA Power of 10 Compliance:
00860  * - Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions
00861  */
00862 rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t* handle)
00863 {
00864   CHECK_DS18B20_HANDLE(handle, s_tag);
00865
00866   /* Reset and check presence */
00867   bool     presence = false;
00868   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00869   if (err != k_rx_ok || !presence) {
00870     rx_log_error(s_tag, "Device not present for EEPROM save");
00871     return k_rx_err_invalid_state;
00872   }
00873
00874   /* Select device */
00875   err = internal_ds18b20_select_device(handle);
00876   if (err != k_rx_ok) {
00877     return err;
00878   }
00879
00880   /* Send Copy Scratchpad command */
00881   err =
00882     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_copy_scratch);
00883   if (err != k_rx_ok) {
00884     rx_log_error(s_tag, "Failed to send copy scratchpad command");
```

```
00885     return err;
00886   }
00887
00888   return k_rx_ok;
00889 }
00890
00891 /**
00892  * @brief Recall configuration from DS18B20 EEPROM into scratchpad RAM
00893  *
00894  * @details
00895  * Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs
00896  * the DS18B20 to copy the TH alarm, TL alarm, and configuration register
00897  * bytes from non-volatile EEPROM back into volatile scratchpad RAM. This
00898  * is the same operation the device performs automatically on power-up.
00899  *
00900  * Use this function to discard unsaved scratchpad changes and revert to the
00901  * last saved EEPROM state without cycling power.
00902  *
00903  * Algorithm steps:
00904  * 1. Validate handle is initialized and non-null
00905  * 2. Issue 1-Wire bus reset and verify device presence pulse
00906  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00907  * 4. Write the Recall E2 command byte (0xB8)
00908  *
00909  *
00910  *
00911  * @pre handle must be initialized via rx_ds18b20_init()
00912  * @pre DS18B20 device must be physically connected to the bus
00913  * @post DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save
00914  * @post Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded
00915  *
00916  * @note Not thread-safe; caller must serialize access to the same handle
00917  * @note The Recall E2 operation is performed automatically by the device on power-up
00918  *
00919  * @par Example:
00920  * @code
00921  * // Revert any unsaved configuration changes
00922  * rx_err_t err = rx_ds18b20_recall_config(&sensor);
00923  * if (err != k_rx_ok) {
00924  *     rx_log_error("app", "Failed to recall DS18B20 config from EEPROM");
00925  * }
00926  * @endcode
00927  *
00928  * @see rx_ds18b20_save_config() Persist scratchpad configuration to EEPROM
00929  * @see rx_ds18b20_set_resolution() Modify the scratchpad configuration
00930  *
00931  * @since Version 1.0.0
00932  *
00933  * @par NASA Power of 10 Compliance:
00934  * - Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions
00935  */
00936 rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t* handle)
00937 {
00938   CHECK_DS18B20_HANDLE(handle, s_tag);
00939
00940   /* Reset and check presence */
00941   bool     presence = false;
00942   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
00943   if (err != k_rx_ok || !presence) {
00944     rx_log_error(s_tag, "Device not present for EEPROM recall");
00945     return k_rx_err_invalid_state;
00946   }
00947
00948   /* Select device */
00949   err = internal_ds18b20_select_device(handle);
00950   if (err != k_rx_ok) {
00951     return err;
00952   }
00953
00954   /* Send Recall E2 command */
00955   err =
00956     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_recall_eeprom);
00957   if (err != k_rx_ok) {
00958     rx_log_error(s_tag, "Failed to send recall EEPROM command");
00959     return err;
00960   }
00961
00962   return k_rx_ok;
00963 }
00964
00965 /**
00966  * @brief Read DS18B20 power supply mode (external vs. parasitic)
00967  *
00968  * @details
00969  * Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads
00970  * back a single bit to determine whether the DS18B20 is operating on an
00971  * external voltage supply or in parasitic (bus-powered) mode.
```

```
00972  *
00973  * Per the DS18B20 datasheet: the sensor pulls the bus low for one read time
00974  * slot to indicate parasitic power mode (0), or releases the bus (1) to
00975  * indicate external power mode.
00976  *
00977  * Algorithm steps:
00978  * 1. Validate handle and output pointer are non-null; handle is initialized
00979  * 2. Issue 1-Wire bus reset and verify device presence pulse
00980  * 3. Select the target device (Match ROM or Skip ROM depending on config)
00981  * 4. Write the Read Power Supply command byte (0xB4)
00982  * 5. Read one bit from the bus (0 = parasitic, 1 = external)
00983  * 6. Write result to *external_power
00984  *
00985  *
00986  *
00987  * @pre handle must be initialized via rx_ds18b20_init()
00988  * @pre DS18B20 device must be physically connected to the bus
00989  * @post *external_power reflects the device power supply mode on k_rx_ok
00990  * @post handle state is unchanged
00991  *
00992  * @note Not thread-safe; caller must serialize access to the same handle
00993  * @warning Some operations (Copy Scratchpad) are unreliable in parasitic power mode
00994  *
00995  * @par Example:
00996  * @code
00997  * bool ext_power = false;
00998  * rx_err_t err = rx_ds18b20_read_power_mode(&sensor, &ext_power);
00999  * if (err == k_rx_ok && !ext_power) {
01000  *     rx_log_warn("app", "DS18B20 in parasitic mode: EEPROM writes may fail");
01001  * }
01002  * @endcode
01003  *
01004  * @see rx_ds18b20_save_config() EEPROM write (requires external power)
01005  * @see rx_ds18b20_init() Initialize sensor handle
01006  *
01007  * @since Version 1.0.0
01008  *
01009  * @par NASA Power of 10 Compliance:
01010  * - Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions
01011  */
01012 rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t* handle, bool* external_power)
01013 {
01014   CHECK_DS18B20_HANDLE(handle, s_tag);
01015   RX_CHECK_NULL_PTR(external_power, s_tag, "external_power is nullptr");
01016
01017   /* Reset and check presence */
01018   bool     presence = false;
01019   rx_err_t err      = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01020   if (err != k_rx_ok || !presence) {
01021     rx_log_error(s_tag, "Device not present for power mode read");
01022     return k_rx_err_invalid_state;
01023   }
01024
01025   /* Select device */
01026   err = internal_ds18b20_select_device(handle);
01027   if (err != k_rx_ok) {
01028     return err;
01029   }
01030
01031   /* Send Read Power Supply command */
01032   err = rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_power);
01033   if (err != k_rx_ok) {
01034     rx_log_error(s_tag, "Failed to send read power command");
01035     return err;
01036   }
01037
01038   /* Read power bit (1 = external, 0 = parasitic) */
01039   *external_power = false;
01040   err             = rx_bus_onewire_read_bit(handle->bus_manager, handle->bus_name, external_power);
01041   if (err != k_rx_ok) {
01042     rx_log_error(s_tag, "Failed to read power bit");
01043     return err;
01044   }
01045
01046   return k_rx_ok;
01047 }
01048
01049 /**
01050  * @brief Read the full 9-byte scratchpad register from DS18B20 sensor
01051  *
01052  * @details
01053  * Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw
01054  * buffer to the caller. The scratchpad layout is:
01055  *
01056  * | Byte | Field          | Description                      |
01057  * |------|----------------|----------------------------------|
01058  * |  0   | Temperature LSB | Low byte of temperature register  |
```

```
01059  * |  1   | Temperature MSB | High byte of temperature register |
01060  * |  2   | TH Register     | High alarm threshold              |
01061  * |  3   | TL Register     | Low alarm threshold               |
01062  * |  4   | Config Register | Resolution configuration byte     |
01063  * |  5-7 | Reserved        | Reserved (0xFF, 0x10, 0x10)       |
01064  * |  8   | CRC             | CRC-8 of bytes 0-7                |
01065  *
01066  * CRC is verified internally by internal_ds18b20_read_scratchpad_raw() before
01067  * the buffer is returned. This function is a thin public wrapper around the
01068  * internal helper.
01069  *
01070  * Algorithm steps:
01071  * 1. Validate handle and scratchpad buffer pointer are non-null
01072  * 2. Verify handle is initialized
01073  * 3. Delegate to internal_ds18b20_read_scratchpad_raw() which issues the
01074  *    1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command,
01075  *    reads 9 bytes, and verifies CRC-8
01076  *
01077  *
01078  *
01079  * @pre handle must be initialized via rx_ds18b20_init()
01080  * @pre scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes
01081  * @post scratchpad[0..8] contains valid data on k_rx_ok
01082  * @post CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok
01083  *
01084  * @note Not thread-safe; caller must serialize access to the same handle
01085  * @note Use rx_ds18b20_read_temperature() for converted Celsius value
01086  *
01087  * @par Example:
01088  * @code
01089  * uint8_t raw[k_ds18b20_scratchpad_bytes];
01090  * rx_err_t err = rx_ds18b20_read_scratchpad(&sensor, raw);
01091  * if (err == k_rx_ok) {
01092  *     uint8_t config_reg = raw[4];
01093  * }
01094  * @endcode
01095  *
01096  * @see rx_ds18b20_read_temperature() High-level temperature read returning float Celsius
01097  * @see rx_ds18b20_read_temperature_raw() Read raw 16-bit temperature with masking
01098  *
01099  * @since Version 1.0.0
01100  *
01101  * @par NASA Power of 10 Compliance:
01102  * - Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions
01103  */
01104 rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t* handle,
01105                                     uint8_t              scratchpad[k_ds18b20_scratchpad_bytes])
01106 {
01107     CHECK_DS18B20_HANDLE(handle, s_tag);
01108     RX_CHECK_NULL_PTR(scratchpad, s_tag, "scratchpad is nullptr");
01109
01110     return internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01111 }
01112
01113 /**
01114  * @brief Get temperature conversion time for current resolution
01115  *
01116  * Returns the required wait time after triggering conversion based on
01117  * the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).
01118  *
01119  *
01120  */
01121 uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t* handle)
01122 {
01123     if (handle == nullptr || !handle->initialized) {
01124         return s_ds18b20_conversion_time_invalid_u32;
01125     }
01126
01127     switch (handle->resolution) {
01128         case k_ds18b20_resolution_9bit:
01129             return k_ds18b20_conv_time_9bit_ms;
01130         case k_ds18b20_resolution_10bit:
01131             return k_ds18b20_conv_time_10bit_ms;
01132         case k_ds18b20_resolution_11bit:
01133             return k_ds18b20_conv_time_11bit_ms;
01134         case k_ds18b20_resolution_12bit:
01135             return k_ds18b20_conv_time_12bit_ms;
01136         default:
01137             return s_ds18b20_conversion_time_invalid_u32;
01138     }
01139 }
01140
01141 /*
     ============================================================================
01142  * Internal Helper Functions Implementation
01143  *
     ============================================================================
```

```
01144  */
01145
01146 /**
01147  * @brief Validate DS18B20 configuration parameters before initialization
01148  *
01149  * @details
01150  * Comprehensive validation of configuration structure and handle state.
01151  * Called by rx_ds18b20_init() to enforce preconditions and prevent
01152  * misconfiguration.
01153  *
01154  * **Algorithm:**
01155  * 1. Check config pointer is not nullptr
01156  * 2. Check bus_manager pointer is not nullptr
01157  * 3. Check bus_name string is not nullptr
01158  * 4. Verify handle is not already initialized (prevent re-initialization)
01159  * 5. Validate resolution is in valid range (0-3)
01160  * 6. If ROM matching enabled, verify family code is 0x28 (DS18B20)
01161  *
01162  * **Validation Rules:**
01163  * ```
01164  * +----------------+---------------+--------------------+
01165  * | Field          | Check         | Error Code         |
01166  * +----------------+---------------+--------------------+
01167  * | config         | != nullptr    | k_rx_err_null_ptr  |
01168  * | bus_manager    | != nullptr    | k_rx_err_null_ptr  |
01169  * | bus_name       | != nullptr    | k_rx_err_null_ptr  |
01170  * | initialized    | == false      | k_rx_err_invalid_state |
01171  * | resolution     | <= 3          | k_rx_err_invalid_arg|
01172  * | rom[0] (if ROM) | == 0x28      | k_rx_err_invalid_arg|
01173  * +----------------+---------------+--------------------+
01174  * ```
01175  *
01176  *
01177  *
01178  * @pre config != nullptr
01179  * @pre handle != nullptr (checked by caller)
01180  * @post If k_rx_ok returned, all fields are valid for initialization
01181  *
01182  * @note This is an internal helper - not exposed in public API
01183  * @warning Do not call after initialization - will return error
01184  *
01185  * @see rx_ds18b20_init()
01186  * @see ds18b20_resolution_t
01187  */
01188 static rx_err_t internal_ds18b20_validate_config(const rx_ds18b20_config_t* config,
01189                                                  const rx_ds18b20_handle_t* handle)
01190 {
01191     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
01192     RX_CHECK_NULL_PTR(config->bus_manager, s_tag, "bus_manager is nullptr");
01193     RX_CHECK_NULL_PTR(config->bus_name, s_tag, "bus_name is nullptr");
01194
01195     if (handle->initialized) {
01196         rx_log_warn(s_tag, "DS18B20 already initialized");
01197         return k_rx_err_invalid_state;
01198     }
01199
01200     if (config->resolution > k_ds18b20_resolution_12bit) {
01201         rx_log_error(s_tag, "Invalid resolution setting");
01202         return k_rx_err_invalid_arg;
01203     }
01204
01205     if ((int)config->use_rom_matching && config->rom[0] != s_ds18b20_family_code) {
01206         rx_log_error(s_tag, "Invalid DS18B20 family code");
01207         return k_rx_err_invalid_arg;
01208     }
01209
01210     return k_rx_ok;
01211 }
01212
01213 /**
01214  * @brief Verify DS18B20 device presence and establish communication
01215  *
01216  * @details
01217  * Three-step verification process to ensure sensor is physically connected
01218  * and responding correctly before completing initialization.
01219  *
01220  * **Algorithm:**
01221  * ```
01222  * @startuml
01223  * start
01224  * :Initialize OneWire bus;
01225  * if (Bus init successful?) then (yes)
01226  *   :Send reset pulse;
01227  *   if (Presence pulse detected?) then (yes)
01228  *     :Read scratchpad (with CRC);
01229  *     if (Scratchpad valid?) then (yes)
01230  *       :Return k_rx_ok;
```

```
01231 *       stop
01232 *     else (CRC fail)
01233 *       :Return error;
01234 *     endif
01235 *   else (no presence)
01236 *     :Log "device not present";
01237 *     :Return k_rx_err_invalid_state;
01238 *     stop
01239 *   endif
01240 * else (bus init fail)
01241 *   :Return error;
01242 * endif
01243 * @enduml
01244 * ```
01245 *
01246 * **Verification Steps:**
01247 * 1. **Bus Initialization** - Configure GPIO for 1-Wire timing
01248 * 2. **Presence Detection** - Send reset, check for presence pulse
01249 * 3. **Communication Test** - Read scratchpad with CRC validation
01250 *
01251 * **Timing:**
01252 * - Reset pulse: 480-960 us LOW
01253 * - Presence detect: 60-240 us after reset
01254 * - Scratchpad read: ~5ms (9 bytes + CRC)
01255 *
01256 *
01257 *
01258 * @pre handle->bus_manager is valid
01259 * @pre handle->bus_name is valid
01260 * @post If k_rx_ok, device is ready for configuration
01261 *
01262 * @note This function is called during rx_ds18b20_init()
01263 * @warning Blocks for ~100ms total (bus init + scratchpad read)
01264 *
01265 * @par Performance:
01266 * Execution time: ~100ms (bus init 50ms + reset 1ms + scratchpad 5ms + margin)
01267 *
01268 * @see rx_bus_onewire_init()
01269 * @see rx_bus_onewire_reset()
01270 * @see internal_ds18b20_read_scratchpad_raw()
01271 */
01272 static rx_err_t internal_ds18b20_verify_device_presence(rx_ds18b20_handle_t* handle)
01273 {
01274   /* Initialize OneWire bus */
01275   rx_err_t err = rx_bus_onewire_init(handle->bus_manager, handle->bus_name);
01276   if (err != k_rx_ok) {
01277     rx_log_error(s_tag, "Failed to initialize OneWire bus");
01278     return err;
01279   }
01280
01281   /* Check device presence */
01282   bool presence = false;
01283   err           = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01284   if (err != k_rx_ok) {
01285     rx_log_error(s_tag, "OneWire reset failed");
01286     return err;
01287   }
01288
01289   if (!presence) {
01290     rx_log_error(s_tag, "DS18B20 device not present on bus");
01291     return k_rx_err_invalid_state;
01292   }
01293
01294   /* Read scratchpad to verify communication */
01295   uint8_t scratchpad[k_ds18b20_scratchpad_bytes];
01296   err = internal_ds18b20_read_scratchpad_raw(handle, scratchpad);
01297   if (err != k_rx_ok) {
01298     rx_log_error(s_tag, "Failed to read scratchpad during init");
01299     return err;
01300   }
01301
01302   return k_rx_ok;
01303 }
01304
01305 /**
01306 * @brief Select DS18B20 device using ROM matching or Skip ROM command
01307 *
01308 * @details
01309 * Sends appropriate ROM command based on bus configuration:
01310 * - **Match ROM (0x55)** - Address specific device on multi-drop bus
01311 * - **Skip ROM (0xCC)** - Broadcast to single device or all devices
01312 *
01313 * **Decision Flow:**
01314 * ```
01315 * @startuml
01316 * start
01317 * if (use_rom_matching?) then (yes)
```

```
01318  *    :Send Match ROM (0x55);
01319  *    :Send 64-bit ROM code;
01320  *    note right
01321  *      Addresses specific device
01322  *      by unique ROM ID
01323  *    end note
01324  * else (no)
01325  *    :Send Skip ROM (0xCC);
01326  *    note right
01327  *      Single device: addresses only device
01328  *      Multiple devices: broadcasts to all
01329  *    end note
01330  * endif
01331  * :Device selected;
01332  * :Ready for function command;
01333  * stop
01334  * @enduml
01335  * ```
01336  *
01337  * **ROM Matching (Multi-Drop Bus):**
01338  * - Used when multiple DS18B20 sensors on one bus
01339  * - Requires 64-bit ROM code (obtained via search or label)
01340  * - Ensures commands go to intended sensor only
01341  *
01342  * **Skip ROM (Single Device):**
01343  * - Faster - no need to send 64-bit ROM
01344  * - Safe when only one device on bus
01345  * - Caution: broadcasts if multiple devices present
01346  *
01347  * **Timing:**
01348  * - Match ROM: ~3ms (1 byte command + 8 bytes ROM)
01349  * - Skip ROM: ~1ms (1 byte command)
01350  *
01351  *
01352  *
01353  * @pre handle->use_rom_matching is configured
01354  * @pre If ROM matching: handle->rom contains valid 64-bit ID
01355  * @post Selected device ready for function commands
01356  *
01357  * @note Called before every DS18B20 function command
01358  * @warning Skip ROM on multi-drop bus affects ALL devices
01359  *
01360  * @par Example: ROM Code Format
01361  * ```
01362  * Byte 0: Family Code (0x28 for DS18B20)
01363  * Byte 1-6: Serial Number (48-bit unique ID)
01364  * Byte 7: CRC-8 checksum
01365  * ```
01366  *
01367  * @see rx_bus_onewire_match_rom()
01368  * @see rx_bus_onewire_skip_rom()
01369  */
01370 static rx_err_t internal_ds18b20_select_device(const rx_ds18b20_handle_t* handle)
01371 {
01372   if (handle->use_rom_matching) {
01373     /* Use Match ROM to address specific device */
01374     rx_err_t err = rx_bus_onewire_match_rom(handle->bus_manager, handle->bus_name, handle->rom);
01375     if (err != k_rx_ok) {
01376       rx_log_error(s_tag, "Failed to send Match ROM");
01377       return err;
01378     }
01379   } else {
01380     /* Use Skip ROM for single device or broadcast */
01381     rx_err_t err = rx_bus_onewire_skip_rom(handle->bus_manager, handle->bus_name);
01382     if (err != k_rx_ok) {
01383       rx_log_error(s_tag, "Failed to send Skip ROM");
01384       return err;
01385     }
01386   }
01387
01388   return k_rx_ok;
01389 }
01390
01391 /**
01392  * @brief Read DS18B20 scratchpad memory with CRC-8 validation
01393  *
01394  * @details
01395  * Reads all 9 bytes of scratchpad memory and validates data integrity using
01396  * CRC-8/MAXIM checksum. This is the primary method for reading temperature
01397  * and configuration data from the sensor.
01398  *
01399  * **Algorithm:**
01400  * ```
01401  * @startuml
01402  * start
01403  * :Send reset pulse;
01404  * if (Presence detected?) then (yes)
```

```
01405  *   :Select device (Match/Skip ROM);
01406  *   :Send Read Scratchpad (0xBE);
01407  *   :Read 9 bytes (8 data + 1 CRC);
01408  *   :Calculate CRC-8 of first 8 bytes;
01409  *   if (CRC matches byte 8?) then (yes)
01410  *     if (Reserved byte == 0xFF?) then (yes)
01411  *       :Return k_rx_ok;
01412  *       stop
01413  *     else (reserved invalid)
01414  *       :Log warning;
01415  *       :Return k_rx_ok anyway;
01416  *       note right: Non-fatal, continue
01417  *     endif
01418  *   else (CRC mismatch)
01419  *     :Log error;
01420  *     :Return k_rx_err_crc_mismatch;
01421  *     stop
01422  *   endif
01423  * else (no presence)
01424  *   :Return k_rx_err_invalid_state;
01425  * endif
01426  * @enduml
01427  * ```
01428  *
01429  * **Scratchpad Memory Map:**
01430  * ```
01431  * +------+-------------+----------------------------+
01432  * | Byte | Field       | Description                |
01433  * +------+-------------+----------------------------+
01434  * |  0   | Temp LSB    | Temperature low byte       |
01435  * |  1   | Temp MSB    | Temperature high byte      |
01436  * |  2   | TH Register | High alarm threshold       |
01437  * |  3   | TL Register | Low alarm threshold        |
01438  * |  4   | Config      | Resolution bits (6:5)      |
01439  * |  5   | Reserved    | Always 0xFF                |
01440  * |  6   | Reserved    | Factory-programmed         |
01441  * |  7   | Reserved    | Count remain (legacy)      |
01442  * |  8   | CRC         | CRC-8/MAXIM checksum       |
01443  * +------+-------------+----------------------------+
01444  * ```
01445  *
01446  * **CRC-8/MAXIM Calculation:**
01447  * - Polynomial: 0x31 (x^8 + x^5 + x^4 + 1)
01448  * - Initial value: 0x00
01449  * - Input: First 8 bytes of scratchpad
01450  * - Result: Must match byte 8
01451  *
01452  * **Post-Conditions:**
01453  * - Scratchpad buffer contains valid sensor data
01454  * - CRC verified (data integrity guaranteed)
01455  * - Reserved byte checked (warning if unexpected)
01456  *
01457  *
01458  *
01459  * @pre handle != nullptr
01460  * @pre handle->initialized == true
01461  * @pre scratchpad != nullptr
01462  * @pre scratchpad size >= 9 bytes
01463  * @post If k_rx_ok: scratchpad contains validated sensor data
01464  * @post If error: scratchpad contents undefined
01465  *
01466  * @note Reserved byte validation is non-fatal (log warning only)
01467  * @warning Do not use scratchpad data if CRC check fails
01468  *
01469  * @par Performance:
01470  * Execution time: ~5ms (reset 1ms + select 1ms + read 3ms)
01471  *
01472  * @par Example: Manual Scratchpad Access
01473  * @code
01474  * uint8_t scratchpad[9];
01475  * rx_err_t err = internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad);
01476  * if (err == k_rx_ok) {
01477  *     int16_t temp_raw = scratchpad[0] | (scratchpad[1] « 8);
01478  *     uint8_t config = scratchpad[4];
01479  *     uint8_t resolution = (config » 5) & 0x03;
01480  * }
01481  * @endcode
01482  *
01483  * @see rx_crc8_maxim()
01484  * @see internal_ds18b20_select_device()
01485  */
01486 static rx_err_t internal_ds18b20_read_scratchpad_raw(const rx_ds18b20_handle_t* handle,
01487                                                      uint8_t scratchpad[k_ds18b20_scratchpad_bytes])
01488 {
01489     bool     presence   = false;
01490     rx_err_t err        = k_rx_ok;
01491     uint32_t crc_calc   = 0U;
```

```
01492   uint8_t  crc_device = 0;
01493
01494   /* Callers always validate handle and scratchpad before calling this internal function.
01495    * No RX_CHECK_NULL_PTR here to avoid unreachable branches. */
01496
01497   /* Reset and check presence */
01498   err = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01499   if (err != k_rx_ok || !presence) {
01500     rx_log_error(s_tag, "Device not present for scratchpad read");
01501     return k_rx_err_invalid_state;
01502   }
01503
01504   /* Select device */
01505   err = internal_ds18b20_select_device(handle);
01506   if (err != k_rx_ok) {
01507     return err;
01508   }
01509
01510   /* Send Read Scratchpad command */
01511   err =
01512     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_read_scratch);
01513   if (err != k_rx_ok) {
01514     rx_log_error(s_tag, "Failed to send read scratchpad command");
01515     return err;
01516   }
01517
01518   /* Read 9 bytes (8 data + 1 CRC) */
01519   err = rx_bus_onewire_read(handle->bus_manager,
01520                             handle->bus_name,
01521                             scratchpad,
01522                             k_ds18b20_scratchpad_bytes);
01523   if (err != k_rx_ok) {
01524     rx_log_error(s_tag, "Failed to read scratchpad data");
01525     return err;
01526   }
01527
01528   /* Validate CRC */
01529   crc_calc = 0U;
01530   /* rx_crc8_maxim always succeeds for valid inputs (pre-validated above) */
01531   (void)rx_crc8_maxim(scratchpad, k_ds18b20_crc_bytes, &crc_calc);
01532   crc_device = scratchpad[k_ds18b20_scratch_crc];
01533
01534   if ((uint8_t)crc_calc != crc_device) {
01535     rx_log_error(s_tag, "Scratchpad CRC mismatch");
01536     return k_rx_err_crc_mismatch;
01537   }
01538
01539   /* Post-condition: Validate scratchpad structure (reserved byte should be 0xFF) */
01540   if (scratchpad[k_ds18b20_scratch_reserved1] != s_ds18b20_reserved_byte_value) {
01541     rx_log_warn(s_tag, "Scratchpad reserved byte unexpected value");
01542   }
01543
01544   return k_rx_ok;
01545 }
01546
01547 /**
01548  * @brief Write DS18B20 scratchpad registers (TH, TL, Config)
01549  *
01550  * @details
01551  * Writes the three user-modifiable scratchpad registers: high alarm (TH),
01552  * low alarm (TL), and configuration (resolution). This is the only way to
01553  * change sensor resolution and alarm thresholds.
01554  *
01555  * **Algorithm:**
01556  * ```
01557  * @startuml
01558  * start
01559  * :Send reset pulse;
01560  * if (Presence detected?) then (yes)
01561  *   :Select device (Match/Skip ROM);
01562  *   :Send Write Scratchpad (0x4E);
01563  *   :Write TH byte;
01564  *   :Write TL byte;
01565  *   :Write Config byte;
01566  *   :Return k_rx_ok;
01567  *   note right
01568  *     Changes take effect immediately
01569  *     but are volatile (lost on power cycle)
01570  *   end note
01571  *   stop
01572  * else (no presence)
01573  *   :Return k_rx_err_invalid_state;
01574  * endif
01575  * @enduml
01576  * ```
01577  *
01578  * **Write Sequence:**
```

```
01579  * 1. Reset bus and check presence
01580  * 2. Select device (Match ROM or Skip ROM)
01581  * 3. Send Write Scratchpad command (0x4E)
01582  * 4. Write 3 bytes in order: TH, TL, Config
01583  *
01584  * **Configuration Register Encoding:**
01585  * ```
01586  * Bit:  7   6   5   4   3   2   1   0
01587  *      +---+---+---+---+---+---+---+---+
01588  *      | 0 |R1 |R0 | 1 | 1 | 1 | 1 | 1 |
01589  *      +---+---+---+---+---+---+---+---+
01590  *          +---+---+
01591  *        Resolution
01592  *
01593  * R1 R0 | Resolution | Conversion Time
01594  * ------+------------+---------------
01595  *  0  0 |   9-bit    |     94 ms
01596  *  0  1 |  10-bit    |    188 ms
01597  *  1  0 |  11-bit    |    375 ms
01598  *  1  1 |  12-bit    |    750 ms
01599  * ```
01600  *
01601  * **Persistence:**
01602  * - Scratchpad writes are **volatile** (RAM only)
01603  * - Lost on power cycle or reset
01604  * - Use Copy Scratchpad (0x48) to save to EEPROM
01605  *
01606  *
01607  *
01608  * @pre handle != nullptr
01609  * @pre handle->initialized == true
01610  * @pre scratchpad != nullptr
01611  * @post Sensor resolution/alarms updated (volatile, RAM only)
01612  *
01613  * @note Changes are immediate but volatile (lost on power cycle)
01614  * @warning No CRC verification - ensure bus integrity
01615  *
01616  * @par Performance:
01617  * Execution time: ~3ms (reset 1ms + select 1ms + write 1ms)
01618  *
01619  * @par Example: Change Resolution to 10-bit
01620  * @code
01621  * uint8_t scratchpad_data[9];
01622  * internal_ds18b20_read_scratchpad_raw(&sensor, scratchpad_data);
01623  *
01624  * ds18b20_scratchpad_write_t write_cfg = {
01625  *     .th = scratchpad_data[2],  // Preserve TH
01626  *     .tl = scratchpad_data[3],  // Preserve TL
01627  *     .config = (0x01 « 5) | 0x1F  // 10-bit: R1=0, R0=1
01628  * };
01629  * internal_ds18b20_write_scratchpad(&sensor, &write_cfg);
01630  *
01631  * // Save to EEPROM to make permanent
01632  * rx_ds18b20_save_config(&sensor);
01633  * @endcode
01634  *
01635  * @see rx_ds18b20_save_config() Make changes permanent
01636  * @see internal_ds18b20_select_device()
01637  * @see ds18b20_scratchpad_write_t
01638  */
01639 static rx_err_t internal_ds18b20_write_scratchpad(const rx_ds18b20_handle_t*         handle,
01640                                                   const ds18b20_scratchpad_write_t* scratchpad)
01641 {
01642   /* Callers always validate handle and scratchpad before calling this internal function.
01643    * No RX_CHECK_NULL_PTR here to avoid unreachable branches. */
01644
01645   /* Reset and check presence */
01646   bool      presence = false;
01647   rx_err_t err       = rx_bus_onewire_reset(handle->bus_manager, handle->bus_name, &presence);
01648   if (err != k_rx_ok || !presence) {
01649     rx_log_error(s_tag, "Device not present for scratchpad write");
01650     return k_rx_err_invalid_state;
01651   }
01652
01653   /* Select device */
01654   err = internal_ds18b20_select_device(handle);
01655   if (err != k_rx_ok) {
01656     return err;
01657   }
01658
01659   /* Send Write Scratchpad command */
01660   err =
01661     rx_bus_onewire_write_byte(handle->bus_manager, handle->bus_name, k_ds18b20_cmd_write_scratch);
01662   if (err != k_rx_ok) {
01663     rx_log_error(s_tag, "Failed to send write scratchpad command");
01664     return err;
01665   }
```

```
01666
01667   /* Write 3 bytes (TH, TL, Config) */
01668   uint8_t write_buf[k_ds18b20_scratchpad_write_bytes];
01669   write_buf[k_ds18b20_write_idx_th]     = scratchpad->th;
01670   write_buf[k_ds18b20_write_idx_tl]     = scratchpad->tl;
01671   write_buf[k_ds18b20_write_idx_config] = scratchpad->config;
01672
01673   err = rx_bus_onewire_write(handle->bus_manager,
01674                              handle->bus_name,
01675                              write_buf,
01676                              k_ds18b20_scratchpad_write_bytes);
01677   if (err != k_rx_ok) {
01678     rx_log_error(s_tag, "Failed to write scratchpad data");
01679     return err;
01680   }
01681
01682   return k_rx_ok;
01683 }
01684
01685 /**
01686  * @brief Convert resolution enum to config register value
01687  *
01688  *
01689  */
01690 static uint8_t internal_ds18b20_resolution_to_config(const ds18b20_resolution_t resolution)
01691 {
01692   /* Resolution bits are R1:R0 at bits 6:5 */
01693   const uint8_t config = (uint8_t)resolution « k_ds18b20_config_r0_bit;
01694
01695   return config;
01696 }
01697
01698 /**
01699  * @brief Get temperature mask for resolution
01700  *
01701  * Lower bits are undefined for resolutions < 12-bit and must be masked.
01702  *
01703  *
01704  */
01705 static uint16_t internal_ds18b20_get_temp_mask(const ds18b20_resolution_t resolution)
01706 {
01707   switch (resolution) {
01708     case k_ds18b20_resolution_9bit:
01709       return k_ds18b20_temp_mask_9bit;
01710     case k_ds18b20_resolution_10bit:
01711       return k_ds18b20_temp_mask_10bit;
01712     case k_ds18b20_resolution_11bit:
01713       return k_ds18b20_temp_mask_11bit;
01714     case k_ds18b20_resolution_12bit: /* fall through */
01715     default:
01716       return k_ds18b20_temp_mask_12bit;
01717   }
01718 }
01719
01720 /**
01721  * @brief Convert raw temperature to Celsius
01722  *
01723  * DS18B20 stores temperature as 16-bit signed value in 1/16degC units.
01724  * Divide by 16 to get Celsius.
01725  *
01726  *
01727  */
01728 static float internal_ds18b20_raw_to_celsius(const int16_t raw_temp)
01729 {
01730   /* Pre-condition: caller validates range; no RX_ASSERT to avoid unreachable false branch */
01731   /* Convert from 1/16degC to degC */
01732   float result = (float)raw_temp / s_temp_conversion_divisor;
01733
01734   /* Post-condition: result is always finite for finite integer input; no RX_ASSERT needed */
01735
01736   return result;
01737 }
```